SHUBHAM DHENGALE - 3574

Experiment No. 1

Title: Design a Responsive Registration Form using Bootstrap and Perform Client-Side Validation using JavaScript

Aim:

To design a responsive buyer registration form for an Indian spice-export business using Bootstrap and to validate the entered details in the browser with JavaScript.

Tools Used:

HTML5

Bootstrap CSS

JavaScript (ES6)

Visual Studio Code

Google Chrome

Methodology:

The registration workflow was planned around the information an international spice buyer needs to share: name, company, business email, country, spice of interest and password confirmation. A mobile-first Bootstrap layout groups these fields into a clear form and keeps labels and controls readable at narrow widths. HTML5 required and email constraints handle common input errors, while JavaScript checks password matching and intercepts submission to show immediate feedback. The process was verified with an empty form and completed fields in the browser. At this stage the form is a client-side demonstration: validated information remains in the browser and is not written to the business database.

Theory:

A responsive form adjusts its layout to the available screen width. Bootstrap provides a grid system and reusable form classes such as container, row, col, form-label, and form-control. These classes help keep labels, fields, and buttons consistent across screen sizes.

HTML form controls support built-in constraints such as required, type=email, and minlength. The browser exposes the result through the Constraint Validation API. JavaScript can add project-specific checks, such as confirming that the password and confirmation fields match.

Client-side validation gives a user immediate feedback before submission. It improves usability but does not replace server-side validation. A production application must validate submitted data again on the server before storing it.

Procedure:

1. Open the spice-export project and identify the Buyer Registration route and BuyerForm component.

2. Group the buyer fields into a semantic form with labels, input types and a clear submit button.

3. Use Bootstrap spacing, grid and form-control classes so fields remain readable on phones and desktop screens.

4. Mark name, company, email, country, spice and password fields as required; use type=email and a suitable password minimum length.

5. On form submission, prevent a full-page reload and compare the password with its confirmation.

6. Call the browser validity checks and show field-level or summary feedback for invalid values.

7. Enter valid sample buyer details and confirm that the local success state appears.

8. Repeat with the form empty and with an invalid email or mismatched password; correct each error.

9. Resize the browser and capture the initial, validation and successful form states.

10. Check keyboard navigation, field focus and readable spacing at a narrow mobile viewport.

11. Record which feedback is browser validation and which message is produced by the form script.

Code:

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Buyer Registration | Spice Export</title>
<link rel="stylesheet" href="bootstrap.min.css">
</head>
<body>
<main class="container py-5">
<div class="row justify-content-center">
<section class="col-12 col-md-9 col-lg-7">
<h1 class="mb-2">Buyer Registration</h1>
<p>Register your interest in sourcing Indian spices.</p>
<form id="buyerForm" class="needs-validation" novalidate>
<label class="form-label" for="buyerName">Full name</label>
<input class="form-control mb-3" id="buyerName" required>
<label class="form-label" for="company">Company</label>
<input class="form-control mb-3" id="company" required>
<label class="form-label" for="email">Business email</label>
<input class="form-control mb-3" id="email" type="email" required>
<label class="form-label" for="country">Country</label>
<input class="form-control mb-3" id="country" required>
<label class="form-label" for="product">Product interest</label>
<select class="form-select mb-3" id="product" required>
<option value="">Choose a spice</option>
<option>Turmeric</option><option>Black pepper</option>
<option>Cardamom</option><option>Chilli</option>
</select>
```

```
<label class="form-label" for="password">Create password</label>
<input class="form-control mb-3" id="password" type="password"
minlength="8" required>
<label class="form-label" for="confirmPassword">Confirm password</label>
<input class="form-control mb-3" id="confirmPassword"
type="password" required>
<button class="btn btn-success" type="submit">Register interest</button>
<p id="formMessage" class="mt-3" role="status"></p>
</form>
</section>
</div>
</main>
<script>
const form = document.getElementById("buyerForm");
const password = document.getElementById("password");
const confirmPassword = document.getElementById("confirmPassword");
const message = document.getElementById("formMessage");
form.addEventListener("submit", (event) => {
event.preventDefault();
confirmPassword.setCustomValidity(
password.value === confirmPassword.value ? "" : "Passwords must match."
);
form.classList.add("was-validated");
if (!form.checkValidity()) {
message.textContent = "Please correct the highlighted fields.";
return;
}
message.textContent = "Details are valid and ready to be submitted.";
});
</script>
</body>
</html>
```

Output:

Buyer Registration displays name, company, business email, country, product interest, password and confirmation fields.

Invalid or missing values trigger browser validation feedback. Matching valid values display a local confirmation; this frontend experiment does not store or transmit the form.

Figure 1.1: Blank registration form before entry.

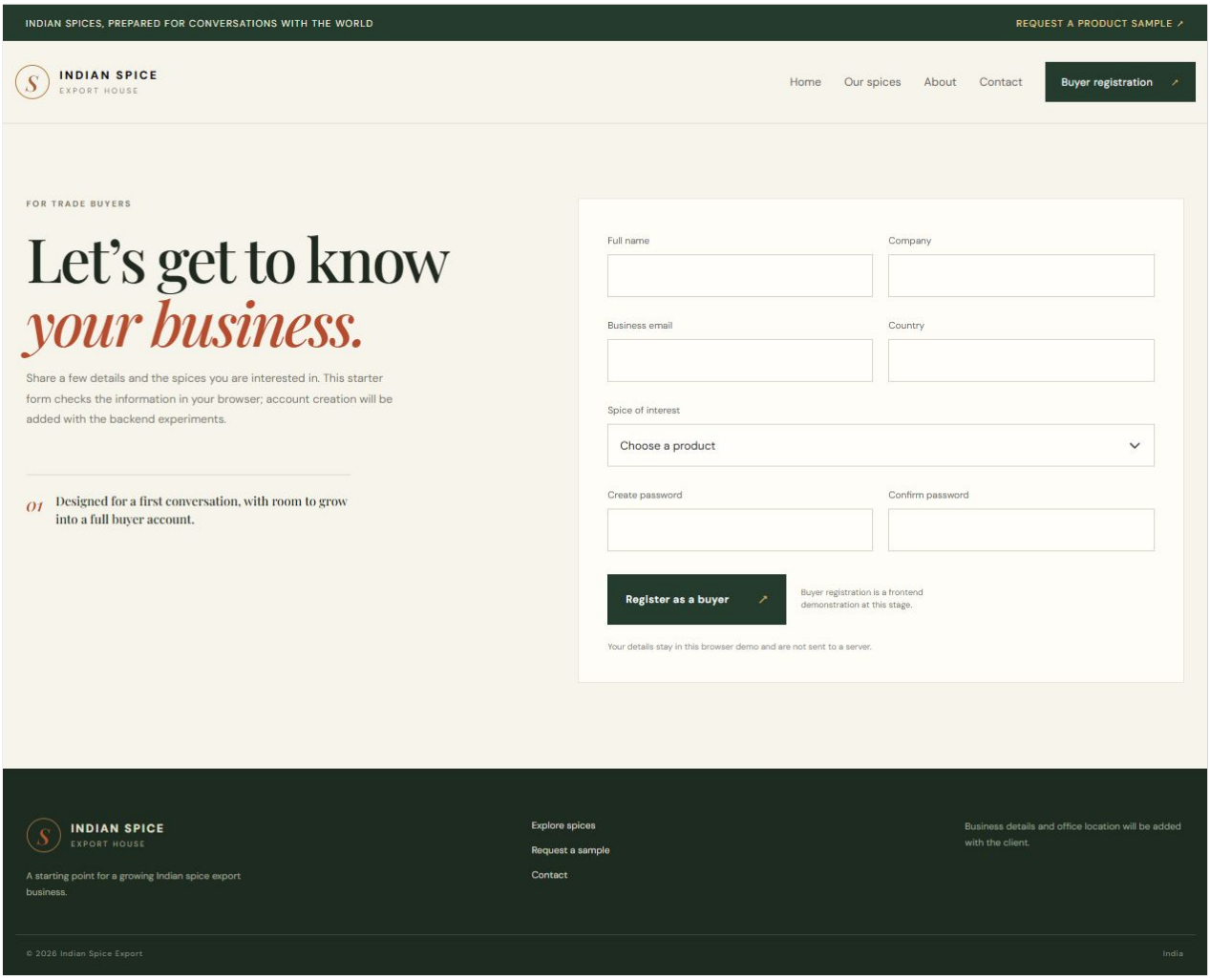


Figure 1.2: Required-field validation after an empty submission.

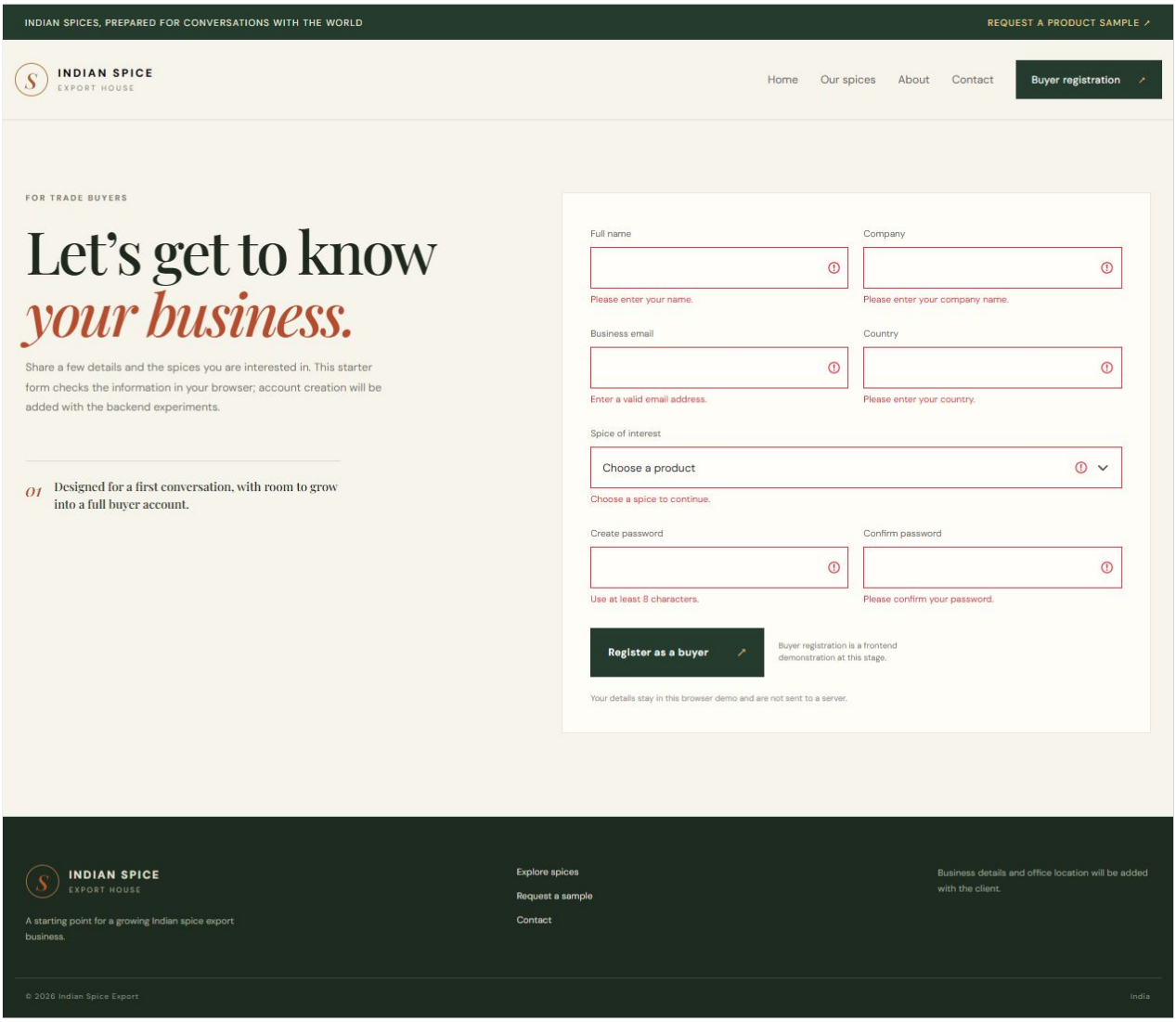


Figure 1.3: Email-format validation after an invalid address.

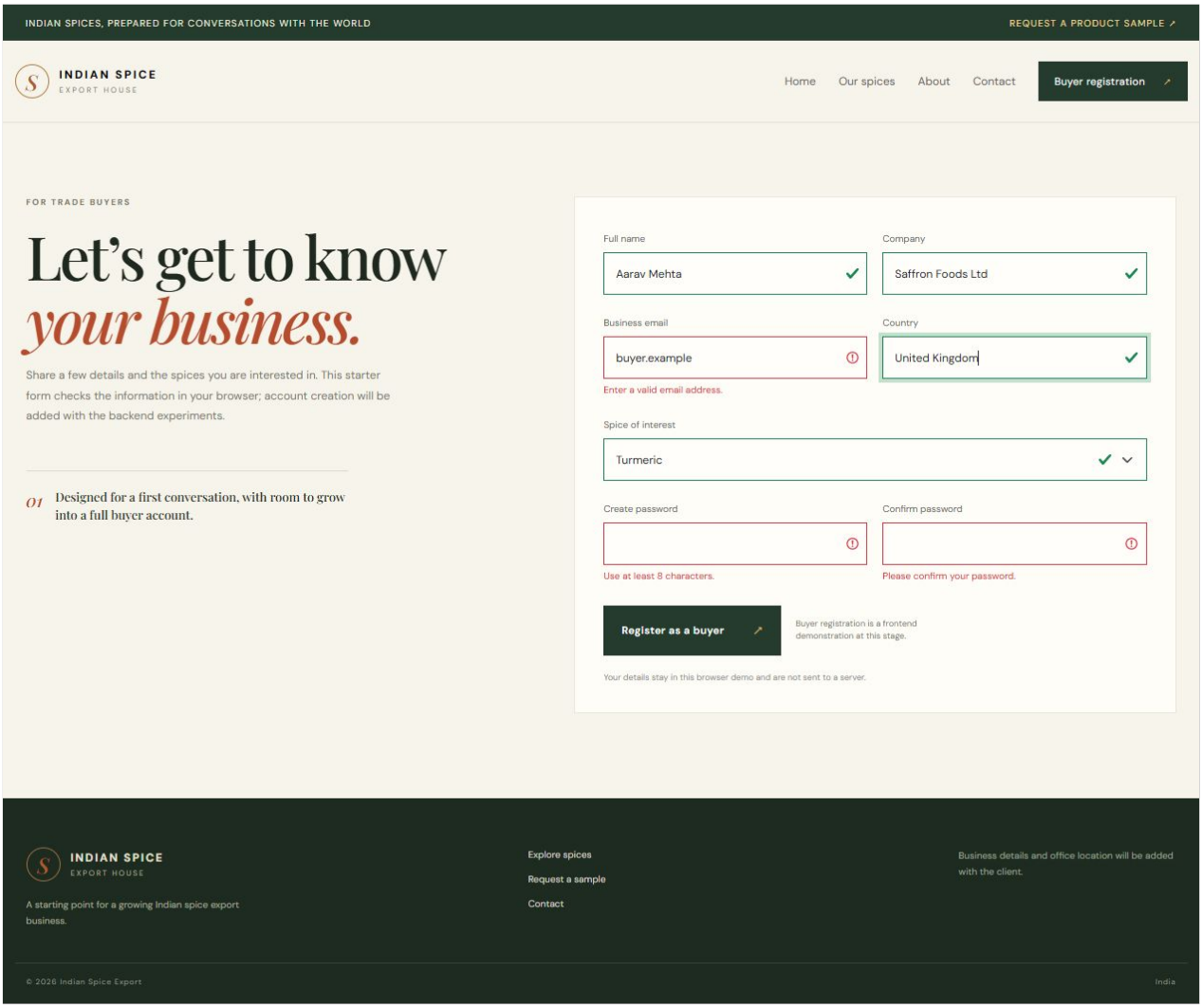


Figure 1.4: Buyer details entered in the registration form.

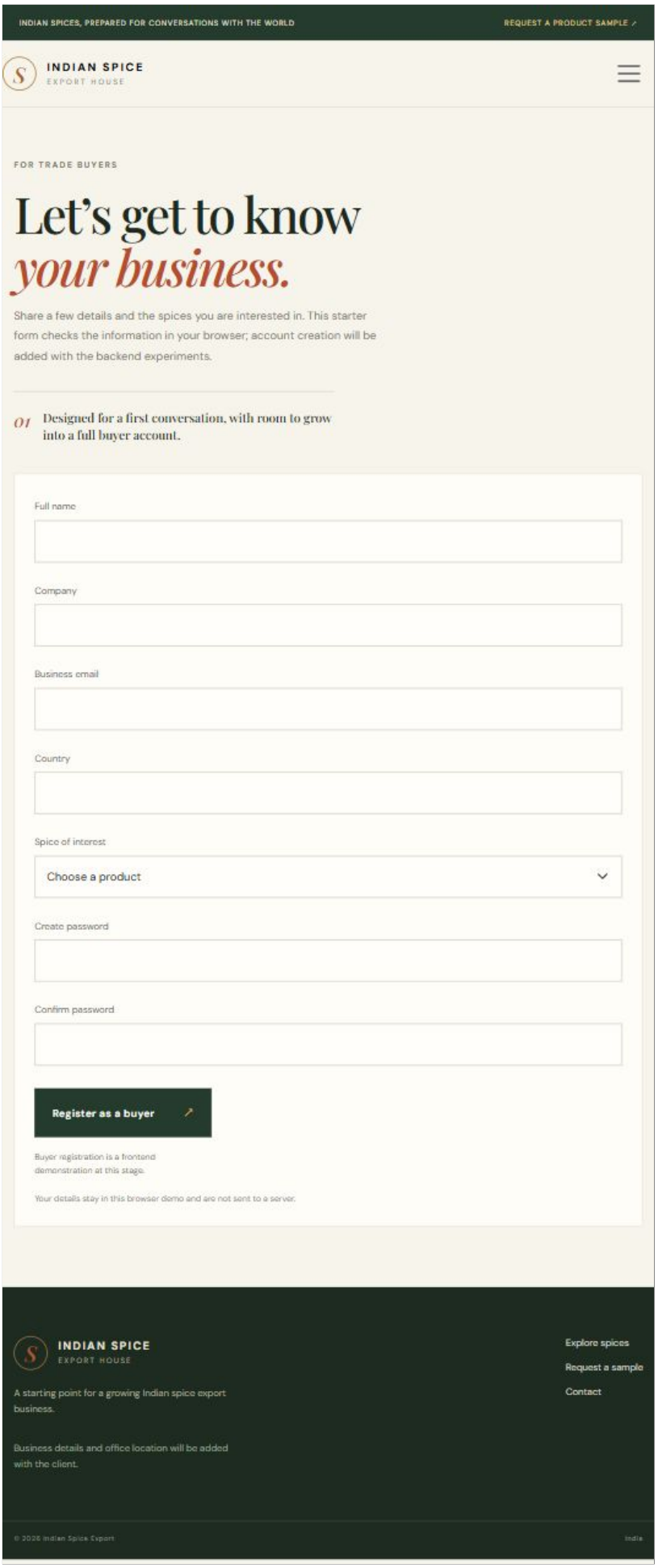


Figure 1.5: Local success state for valid buyer details.

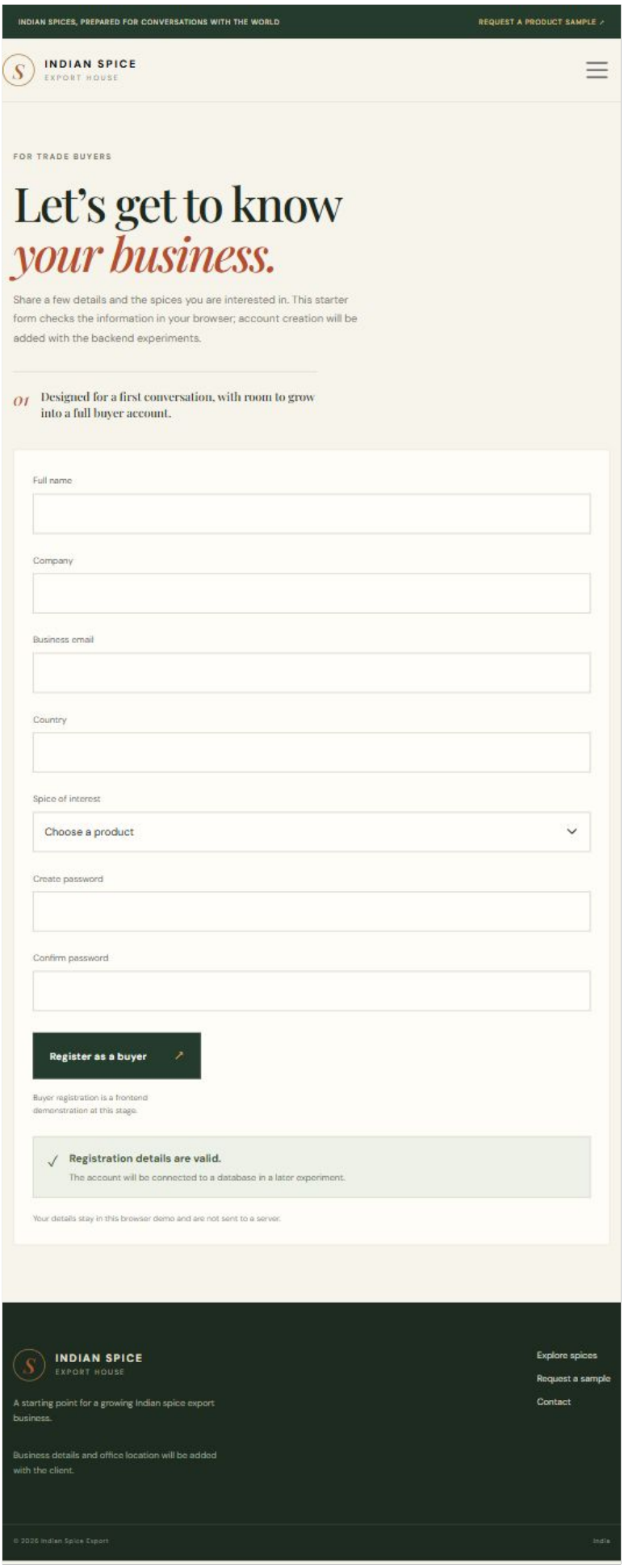


Figure 1.6: Desktop validation feedback.

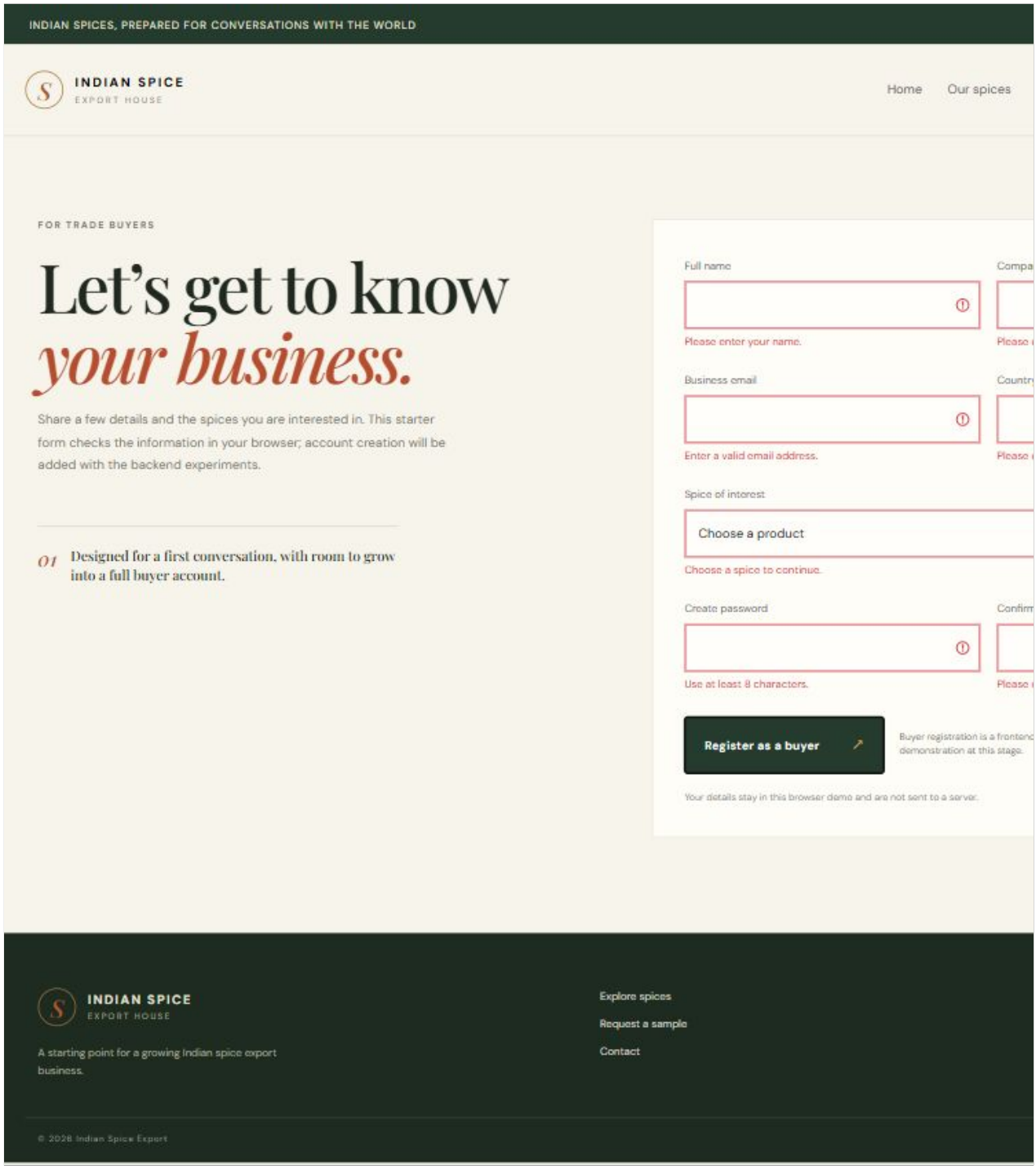


Figure 1.7: Responsive desktop registration layout.

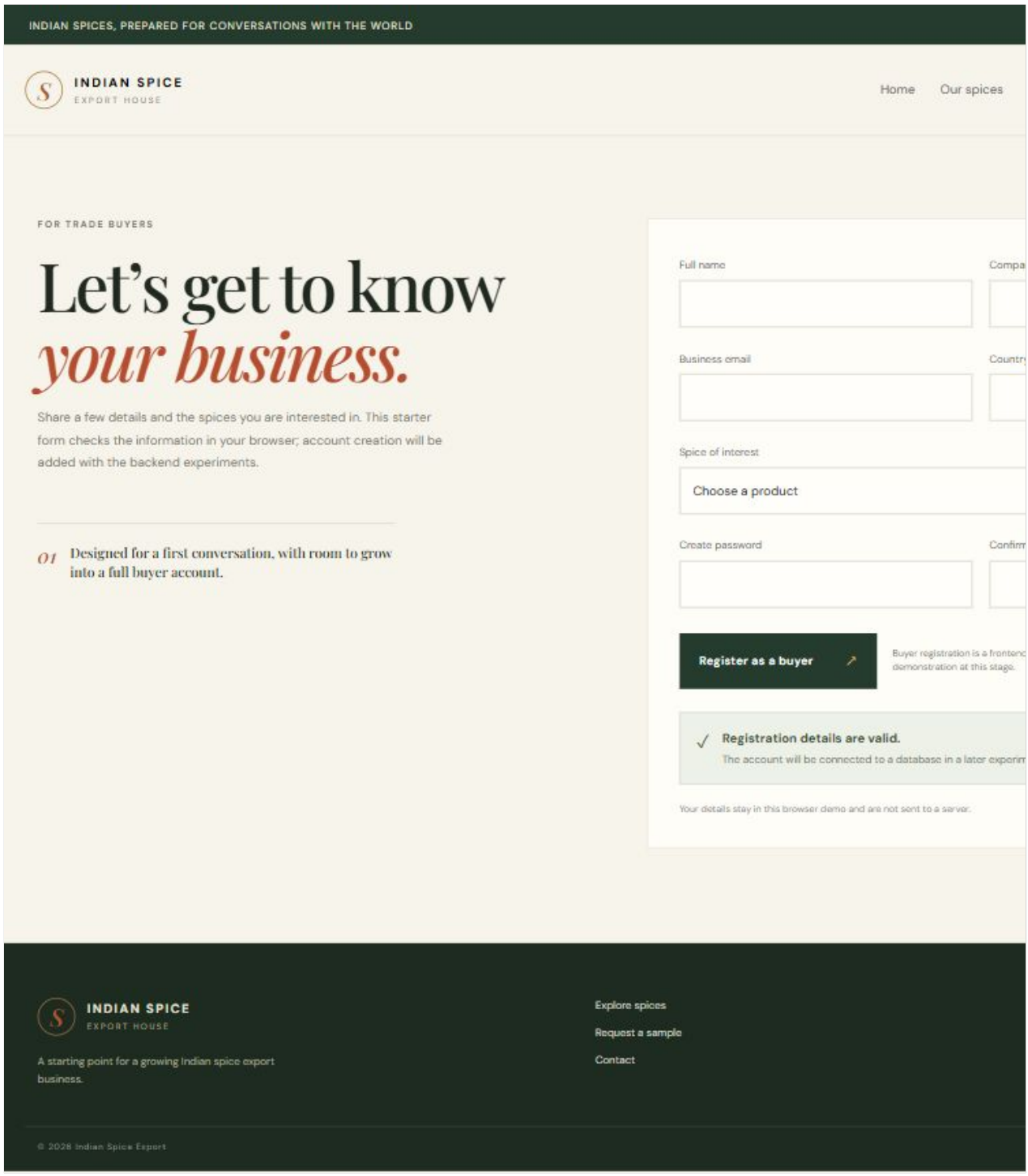
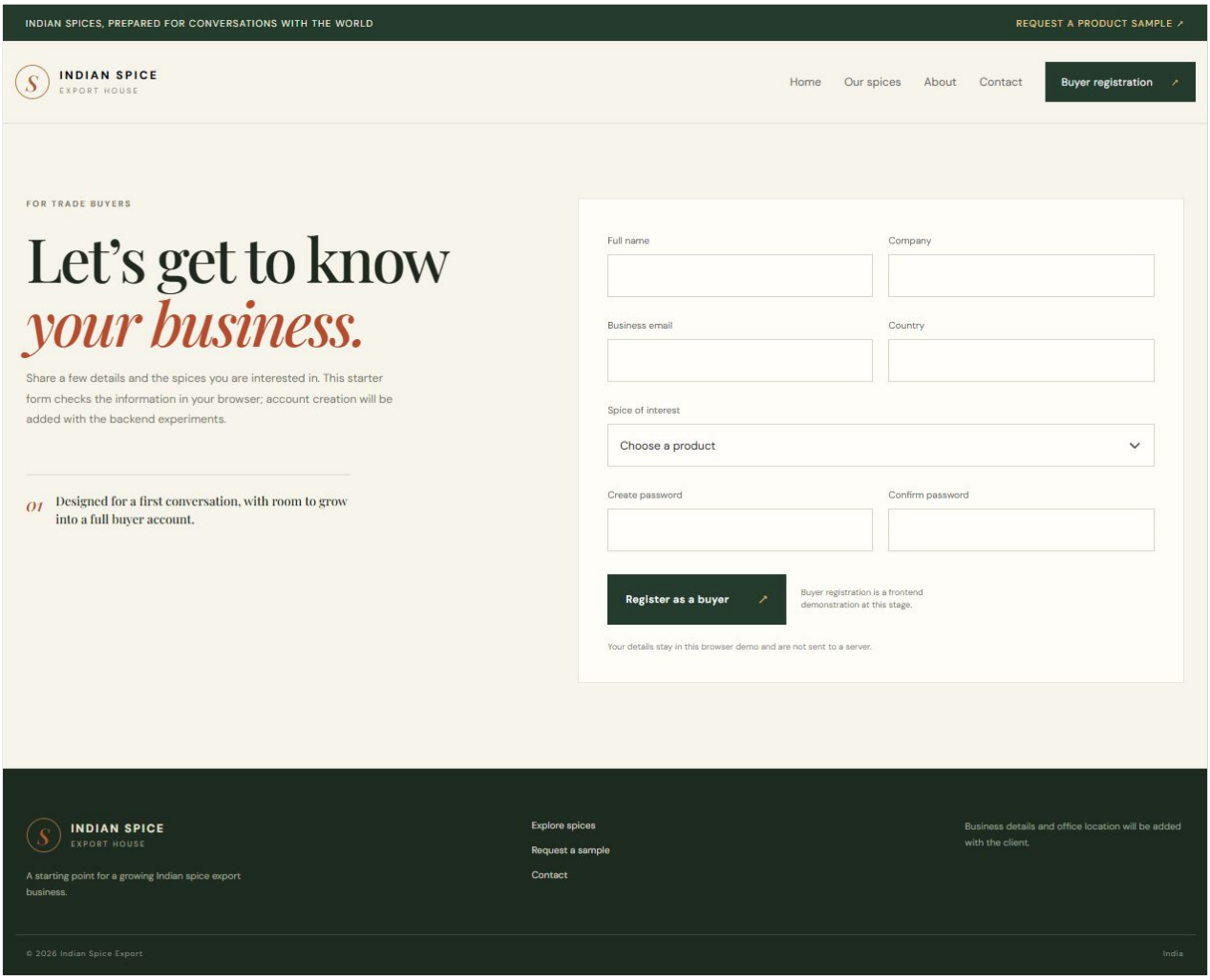


Figure 1.8: Final registration route in the local website.



Conclusion:

This experiment produced a responsive buyer-registration interface for the spice-export website and demonstrated how Bootstrap and JavaScript work together during form entry. Required fields and the browser email constraint prevent common input mistakes, while custom password matching demonstrates application-specific validation. The invalid and valid browser states give the buyer immediate feedback and keep the page from reloading during the demonstration. The layout was checked as a website screen at different widths. Because no persistence endpoint is connected in this experiment, the success message confirms only client-side validation; it does not create an account or send the buyer's details to the client. A later backend step can add secure server validation and storage.

The registration form is intentionally designed as the entry point for the buyer journey: a trade buyer records a company and product interest before requesting export information. Before production use, the server must repeat all checks, protect credentials with secure password hashing, and save the buyer record only after successful validation. The current screenshots establish the interface and browser rules so later backend work can be compared against a clear baseline.

SHUBHAM DHENGALE - 3574

Experiment No. 2

Title: Develop JavaScript Programs using ES6 Features (Arrow Functions, Anonymous Functions, Arrays, Events, and Popup Boxes)

Aim:

To demonstrate ES6 functions, arrays, array methods, event handling, and popup boxes using spice products and sample requests from the export website.

Tools Used:

JavaScript (ES6)

HTML5

Visual Studio Code

Google Chrome Developer Console

Methodology:

The spice website catalogue provides realistic data for practicing modern JavaScript. Product records are stored as an array of objects, then processed with arrow functions and array callbacks. A search input and category buttons raise DOM events; filter() selects matching products, map() formats the visible cards, and reduce() can summarize catalogue information such as available sample counts. Anonymous functions are used as event callbacks where an inline action is useful. Browser alert, confirm and prompt examples are attached to clear sample-request interactions. Each operation is checked in the rendered website so the data operation and its visible result can be related.

Theory:

Arrow functions provide concise function syntax and are commonly used as callbacks. Anonymous functions have no declared name and can be passed directly to methods such as forEach() or addEventListener().

Arrays store ordered collections. map() creates a transformed array, filter() selects elements that match a condition, and reduce() combines array values into one result. These methods are useful for creating catalogue views and calculating simple summaries.

Events let a page respond to actions such as clicking a button or submitting a form. The browser's alert() and confirm() functions display simple popup dialogs. Popup functions are suitable for demonstrations; a finished website can replace them with accessible in-page messages.

Procedure:

1. Open the products route and inspect the local spice array, including names, categories, origins and sample availability.

2. Create small arrow functions for formatting product names or availability labels.

3. Use map() to transform each matching product into a catalogue card.

4. Use filter() to match a search phrase and the selected category.

5. Use reduce() to calculate a simple total, such as the number of products with samples available.

6. Attach input and click listeners to the search field, category chips and sample buttons.

7. Use anonymous callbacks for concise event actions and keep the selected product in the message.

8. Demonstrate alert(), confirm() and prompt() with a non-destructive sample enquiry flow.

9. Test a matching search, a no-match search, each category and the reset-to-All action; capture the output states.

10. Combine a text search with a category filter and check that the catalogue remains understandable.

11. Follow a product sample link and verify that it opens the sample-request route with the selected item.

Code:

```
  // Add these controls to the product page:
// <input id="search" placeholder="Search spices">
// <ul id="spiceList"></ul>
// <button id="sampleBtn">Request a sample</button>
const spices = [
{ name: "Turmeric", origin: "India", sampleAvailable: true },
{ name: "Black pepper", origin: "India", sampleAvailable: true },
{ name: "Cardamom", origin: "India", sampleAvailable: false },
{ name: "Red chilli", origin: "India", sampleAvailable: true }
];
const list = document.getElementById("spiceList");
const search = document.getElementById("search");
const renderSpices = (items) => {
list.innerHTML = items.map((spice) =>
`<li>${spice.name} — ${spice.origin}</li>`
).join("");
};
// Anonymous function callback for each catalogue entry.
spices.forEach(function (spice) {
console.log(`${spice.name}: sample available = ${spice.sampleAvailable}`);
});
const availableCount = spices.reduce(
(count, spice) => count + Number(spice.sampleAvailable), 0
);
console.log("Samples available:", availableCount);
search.addEventListener("input", (event) => {
const query = event.target.value.toLowerCase();
const matches = spices.filter((spice) =>
spice.name.toLowerCase().includes(query)
);
```

```
renderSpices(matches);
});
document.getElementById("sampleBtn").addEventListener("click", () => {
const selected = spices.find((spice) => spice.sampleAvailable);
if (selected && confirm(`Request a ${selected.name} sample?`)) {
alert(`Your ${selected.name} sample request is ready to submit.`);
}
});
renderSpices(spices);
```

Output:

The products view displays spice cards from the local catalogue. Typing in search filters the visible items; clearing the query restores the catalogue.

Request Sample opens the confirmation interaction for the selected spice. The data and action are local to the frontend.

Figure 2.1: Full product catalogue route.

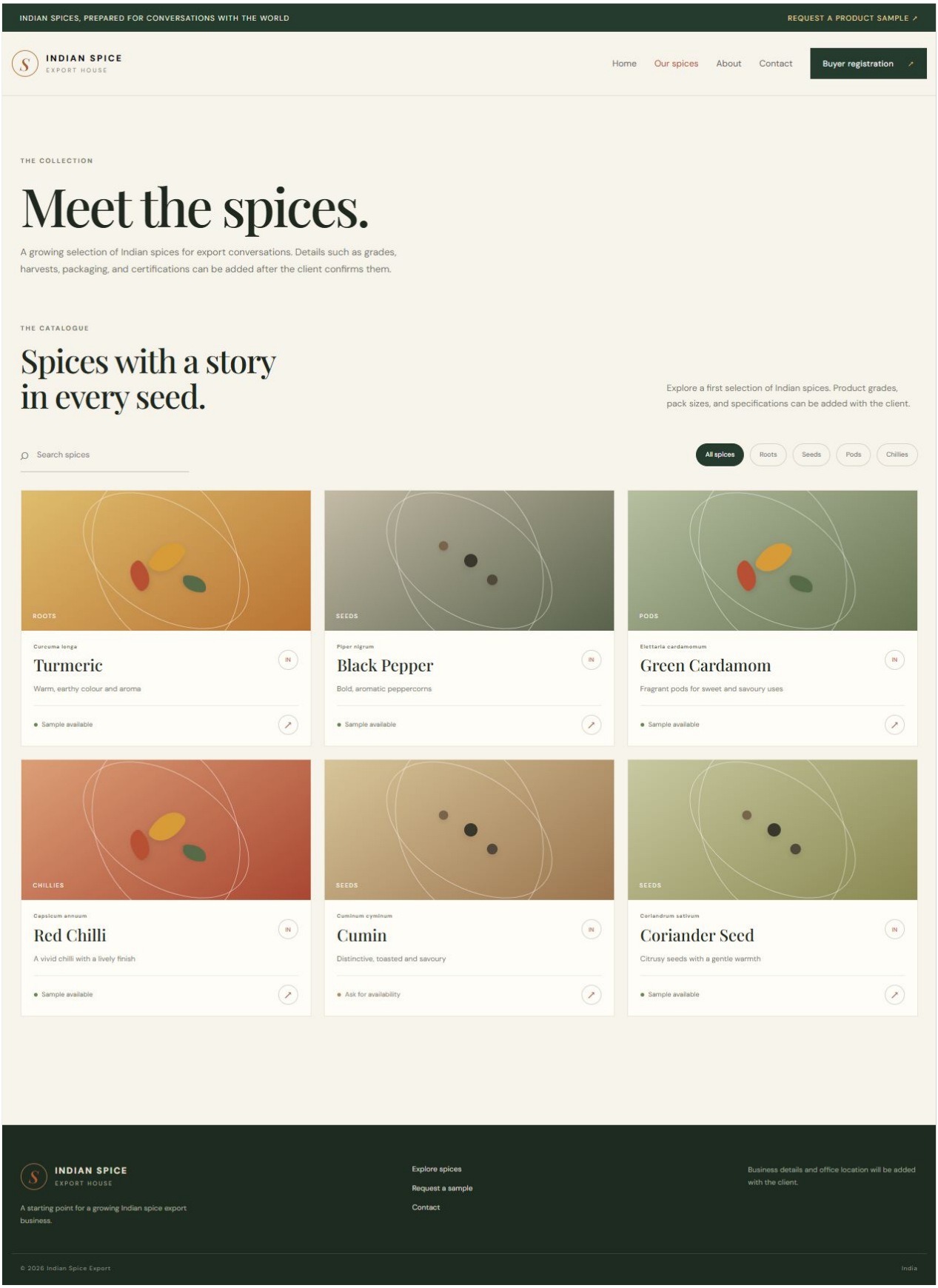


Figure 2.2: Catalogue with spice product cards.

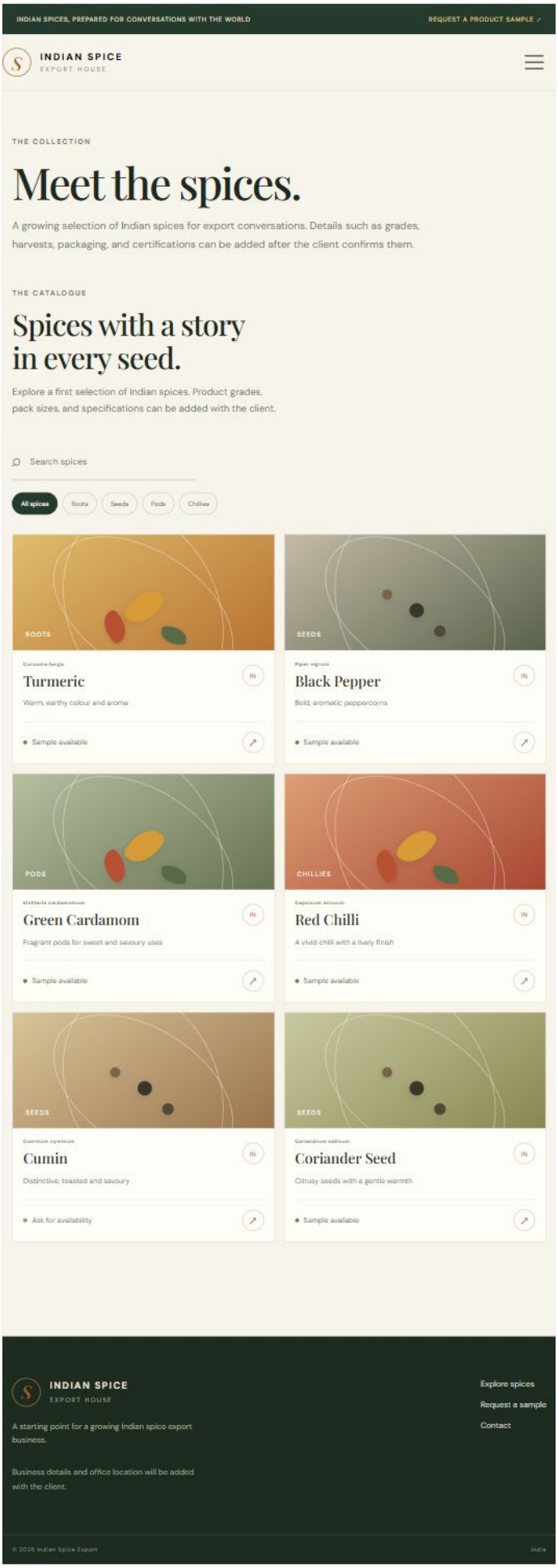


Figure 2.3: Desktop catalogue layout.

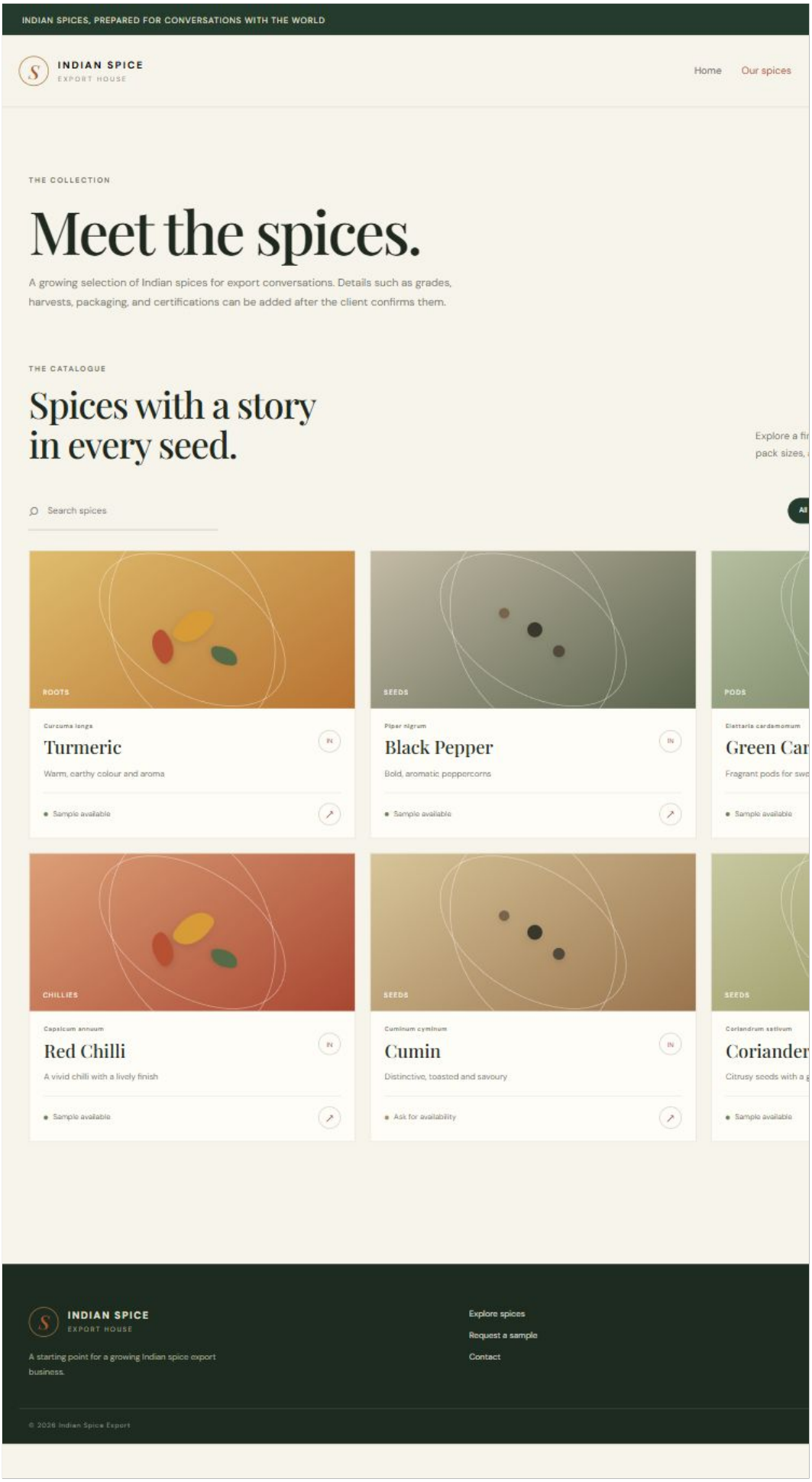


Figure 2.4: Catalogue filtered to the Roots category.

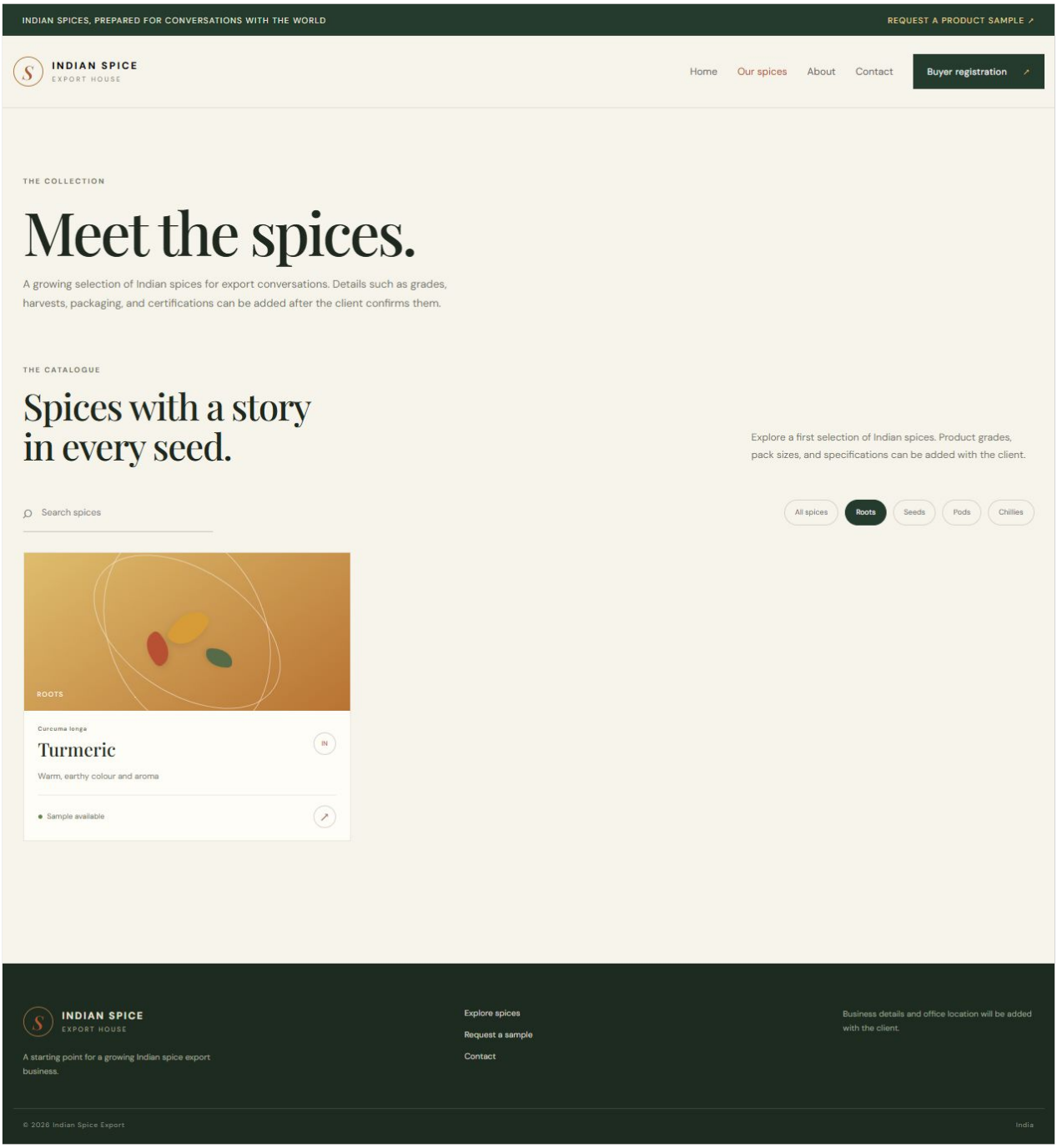


Figure 2.5: Catalogue search narrowed to Cumin.

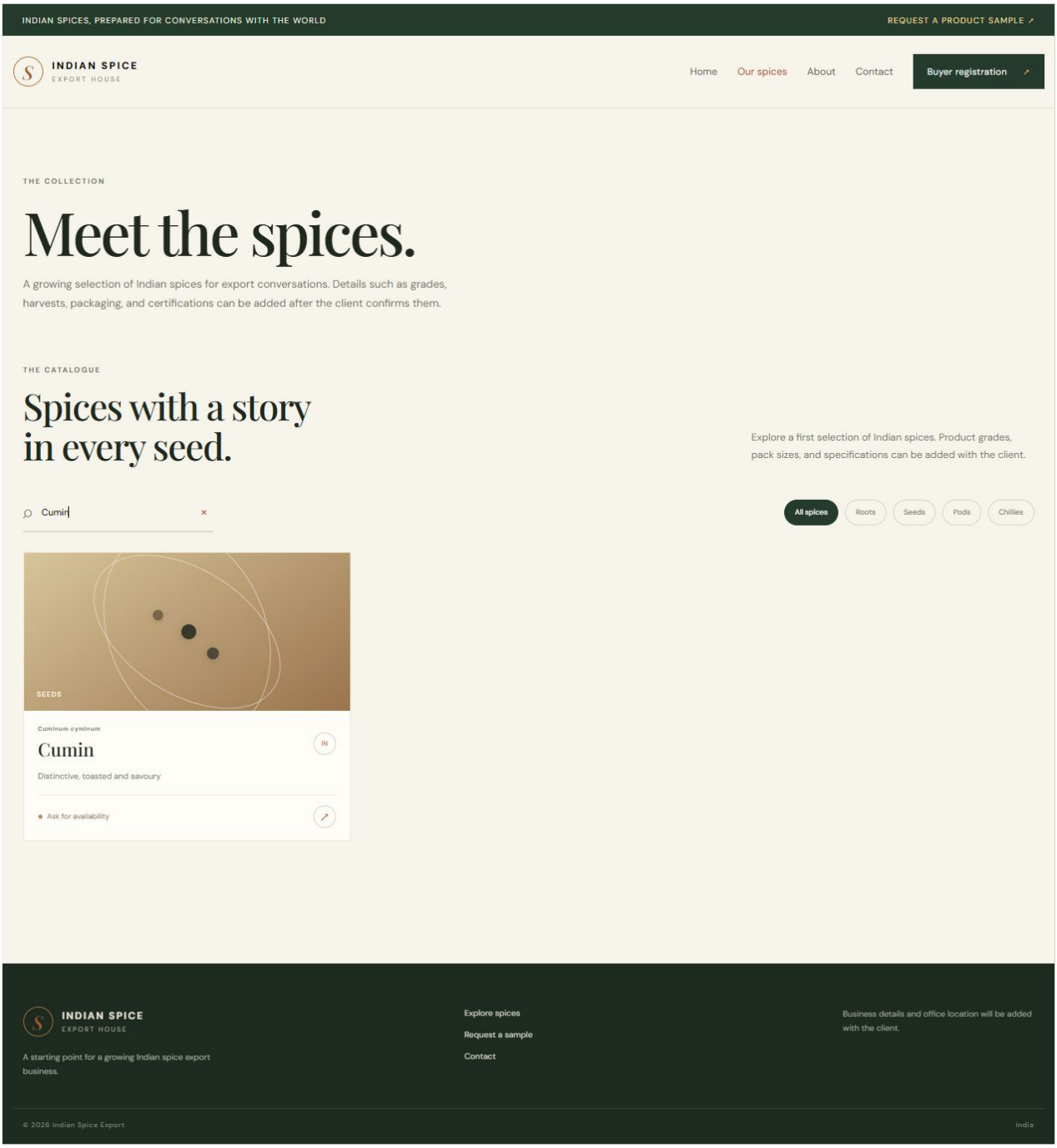


Figure 2.6: Website home page linked to the catalogue.

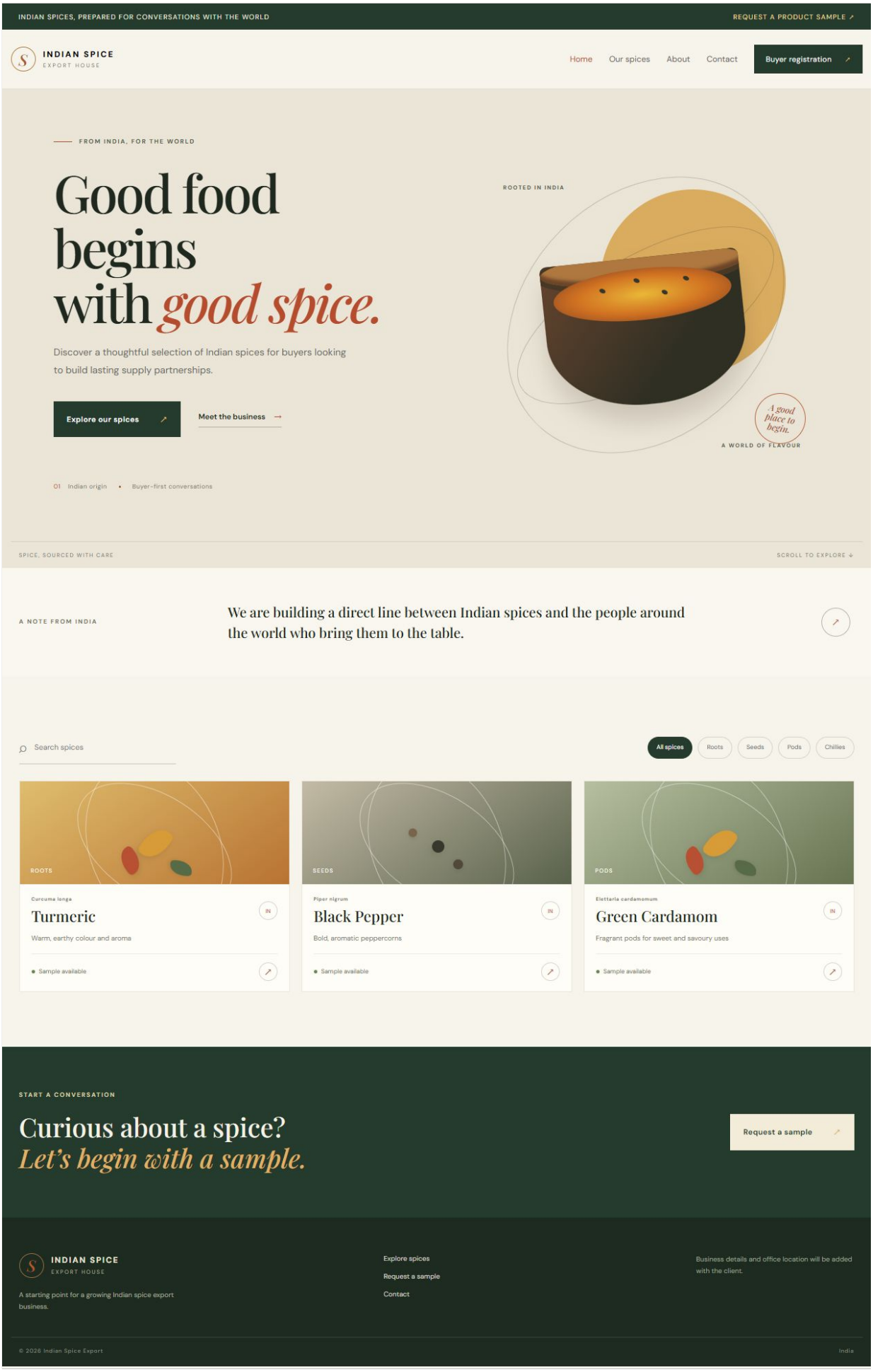


Figure 2.7: About route in the local website.

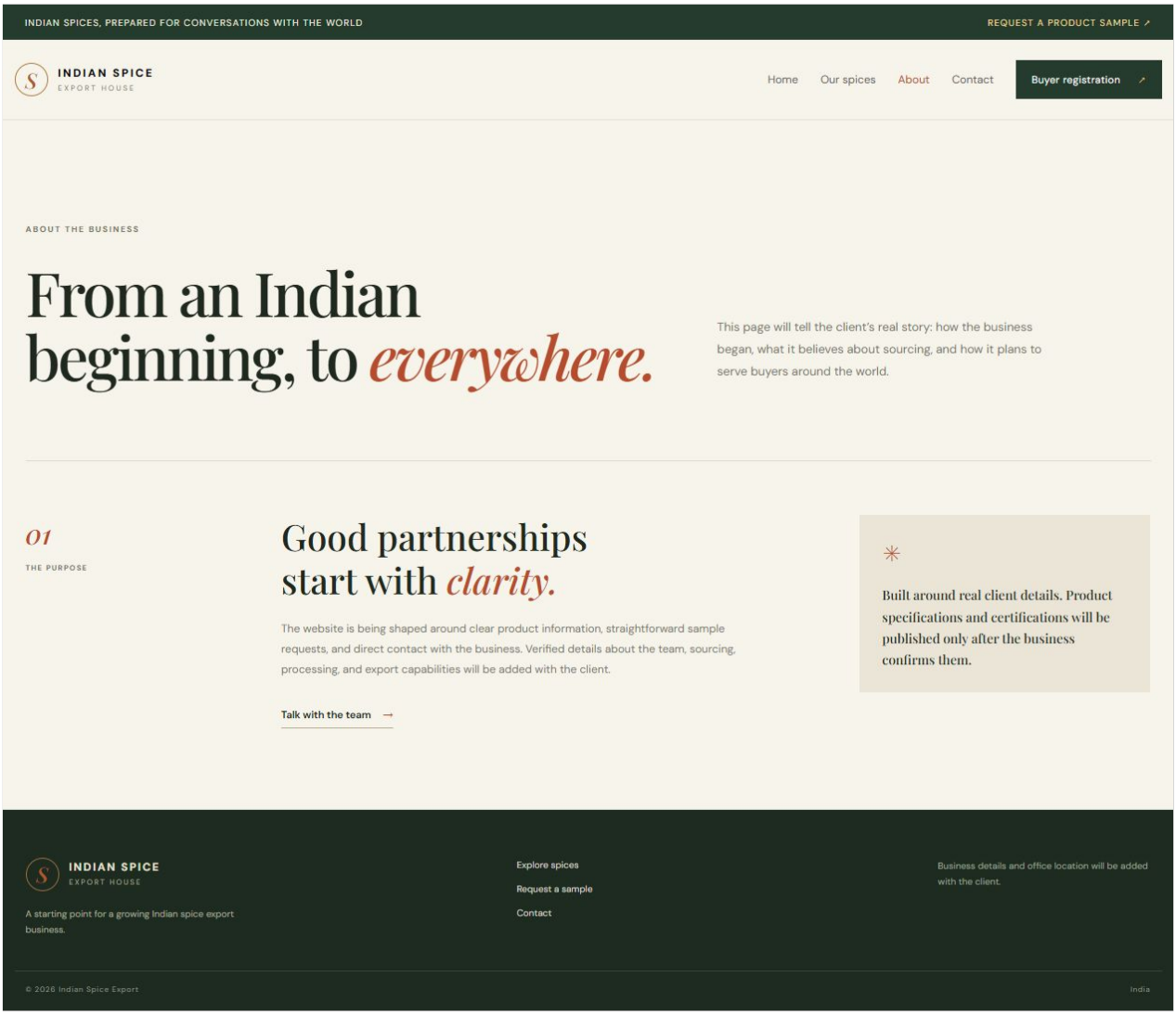
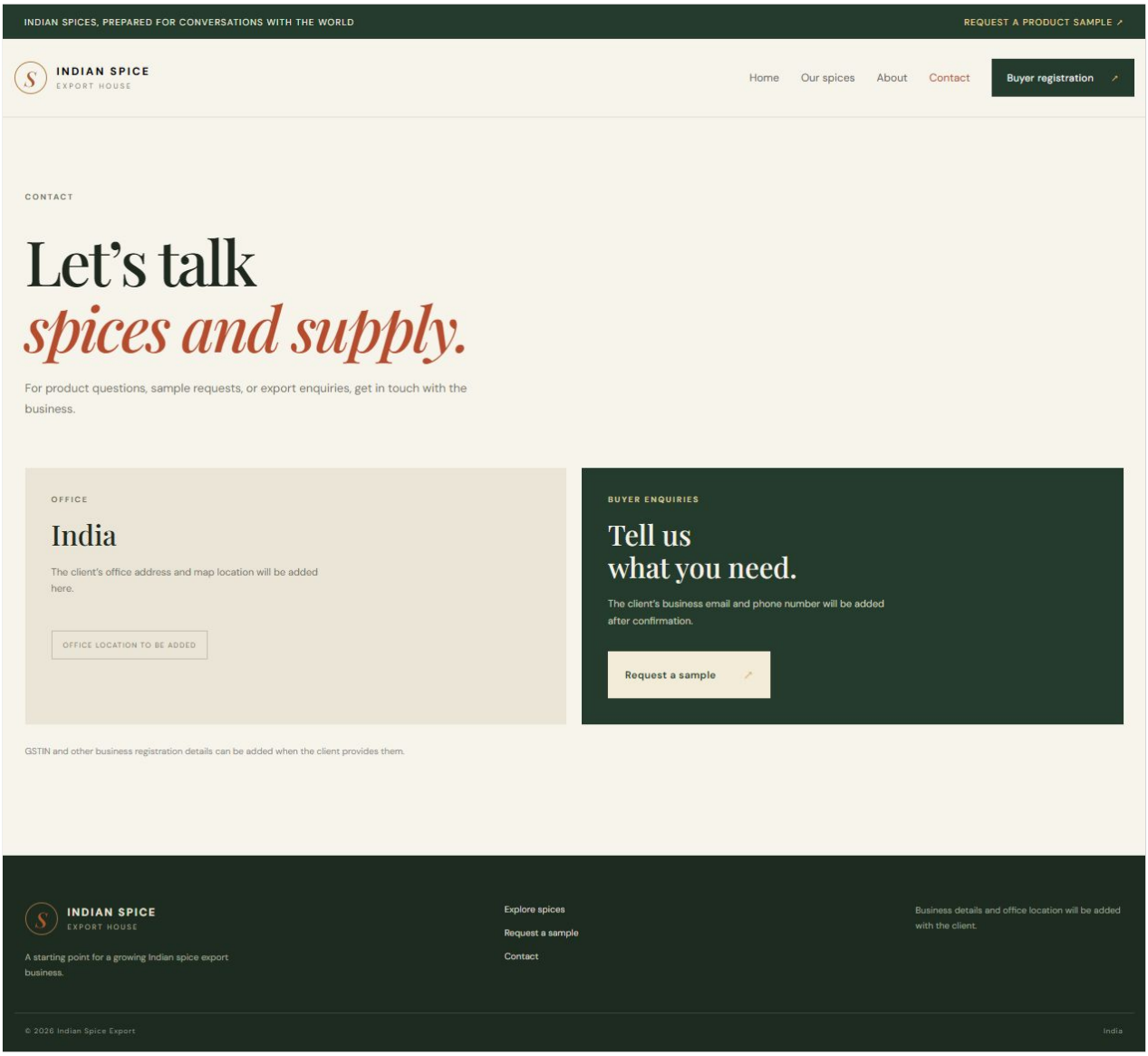


Figure 2.8: Contact route in the local website.



Conclusion:

The catalogue exercise applied ES6 arrow functions, callback functions, arrays and browser events to real spice-export data. Array methods separated data selection from card rendering: filter() narrowed the catalogue, map() produced the visible product entries, and reduce() supported a simple summary. Input and button events connected those transformations to the buyer's actions, while popup examples demonstrated ways to request confirmation or a short value. The visible catalogue and filtered state provide practical output for the report. Product data still comes from a local array, so the interactions do not depend on a backend or save an enquiry. This experiment prepared the data-rendering and event-handling ideas used by later React components.

The examples keep calculations close to their purpose and use clear event handlers so the catalogue can be extended without mixing every action into one callback. Search terms should be normalized before comparison, category filters should combine with rather than overwrite search, and the empty state should explain how to broaden the query. These decisions make the JavaScript demonstrations relevant to an actual buyer-facing product list.

SHUBHAM DHENGALE - 3574

Experiment No. 3

Title: Create a React Application using Functional Components, JSX, Props, and State

Aim:

To create a React spice catalogue using functional components, JSX, props, and state, with an interactive product search and sample-request action.

Tools Used:

React

JavaScript (ES6)

JSX

Vite

Visual Studio Code

Google Chrome

Methodology:

This experiment converts the catalogue into a React application using functional components and JSX. A shared site layout holds navigation and page structure, while product cards are rendered from local spice data. Props pass each product's name, botanical label, origin, description and sample availability to a reusable card. State stores the search term and selected category; event handlers update those values and React re-renders the matching result without manually editing the DOM. The website was started in a browser and checked by viewing the catalogue, changing its controls and observing the resulting cards. This establishes the component and state foundation before adding multiple routed views.

Theory:

A functional component is a JavaScript function that returns React elements. Components divide an interface into smaller parts that can be reused and maintained separately.

JSX is a syntax extension that lets developers write HTML-like element descriptions in JavaScript. A build tool such as Vite transforms JSX into code React can render.

Props are read-only values passed from a parent component to a child component. They let a reusable component display different data or call a parent-provided function.

State stores values that can change while a component is in use. The useState Hook provides the current value and an update function; updating state causes React to render the affected interface again.

Procedure:

1. Create or inspect the Vite React entry point and mount the root App component.
2. Build the shared site layout as a functional component and return page structure with JSX.
3. Keep spice catalogue records in a data module as an array of objects.
4. Create a reusable SpiceCard component that receives the product record and action callback through props.
5. Render the card collection with map() and a stable key for each product.
6. Add useState for the search text and selected product category.

7. Connect the search field and category buttons to state-setting event handlers.

8. Derive the visible list from state and display an empty message when no product matches.

9. Run the website and verify the card props, filtered output and responsive layout in the browser.

10. Change component state through the search box and category controls, then observe the rendered cards.

11. Check the no-results state and confirm that the original product list returns when the filters are cleared.

Code:

```
import { useState } from 'react';
const products = [
{ id: 1, name: "Turmeric", origin: "India", sample: true },
{ id: 2, name: "Black Pepper", origin: "India", sample: true },
{ id: 3, name: "Cardamom", origin: "India", sample: false }
];
function SpiceCard({ product, onRequestSample }) {
return (
<article className="spice-card">
<h2>{product.name}</h2>
<p>Origin: {product.origin}</p>
<p>{product.sample ? "Sample available" : "Ask for availability"}</p>
<button onClick={() => onRequestSample(product.name)}>
Request sample
</button>
</article>
);
}
function ProductGrid({ items, onRequestSample }) {
return (
<div className="product-grid">
{items.map((product) => (
<SpiceCard key={product.id} product={product}
onRequestSample={onRequestSample} />
))}
</div>
);
}
export default function App() {
const [query, setQuery] = useState("");
```

```
const visibleProducts = products.filter((product) =>
product.name.toLowerCase().includes(query.toLowerCase())
);
const requestSample = (name) => {
alert(`Sample request selected: ${name}`);
};
return (
<main>
<h1>Indian Spices for Export</h1>
<label htmlFor="productSearch">Search products</label>
<input id="productSearch" value={query}
onChange={(event) => setQuery(event.target.value)} />
<ProductGrid items={visibleProducts}
onRequestSample={requestSample} />
</main>
);
}
```

Output:

React renders reusable spice cards with product name, origin and availability. Updating the search state filters the cards without a full-page reload.

A card button calls its callback prop and reports the selected spice. The catalogue is read from local project data.

Figure 3.1: React spice catalogue rendered.

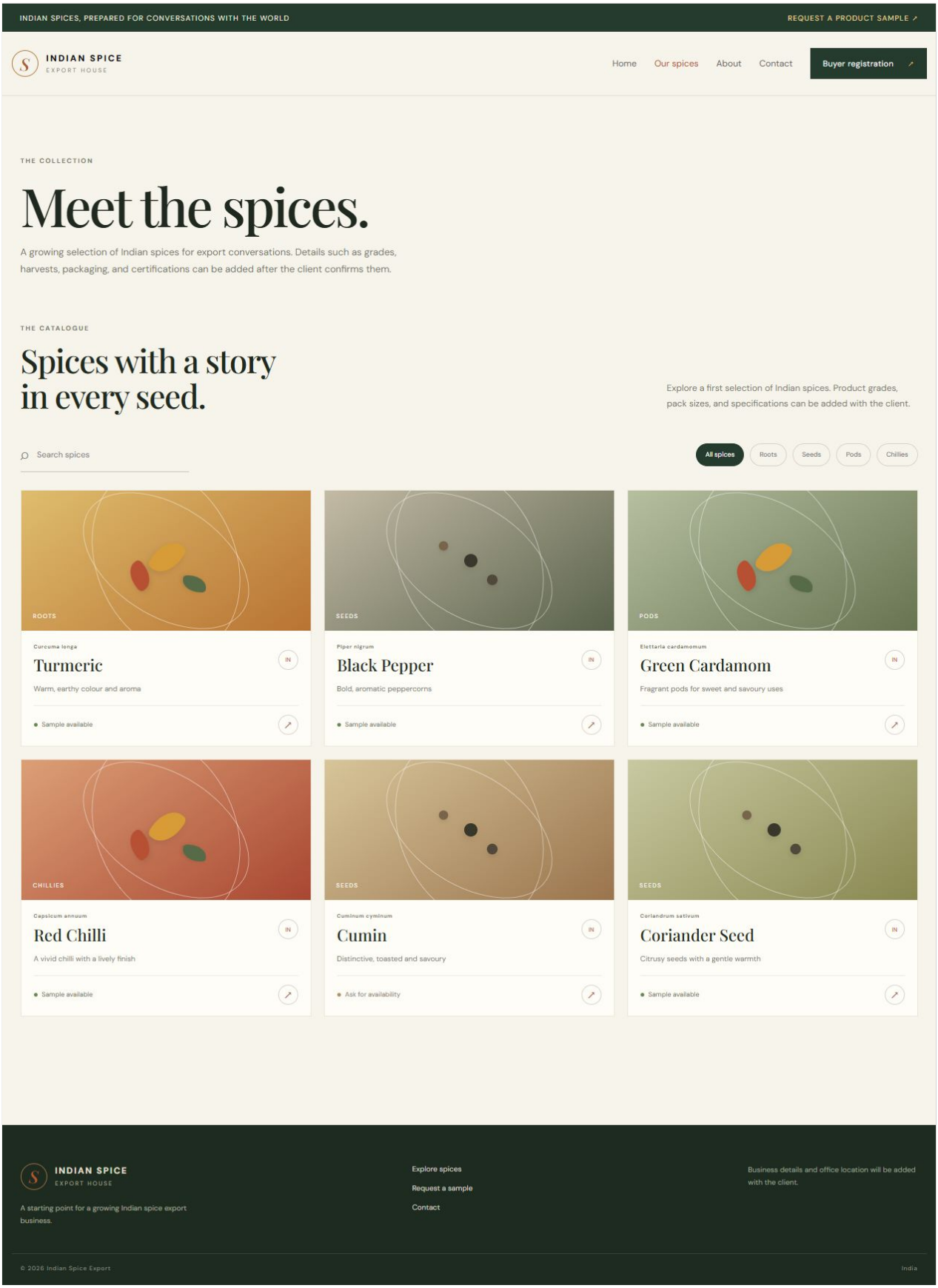


Figure 3.2: React catalogue card output.

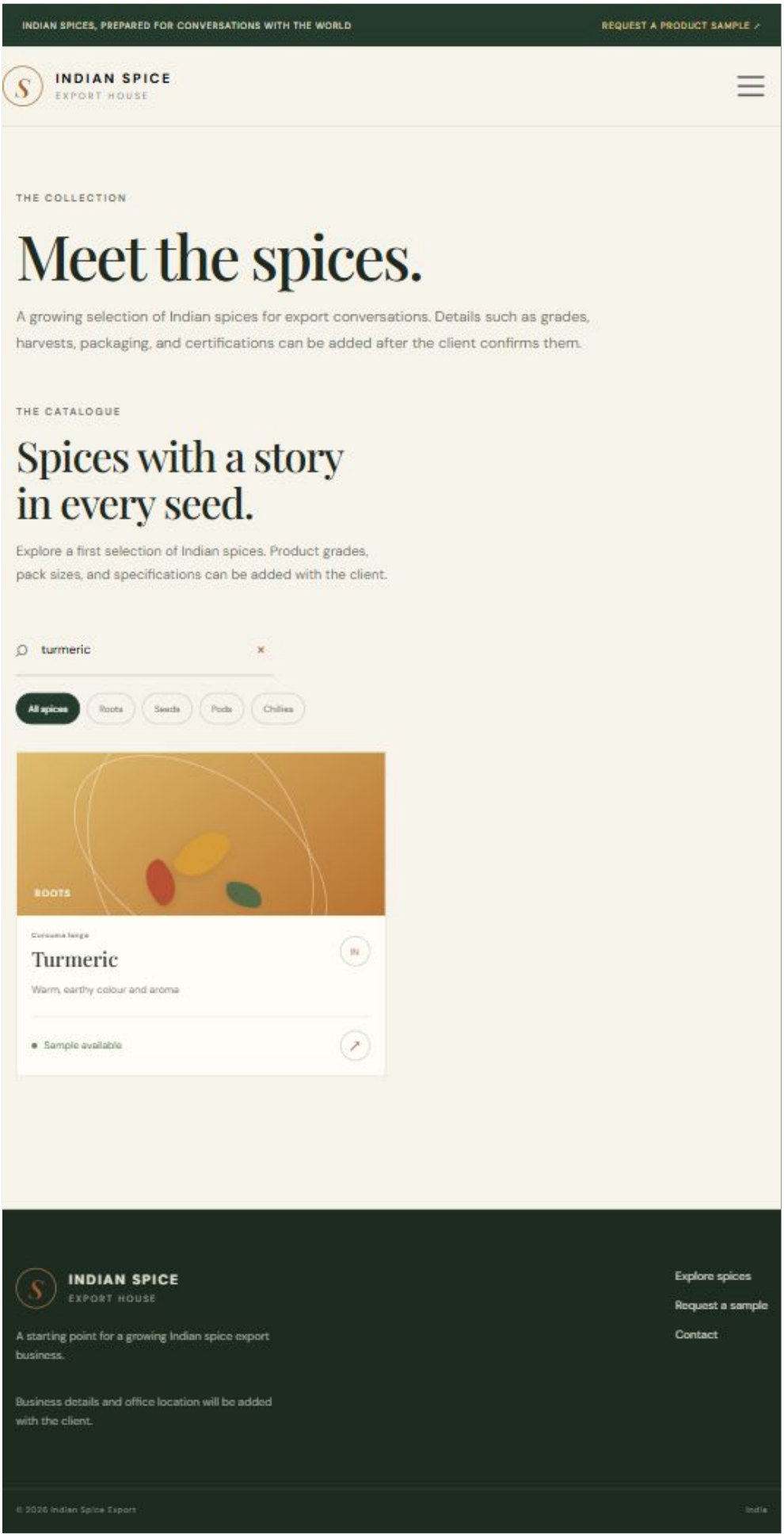


Figure 3.3: Category-filtered Roots output.

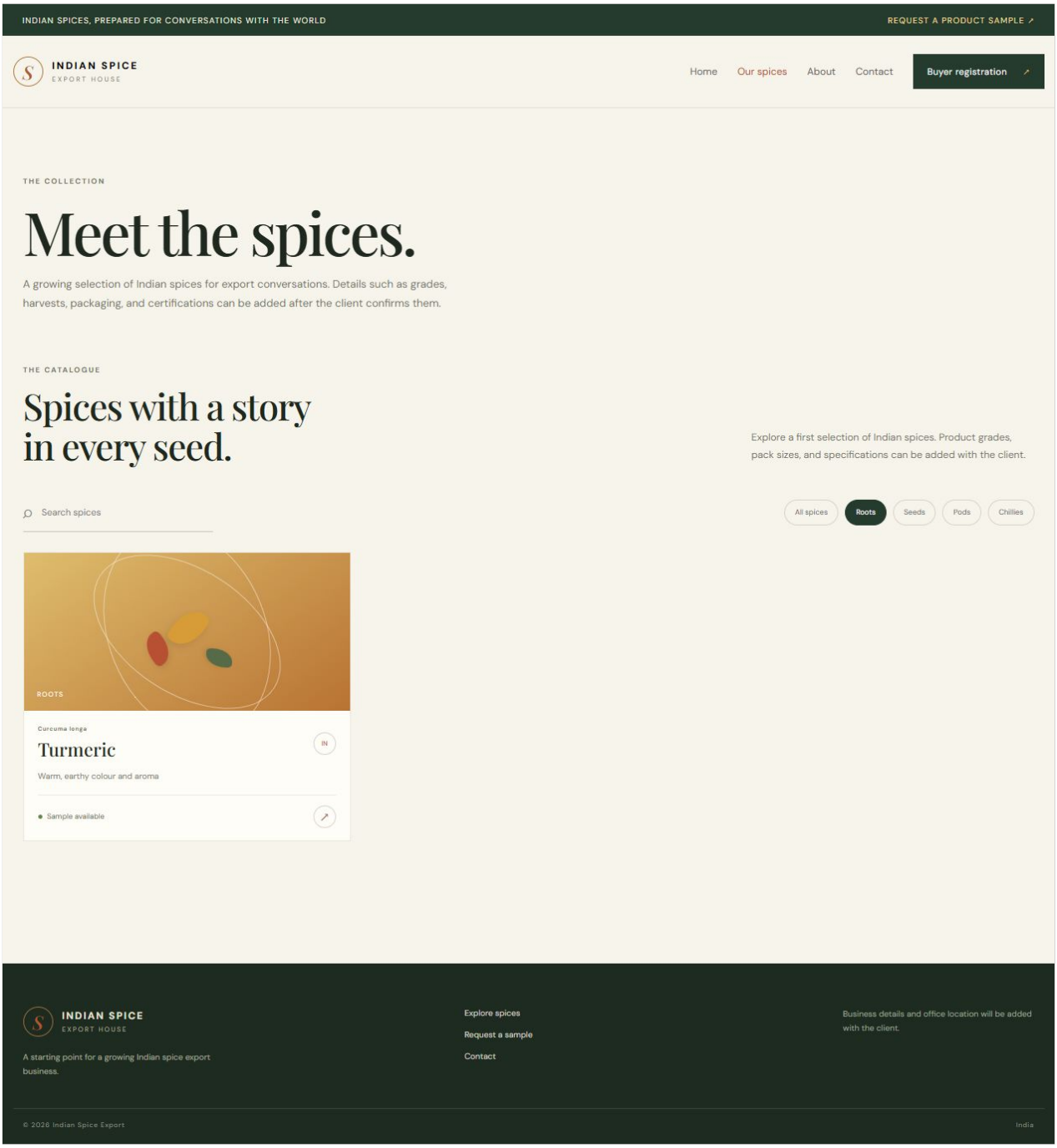


Figure 3.4: Search-filtered Cumin output.

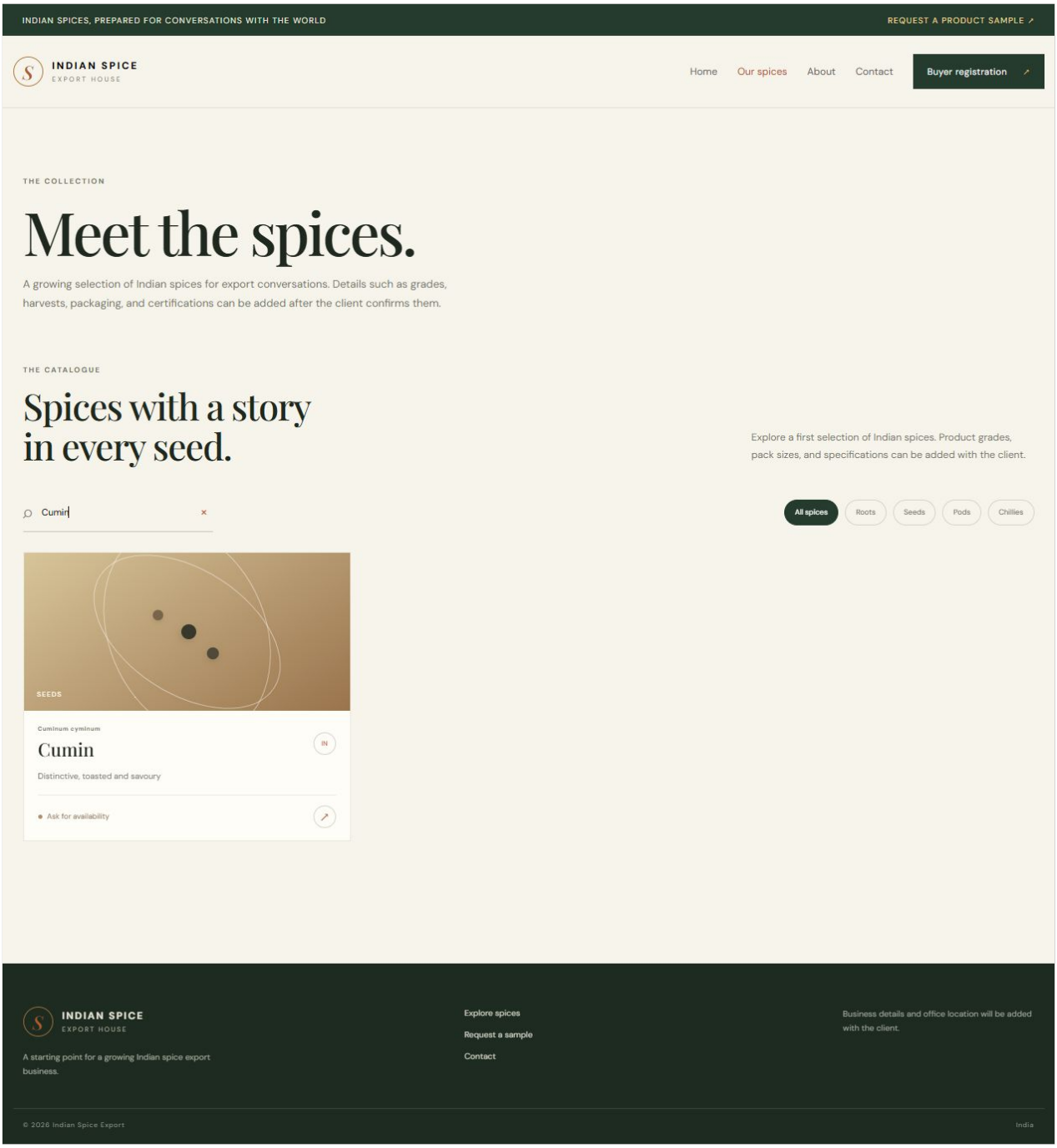


Figure 3.5: Product cards and origin details.

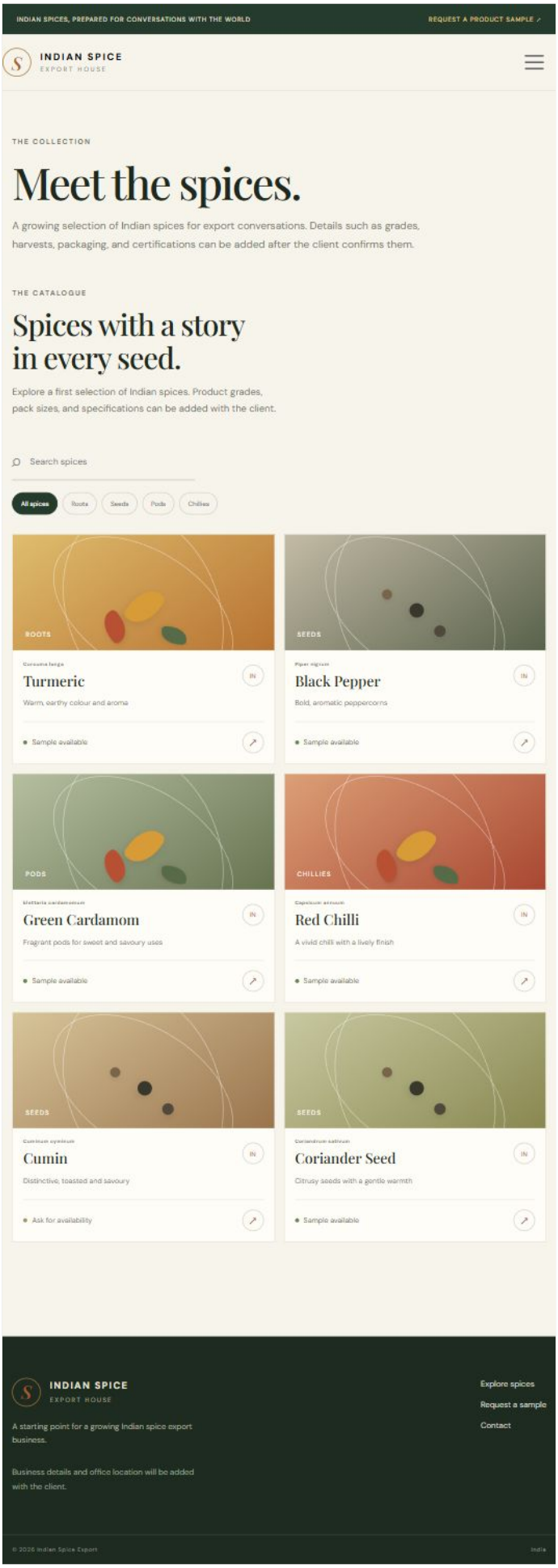


Figure 3.6: Desktop product grid.

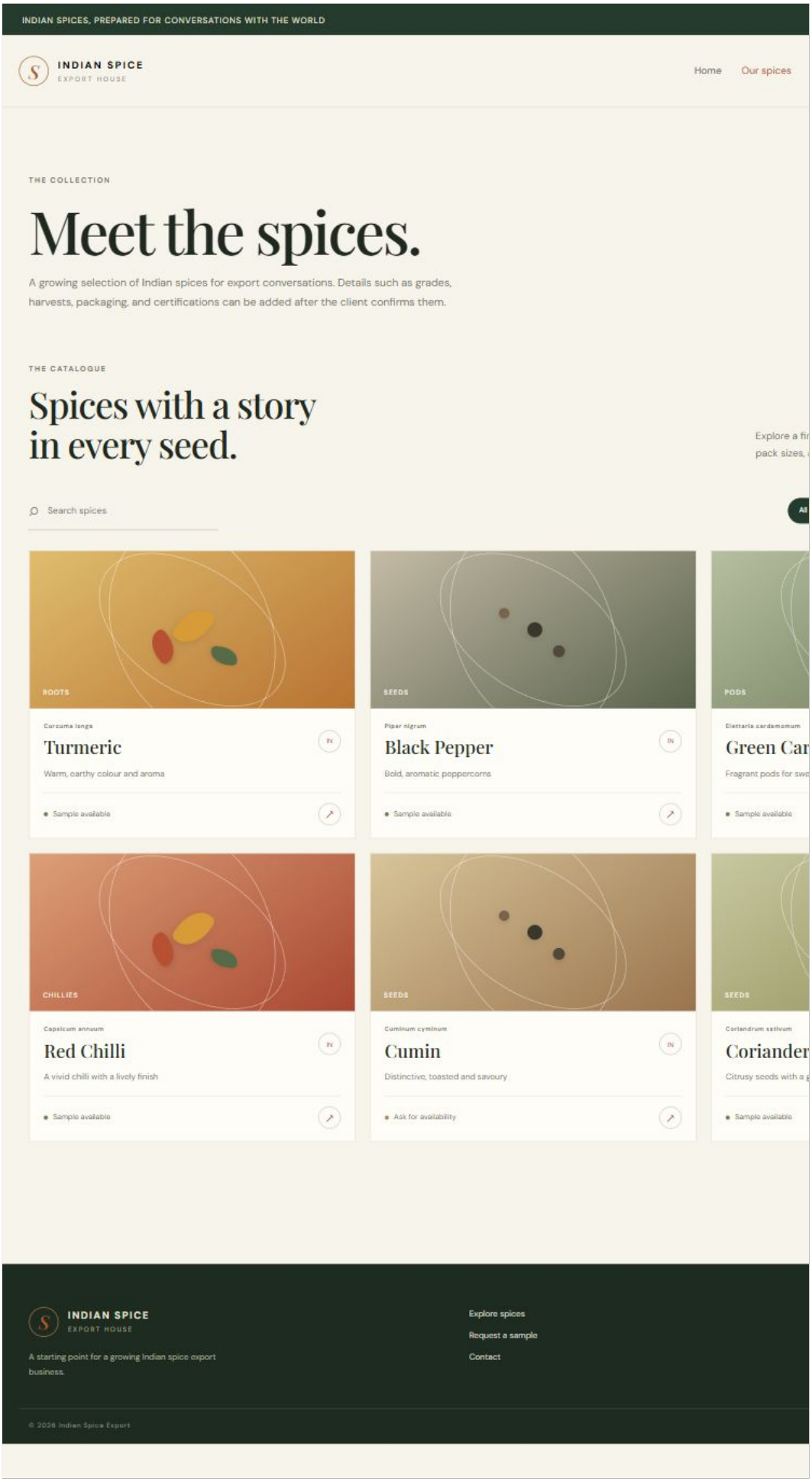


Figure 3.7: Sample route connected from a product.

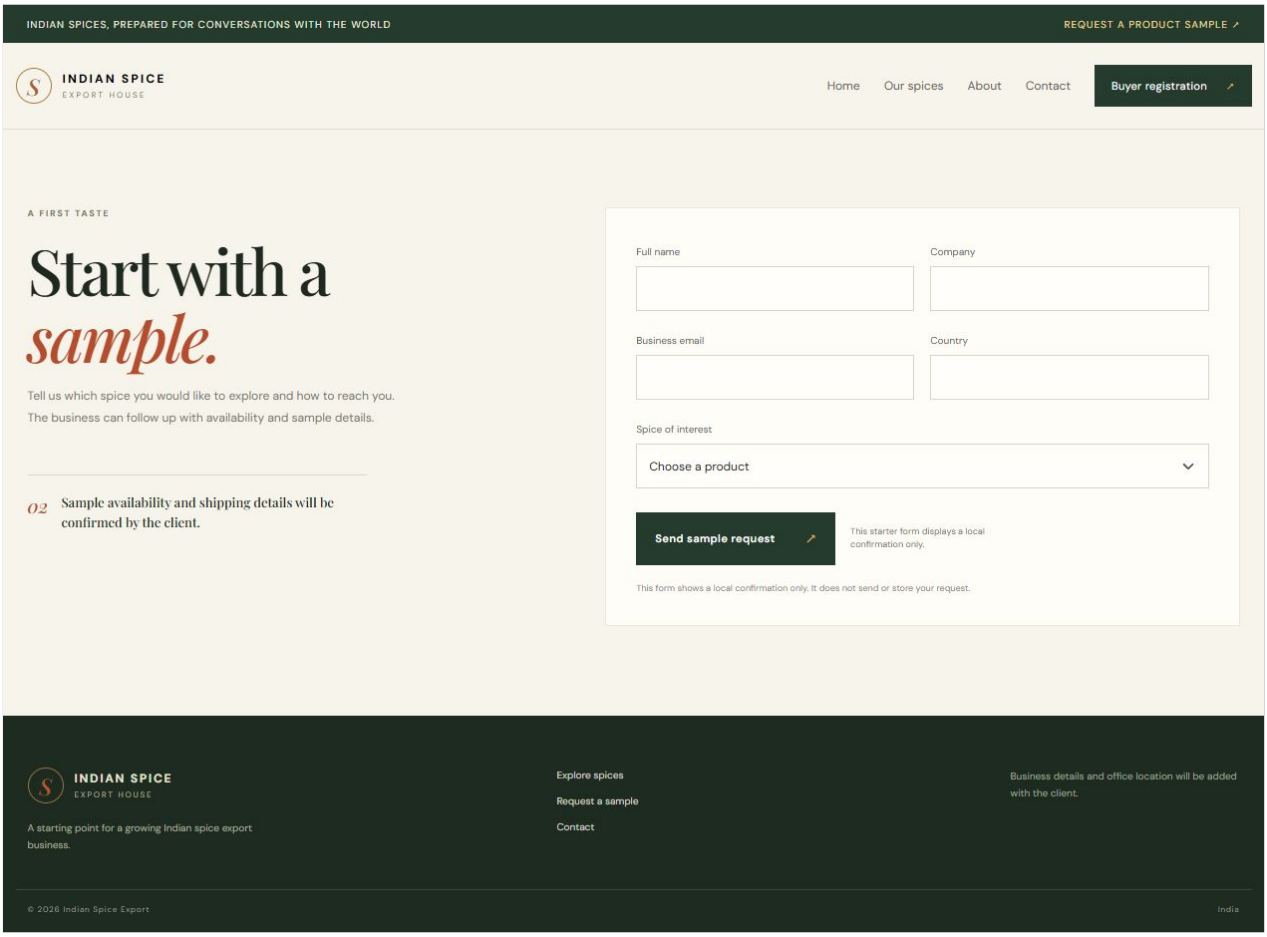
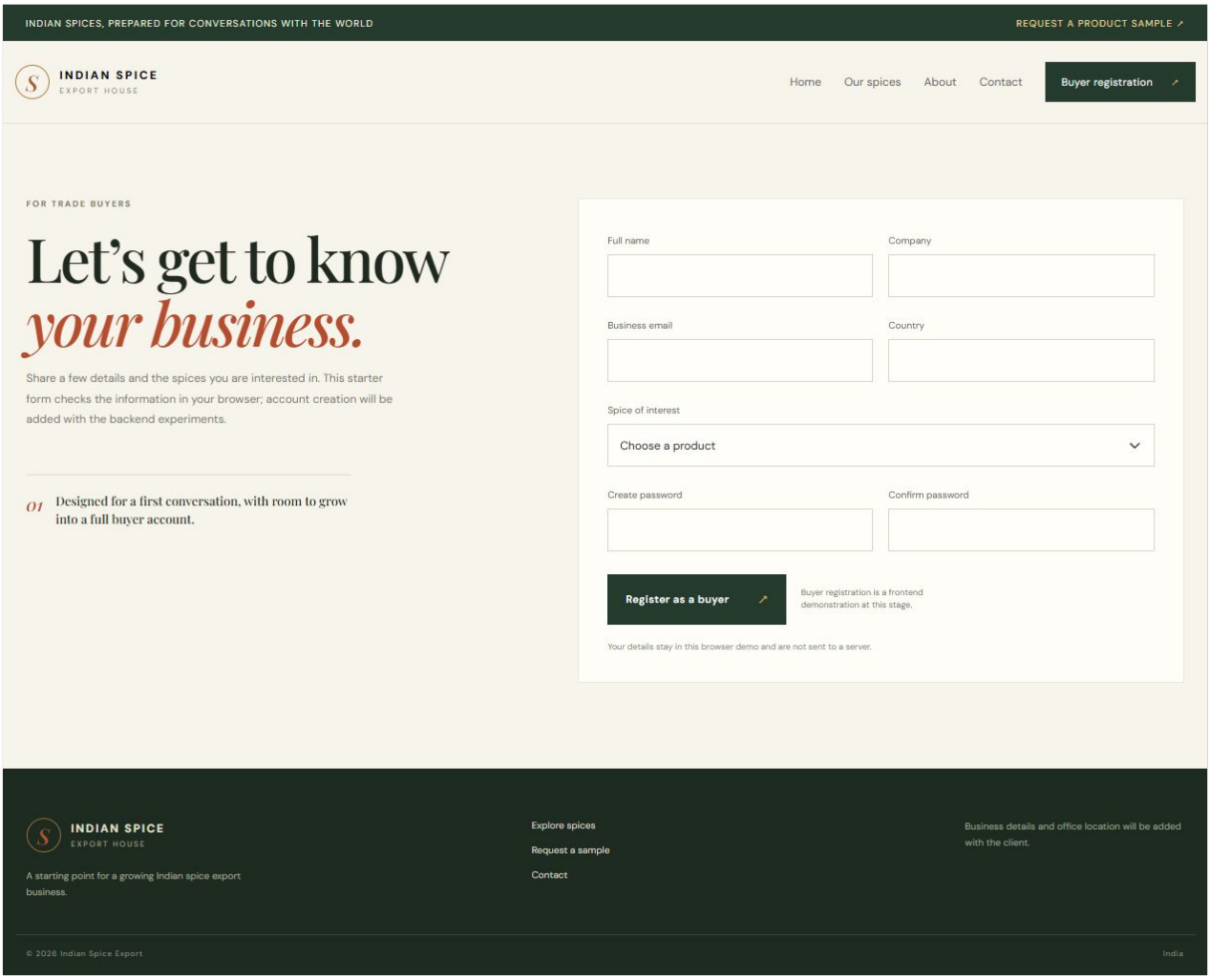


Figure 3.8: Registration route available from navigation.



Conclusion:

The React catalogue demonstrates the main ideas of functional components, JSX, props and state in a spice-export context. JSX defines the card and page markup, while the reusable card receives product details and a callback from its parent. The parent component owns search and category state, which makes the visible catalogue respond to user input without a full-page reload. Browser inspection of the cards and filtered state confirms that data flows from the local array through component props into the interface. The data source is still local, so this result is not evidence of database retrieval. The component design is ready to be reused across the routed website and later connected to API responses.

The component boundary makes future updates easier: a change to one spice card need not require rewriting the page, and the same card can be reused wherever product data appears. State belongs to the component that coordinates the search and category controls, while the card receives display data through props. This arrangement also gives the API integration in Experiment 9 a predictable place to replace the local data source.

SHUBHAM DHENGALE - 3574

Experiment No. 4

Title: Develop a Single Page Application (SPA) using React Router, Hooks, and Component Lifecycle Methods

Aim:

To develop a spice-export Single Page Application with React Router routes and React Hooks for navigation, page state, and lifecycle-equivalent side effects.

Tools Used:

React

React Router

JavaScript (ES6)

Vite

React Hooks (useState and useEffect)

Google Chrome

Methodology:

The website is organized as a single-page application so Home, Products, About, Buyer Registration, Sample Request and Contact can share one React runtime. React Router maps each URL path to a page component and keeps the shared navigation and footer in a layout. Route changes are checked in the browser, including direct entry to a page URL. React hooks manage local form state and side effects such as the document title; cleanup is included for any registered event listener. The sample route demonstrates a local confirmation state. These checks verify navigation and component behavior without claiming that form submissions or buyer accounts are persisted.

Theory:

A Single Page Application loads its main application shell once and updates the displayed view as the user navigates. Client-side navigation can avoid requesting a complete new page for each route.

React Router maps paths to components. BrowserRouter enables URL-based navigation, Link creates client-side navigation links, and Routes and Route select the component matching the current path.

React Hooks let functional components use state and side effects. useState() stores changing values, while useEffect() runs side-effect code after rendering. Its optional cleanup function runs before the effect is repeated and when its component is removed.

Class components used lifecycle methods such as componentDidMount and componentWillUnmount. In function components, useEffect() and its cleanup function provide the corresponding setup and cleanup pattern.

Procedure:

1. Wrap the shared website layout in BrowserRouter at the application root.

2. Declare route paths for Home, Products, About, Register, Samples and Contact.

3. Render each page as a separate functional component within the shared layout.

4. Add navigation links that point to the route paths and retain the shared site header and footer.

5. Use useState for sample-form values and the local submitted/confirmation state.

6. Use useEffect only for route-related side effects such as the document title, and return cleanup for listeners.

7. Navigate among routes using the visible links and confirm the URL and page content update without a full reload.

8. Paste a route URL directly into the browser to check that the page renders on direct entry.

9. Capture the About and Contact routes and note the client details still awaiting confirmation.

10. Navigate between each route from the shared navigation and check that the SPA remains responsive.

11. Open a nested route directly and refresh it to check the configured route fallback.

Code:

```
  import { useEffect, useState } from 'react';
import { BrowserRouter, Link, Route, Routes, useLocation }
from 'react-router-dom';
function Home() { return <h1>Indian Spices for Export</h1>; }
function About() { return <h1>About Us</h1>; }
function Products() { return <h1>Our Spices</h1>; }
function Contact() { return <h1>Contact and Office Location</h1>; }
function RequestSample() {
const [email, setEmail] = useState("");
const [message, setMessage] = useState("");
function handleSubmit(event) {
event.preventDefault();
setMessage(`Sample request prepared for ${email}.`);
}
return (
<section>
<h1>Request a Sample</h1>
<form onSubmit={handleSubmit}>
<label htmlFor="buyerEmail">Business email</label>
<input id="buyerEmail" type="email" required value={email}
onChange={(event) => setEmail(event.target.value)} />
<button type="submit">Prepare request</button>
</form>
<p role="status">{message}</p>
</section>
);
}
```

```
function SiteLayout() {
const location = useLocation();
const [width, setWidth] = useState(window.innerWidth);
useEffect(() => {
document.title = `Spice Export | ${location.pathname}`;
const updateWidth = () => setWidth(window.innerWidth);
window.addEventListener("resize", updateWidth);
return () => window.removeEventListener("resize", updateWidth);
}, [location.pathname]);
return (
<>
<nav aria-label="Main navigation">
<Link to="/">Home</Link> | <Link to="/about">About Us</Link> |
<Link to="/products">Products</Link> |
<Link to="/sample">Request a Sample</Link> |
<Link to="/contact">Contact</Link>
</nav>
<small>Viewport width: {width}px</small>
<Routes>
<Route path="/" element={<Home />} />
<Route path="/about" element={<About />} />
<Route path="/products" element={<Products />} />
<Route path="/sample" element={<RequestSample />} />
<Route path="/contact" element={<Contact />} />
</Routes>
</>
);
}
export default function App() {
return <BrowserRouter><SiteLayout /></BrowserRouter>;
}
```

Output:

React Router navigates among Home, About, Products, Request a Sample and Contact routes while keeping the application loaded.

The sample form shows a state-based confirmation. useEffect handles route-related title and resize-listener setup and cleanup.

Figure 4.1: SPA home route.

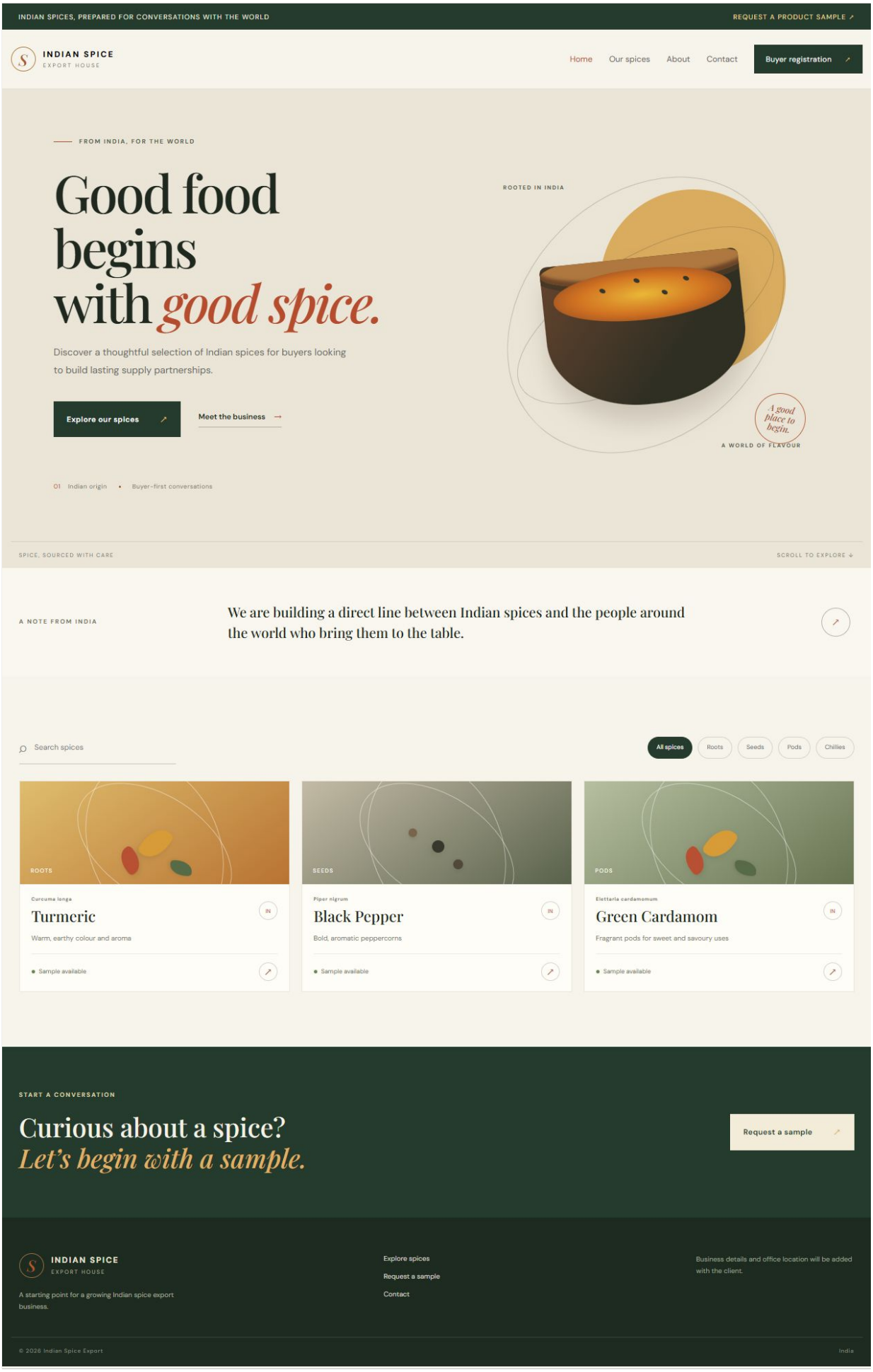


Figure 4.2: SPA products route.

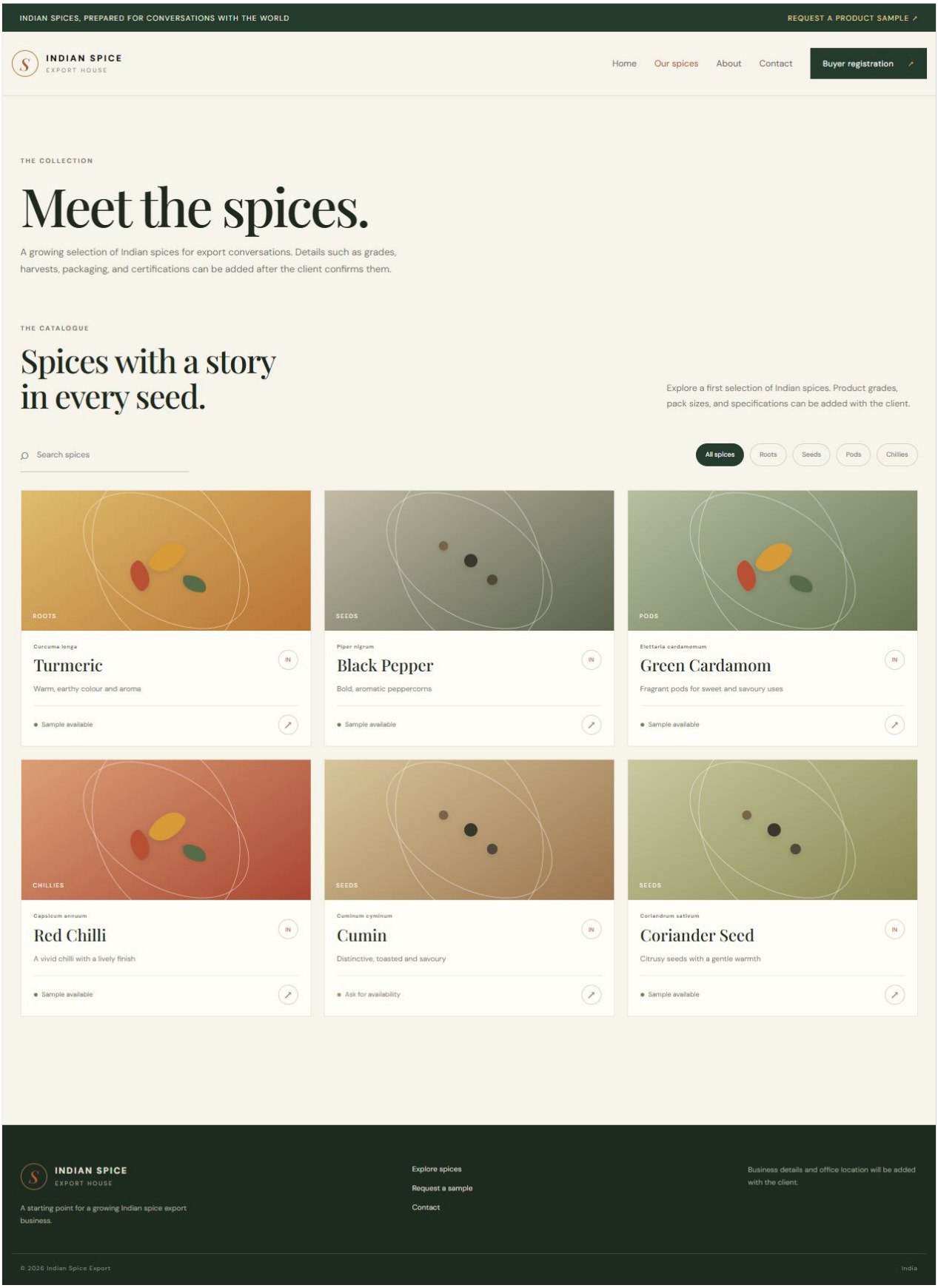


Figure 4.3: SPA about route.

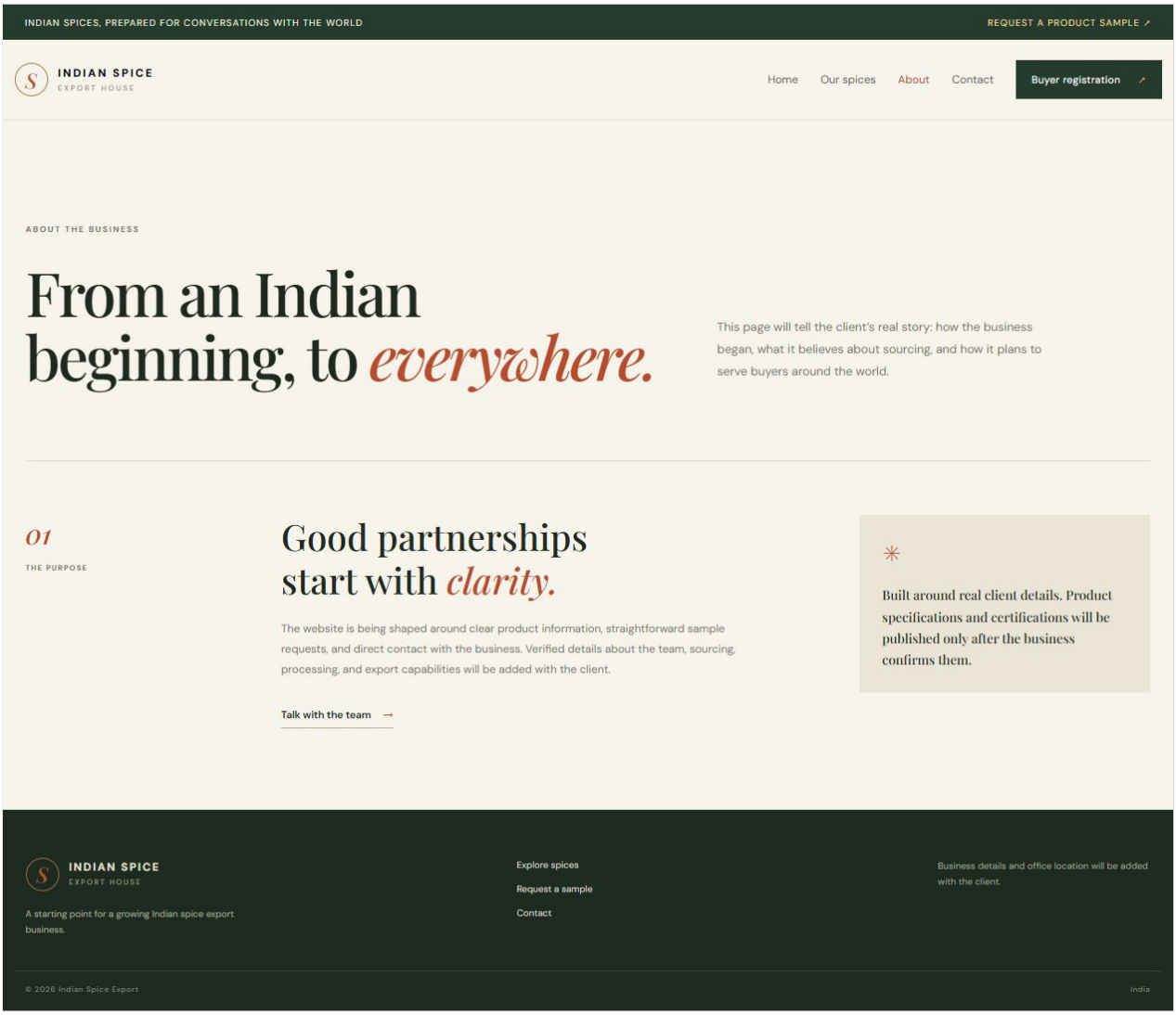


Figure 4.4: SPA contact route.

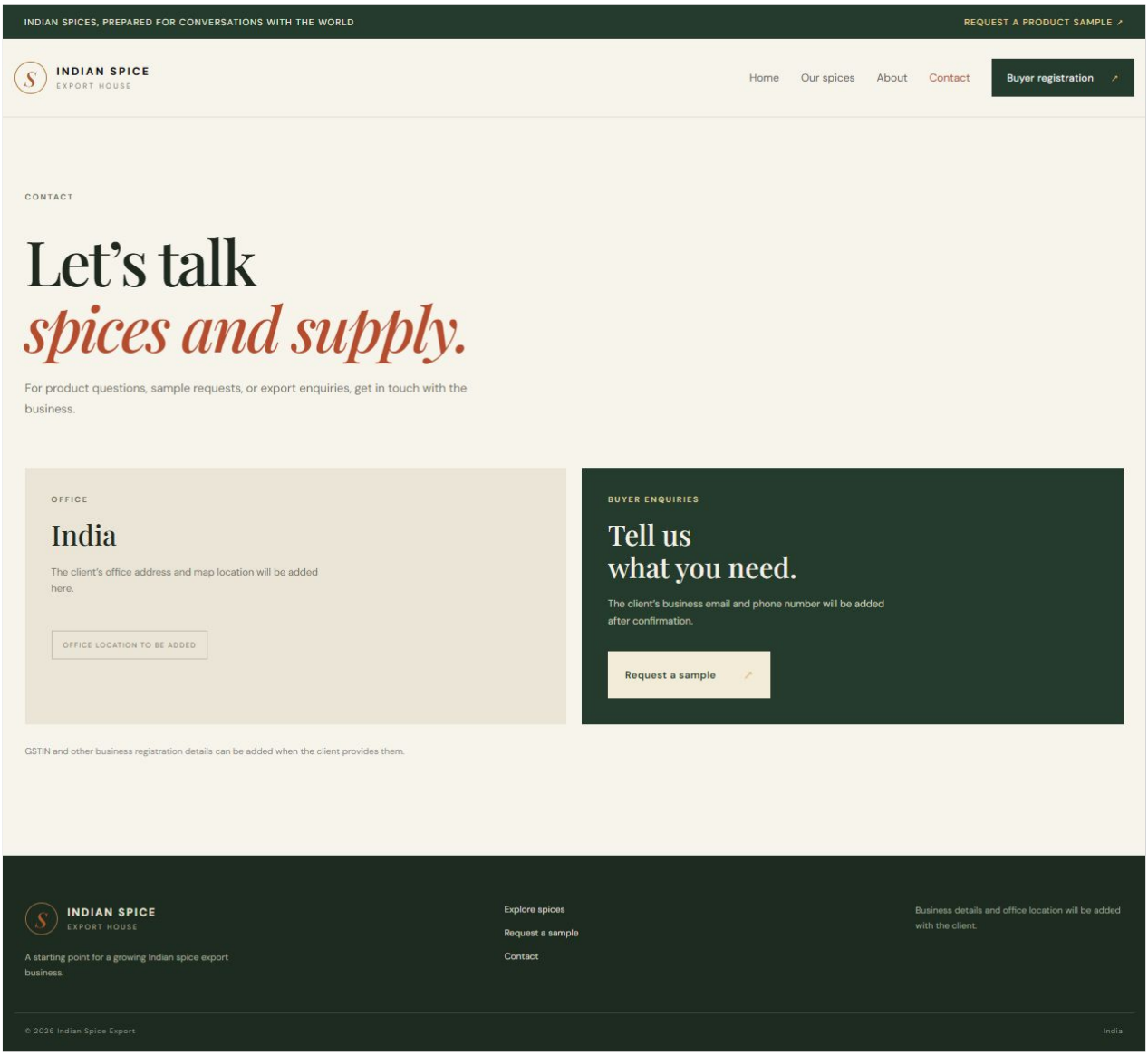


Figure 4.5: SPA sample-request route.

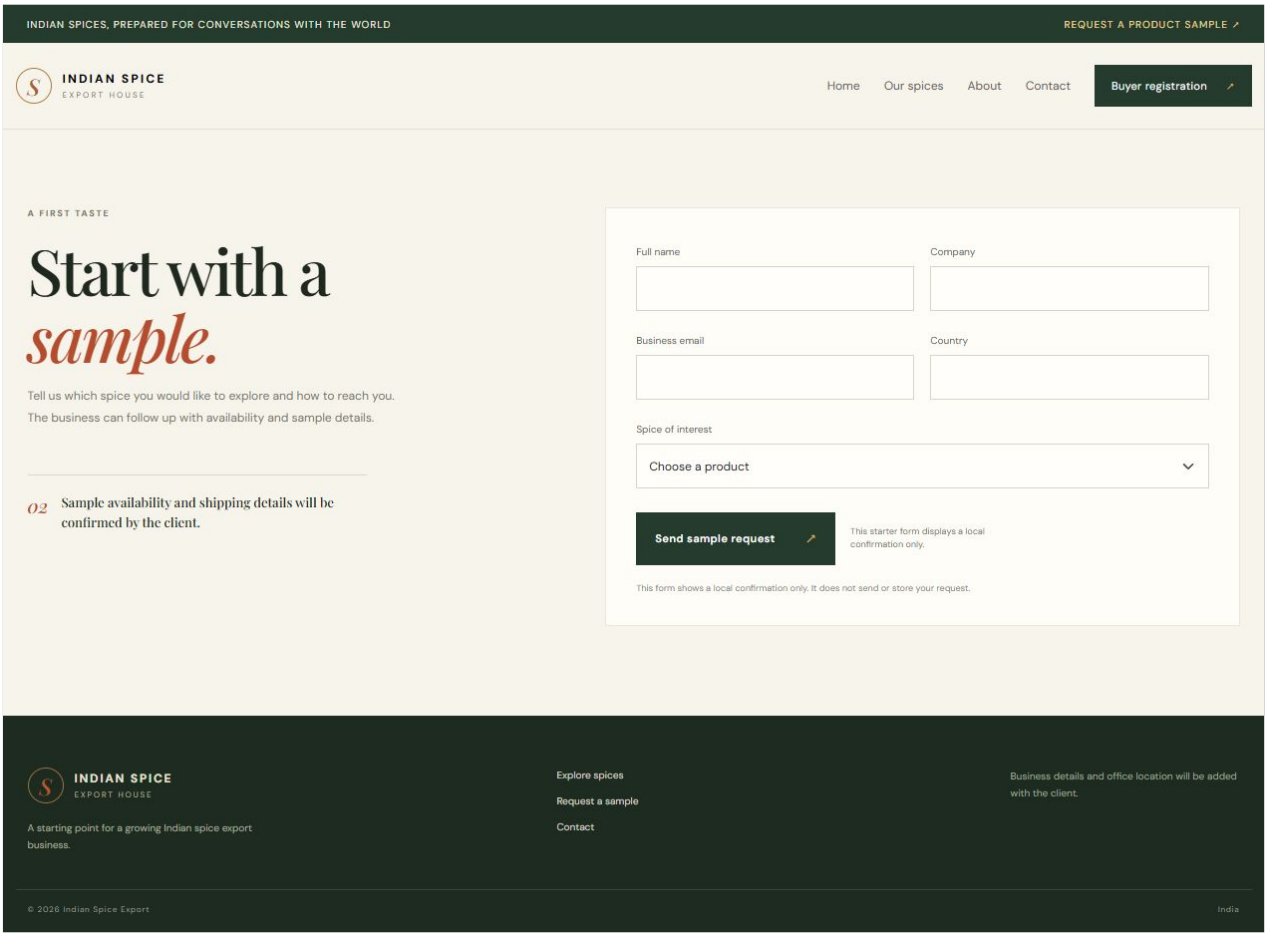


Figure 4.6: SPA buyer-registration route.

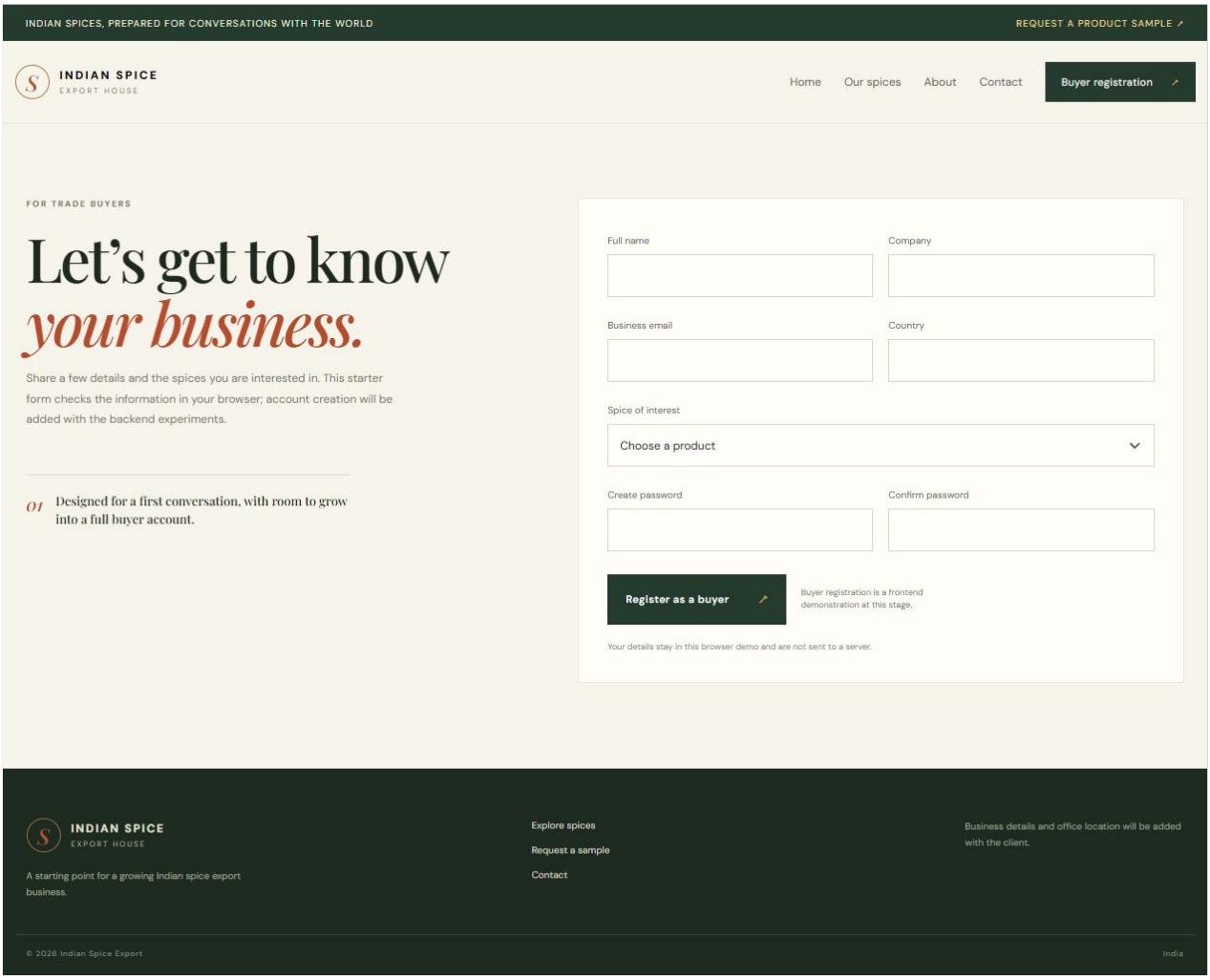


Figure 4.7: Category-filtered client-side catalogue.

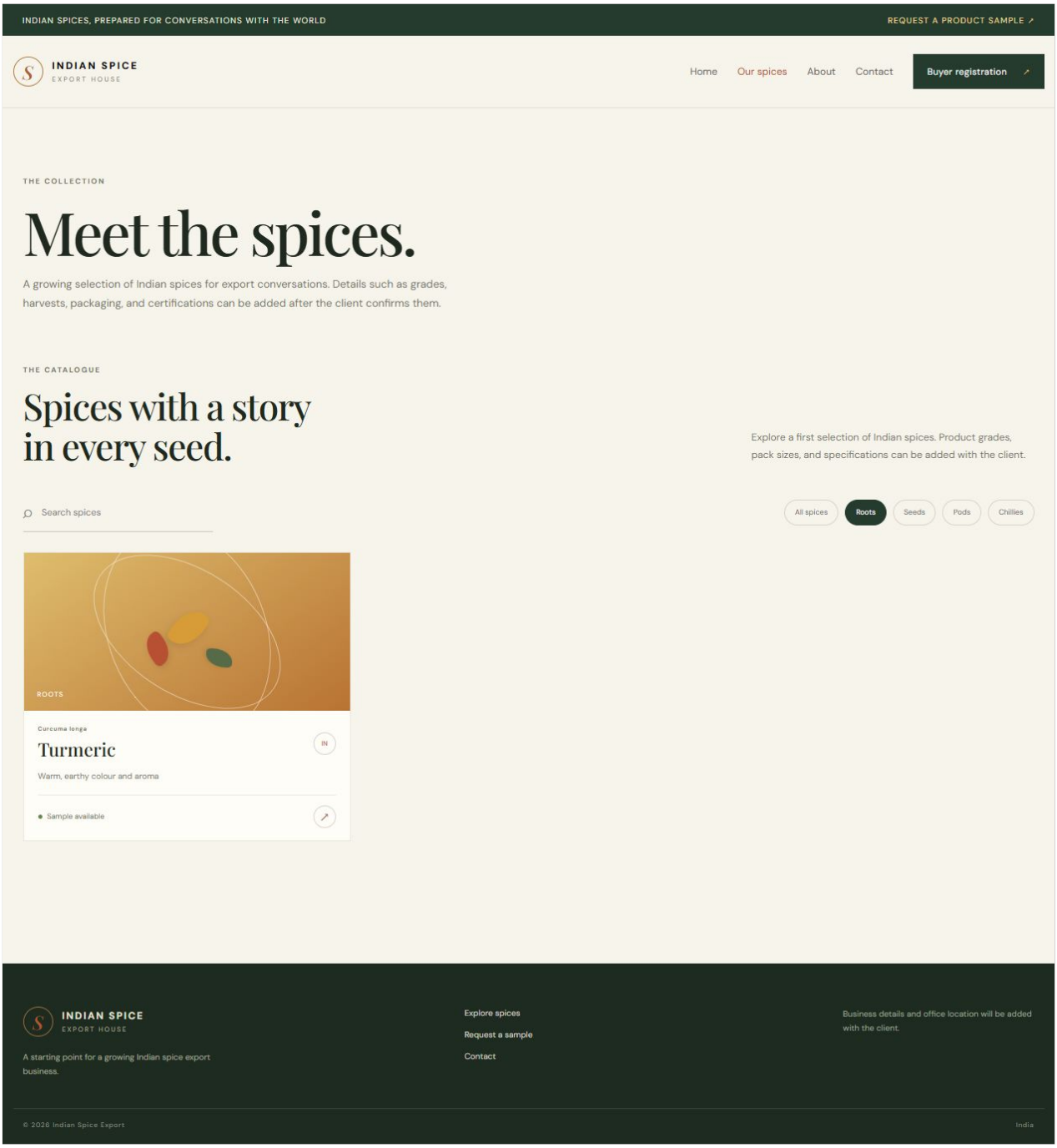
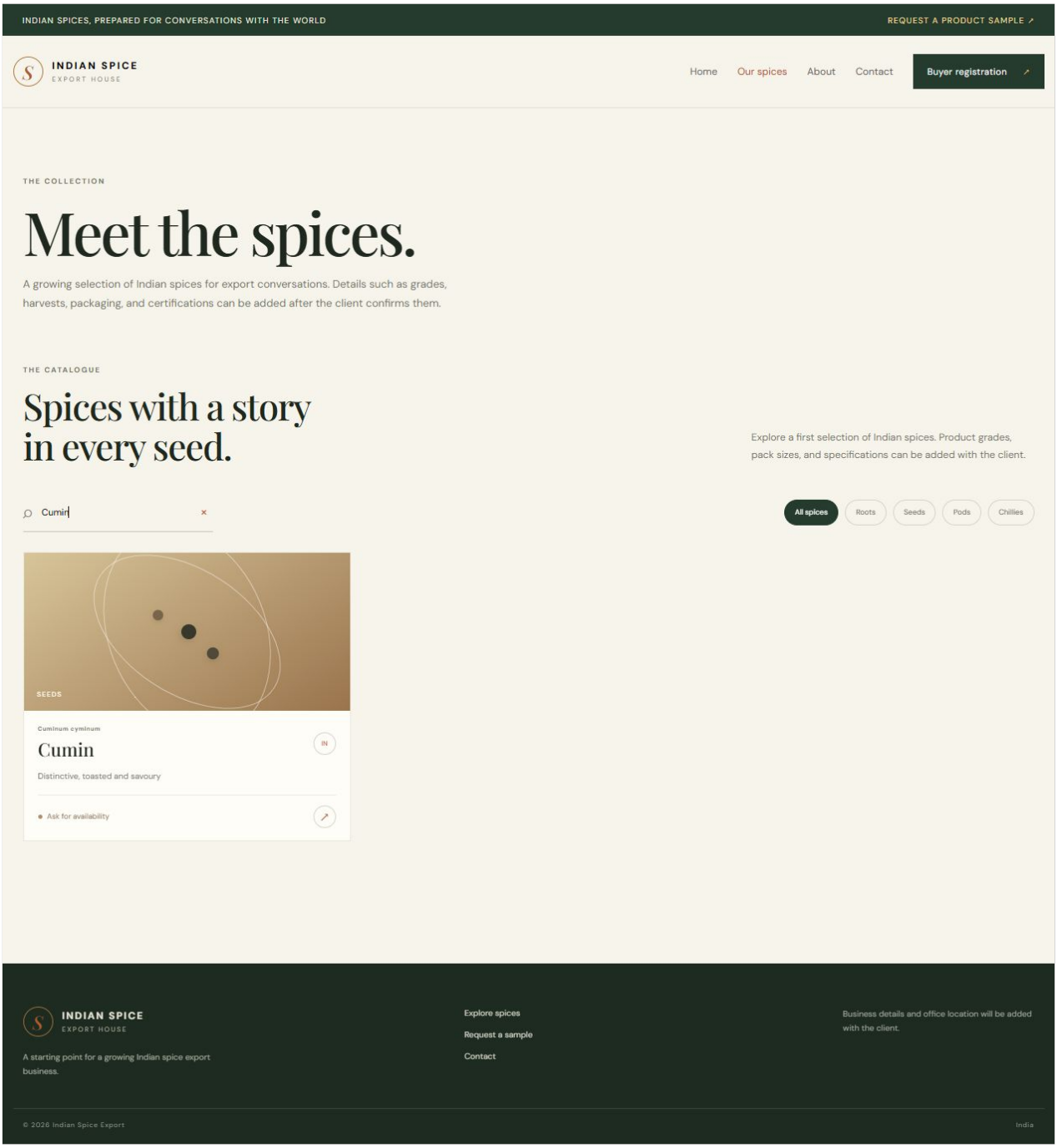


Figure 4.8: Search-filtered catalogue state.



Conclusion:

This experiment assembled the spice-export pages into a React single-page application using React Router and hooks. Route components separate the Home, catalogue, About, registration, sample and contact content, while the shared layout keeps navigation and page identity consistent. The browser checks showed that the route content changes as the URL changes and that pages can be opened directly. React state supports the sample form's local confirmation, and useEffect demonstrates how a component can set up and clean up side effects. These results establish website navigation and client-side interaction. Persistent registrations, sample requests, verified office details and production routing require later backend configuration and client-approved business information.

Direct route loading is an important SPA check because buyers may save or share a product or contact URL. A production host must also be configured to return the SPA entry document for known application routes. The local route checks establish the client-side behavior; deployment rewrite rules, persistent submissions,

confirmed address details and the client’s official business identity remain later release tasks.

SHUBHAM DHENGALE - 3574

Experiment No. 5

Title: Install and Configure MongoDB. Create Database, Collections, Documents, and Perform CRUD Operations

Aim:

To organize spice-export product and sample-request data in MongoDB and demonstrate Create, Read, Update and Delete (CRUD) using mongosh and Mongoose.

Tools Used:

MongoDB Community Server or Atlas, mongosh, MongoDB Compass, Node.js, Mongoose, Visual Studio Code.

Methodology:

The data model was designed from the website's two main resources: export products and buyer sample requests. The Product schema defines required identifiers and descriptive fields, while SampleRequest stores buyer contact details, selected spice, message and request status. Mongoose validation was executed locally against a valid product and a product missing required fields; this test does not need a database connection. The report also describes the corresponding MongoDB database, collections and CRUD commands so they can be run when a local MongoDB instance or Atlas URI is available. CRUD persistence is kept distinct from schema validation in the recorded evidence.

Theory:

MongoDB is a document database. A database contains collections, and collections contain BSON documents with fields such as product name, origin and availability.

CRUD means Create, Read, Update and Delete. MongoDB methods insertOne(), find(), updateOne() and deleteOne() implement these operations. Mongoose supplies schemas and models for Node.js applications.

Procedure:

1. Review the Product and SampleRequest Mongoose schema files in the server project.

2. Identify required fields, field types, defaults, trimming rules, maximum lengths and the sample-request status values.

3. Choose the spice_export_db database and the products and samplerequests collections for the case study.

4. Create a valid Product model instance using a spice already listed in the local catalogue.

5. Run validateSync() and inspect the validated field names without saving the document.

6. Create a second instance with required values missing and inspect the reported validation errors.

7. When MongoDB is configured, select the database and insert a test product document with mongosh.

8. Read the inserted record, update one field, verify the update, then delete the test record.

9. Repeat database operations only against a test record and capture genuine mongosh output after a database connection is available.

10. Check each CRUD result by reading the document after insert and update, and confirm the test record after deletion.

11. Save screenshots only after running mongosh against a configured test database; do not treat the listed expected results as a live run.

Code:

```
use spice_export_db
```

db.createCollection('products')

db.createCollection('samplerequests')

db.products.insertOne({ slug:'turmeric', name:'Turmeric', origin:'India', sampleAvailable:true, active:true })

db.products.find({ active:true })

db.products.updateOne({ slug:'turmeric' }, { $set:{ sampleAvailable:false } })

db.products.findOne({ slug:'turmeric' })

db.products.deleteOne({ slug:'turmeric' })

Output:

Expected mongosh results after MongoDB is configured:

use spice_export_db -> switched to db spice_export_db

insertOne(...) -> acknowledged: true; insertedId: generated ObjectId

find({ active:true }) -> returns the Turmeric document

updateOne(...) -> matchedCount: 1; modifiedCount: 1

deleteOne(...) -> deletedCount: 1

ObjectId and shell formatting vary by database instance; no credentials are included.

Figure 5.1: Actual Mongoose schema validation output; no database write was made.

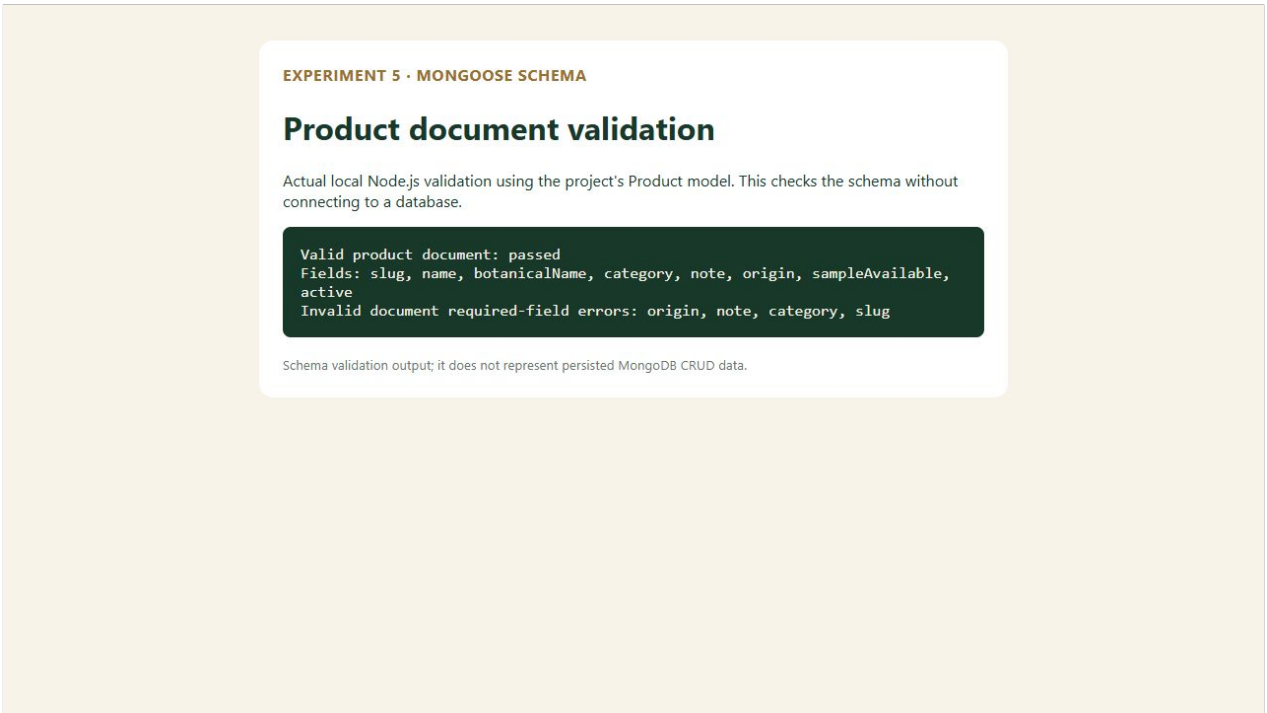


Figure 5.2: Product catalogue used as the CRUD case-study dataset.

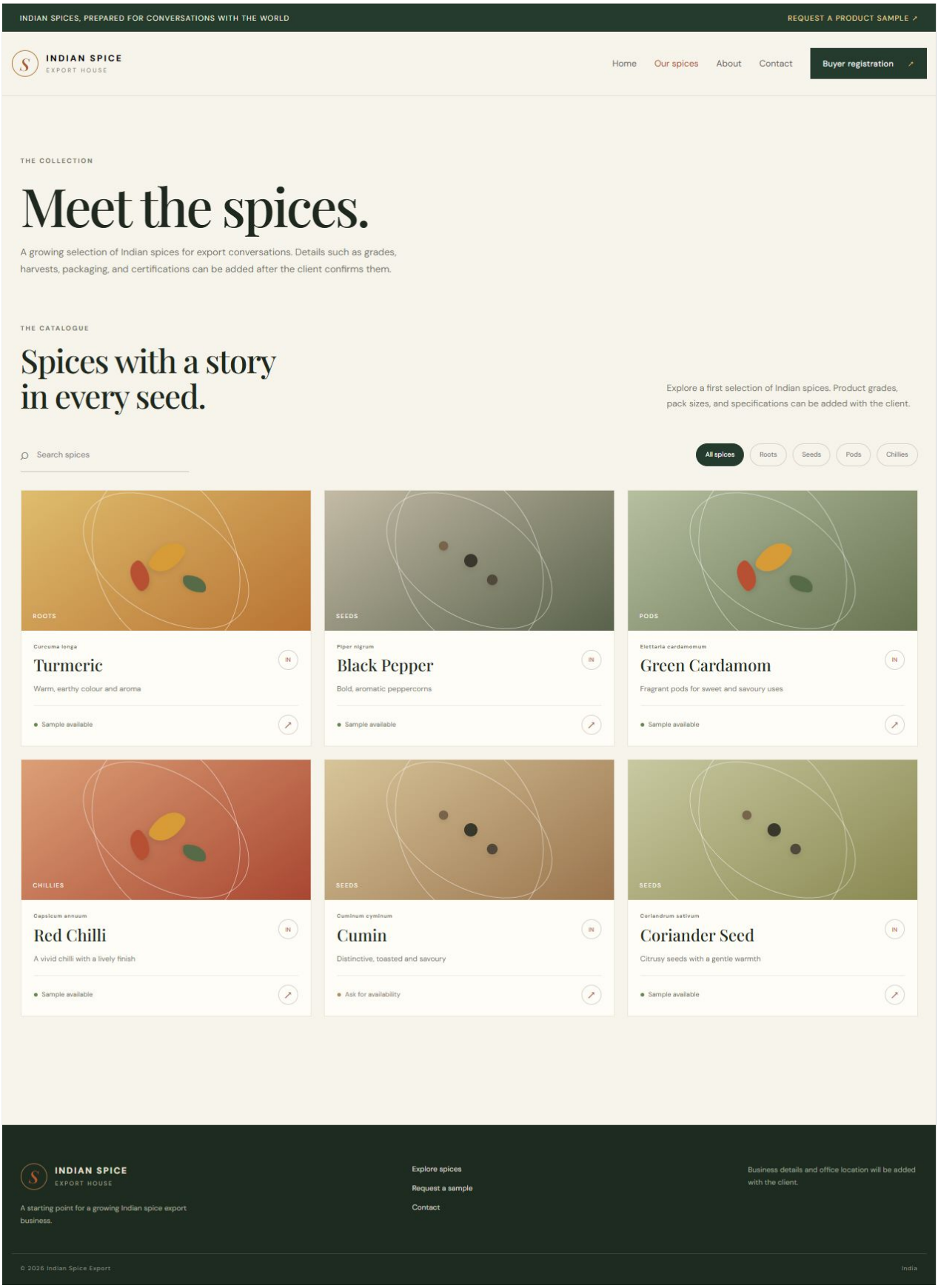


Figure 5.3: Product category state in the client catalogue.

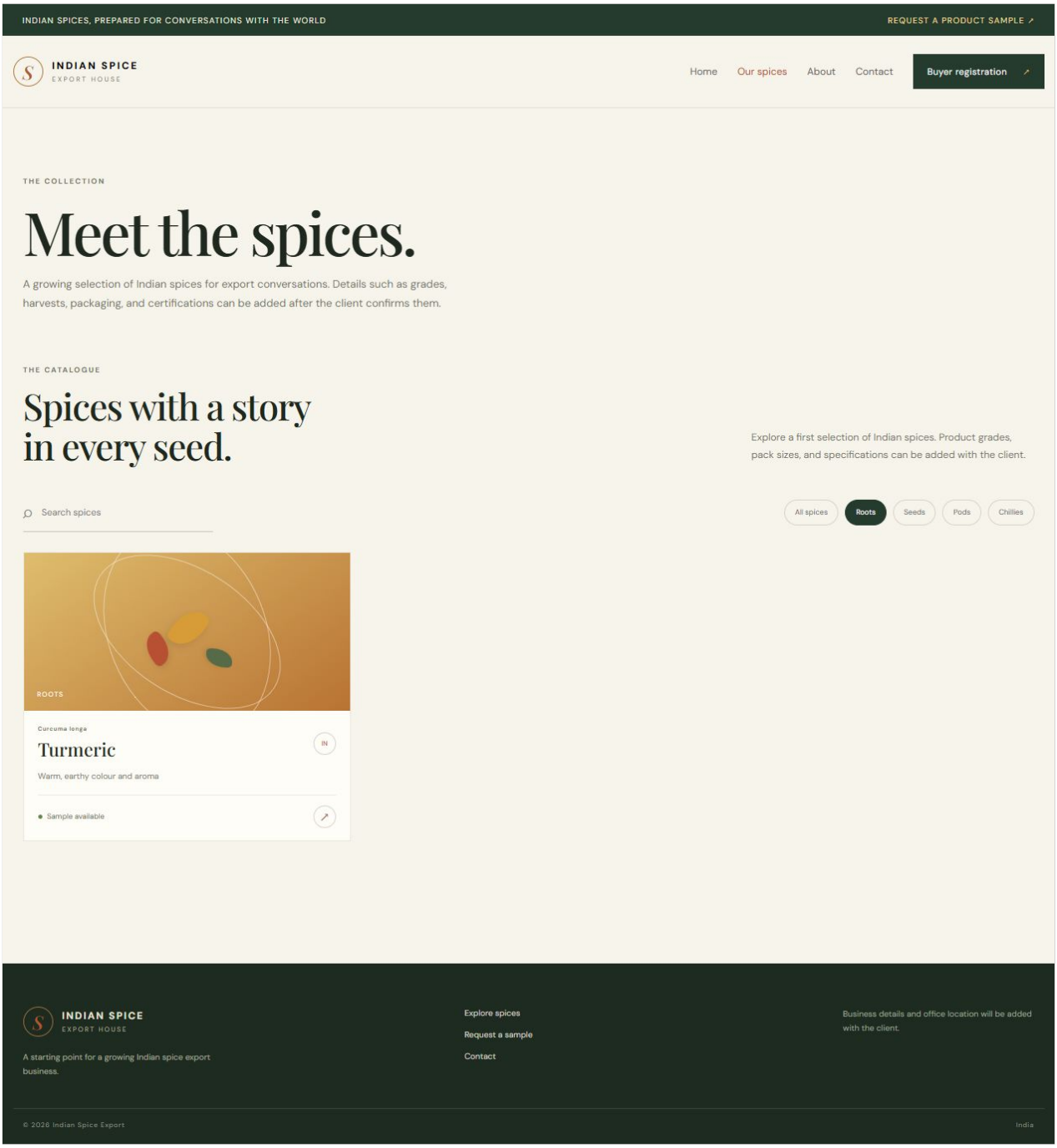


Figure 5.4: Search result for a spice product.

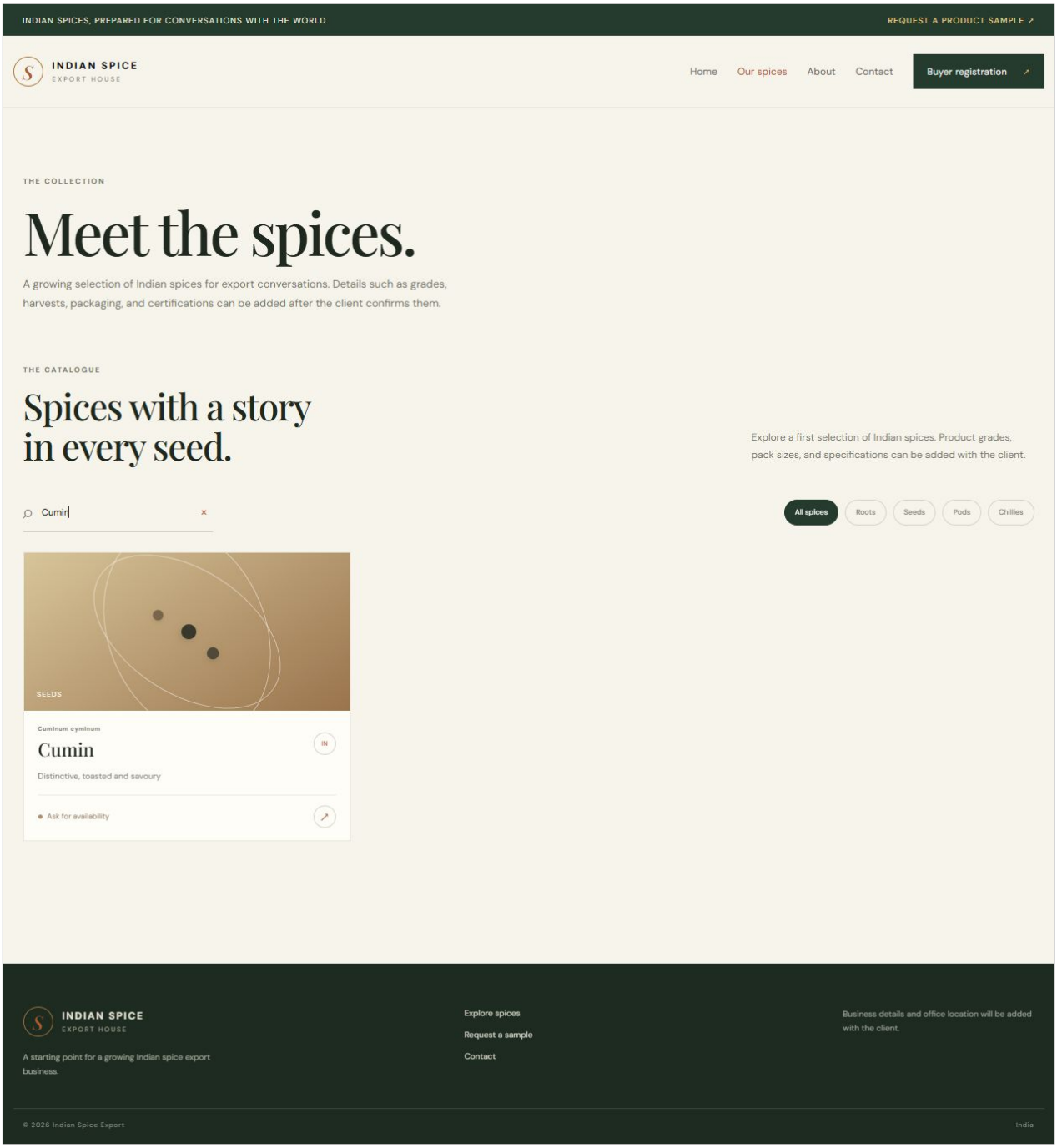


Figure 5.5: About route in the same project.

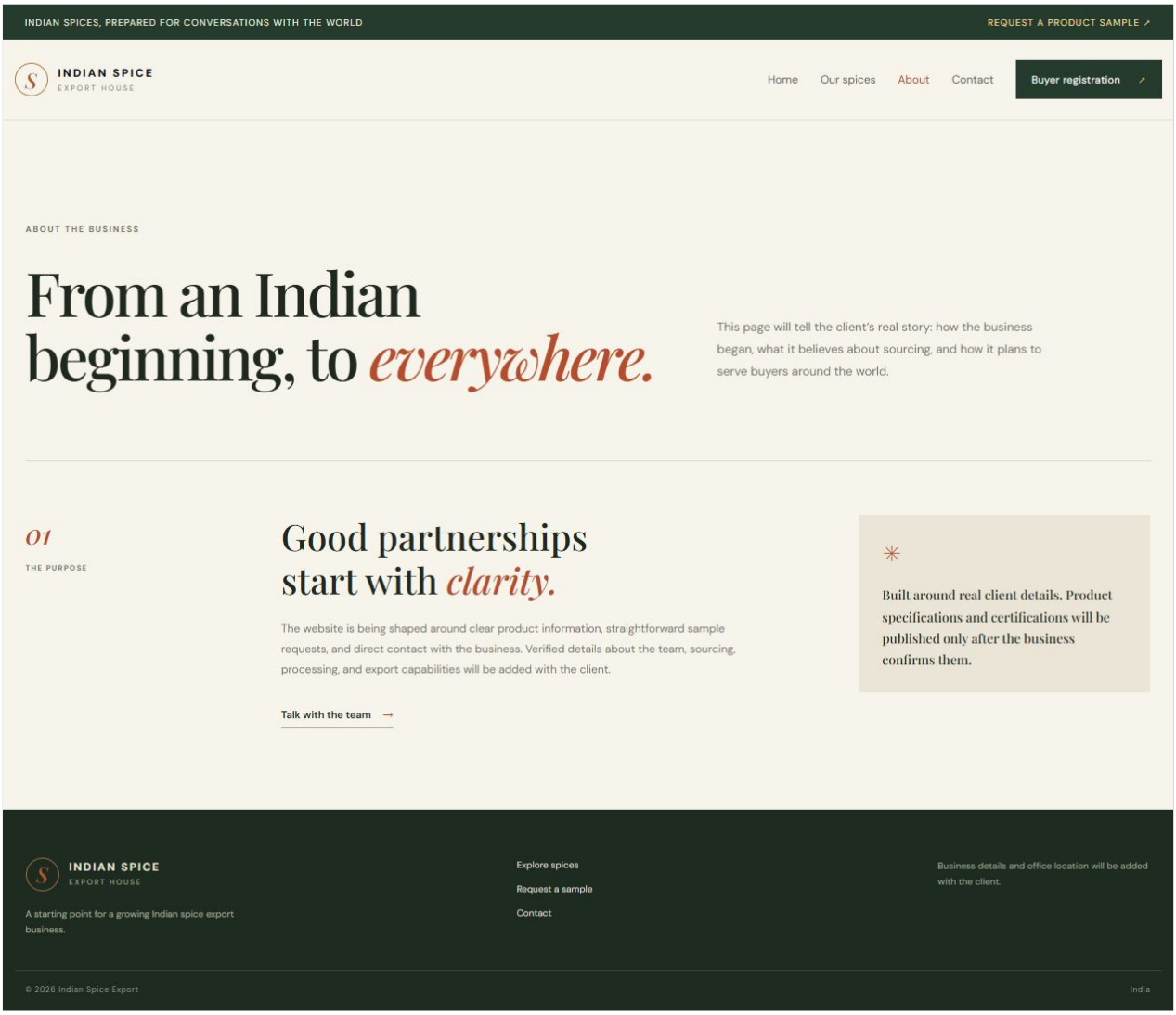


Figure 5.6: Contact route in the same project.

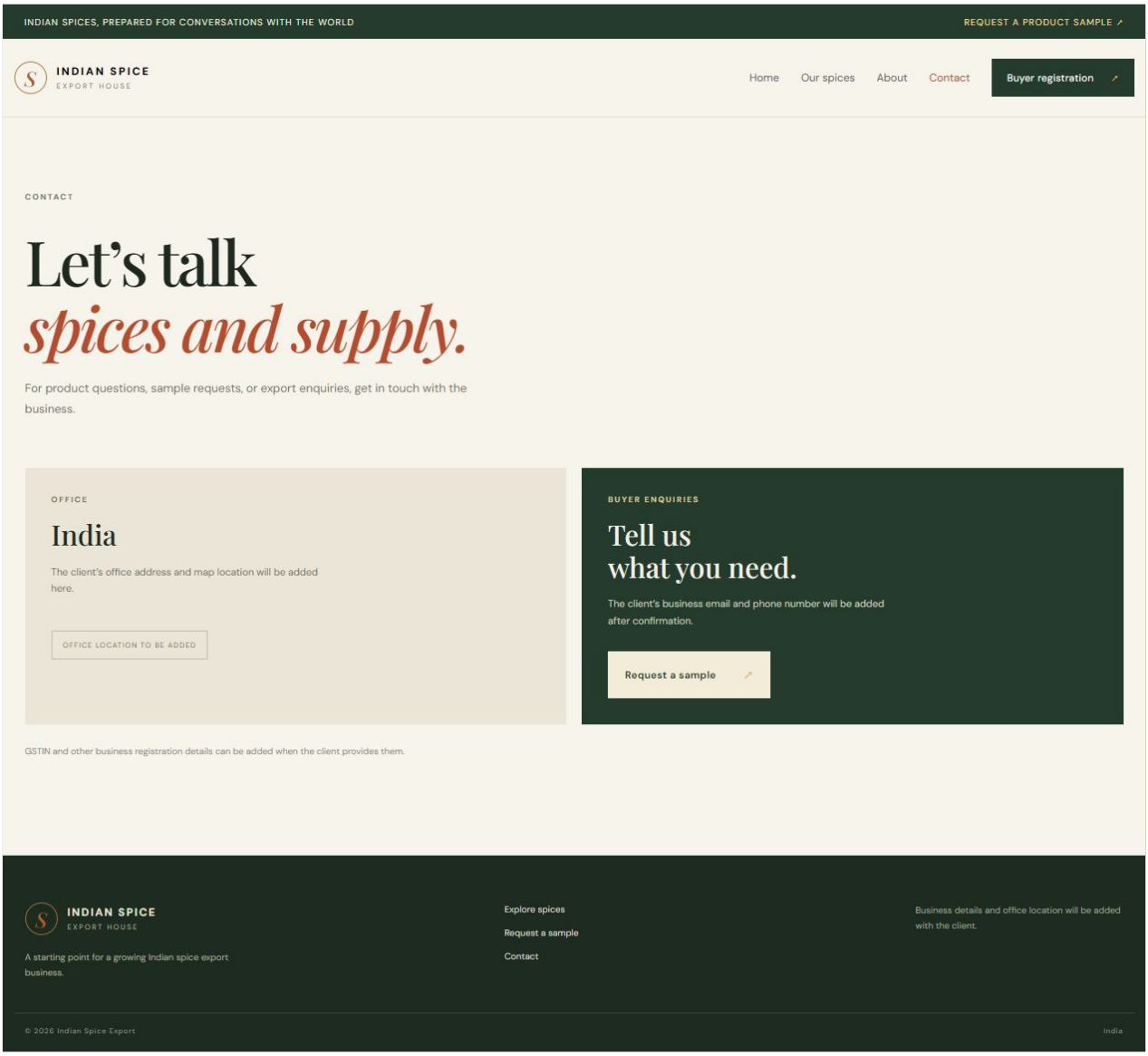


Figure 5.7: Buyer registration form (frontend demonstration).

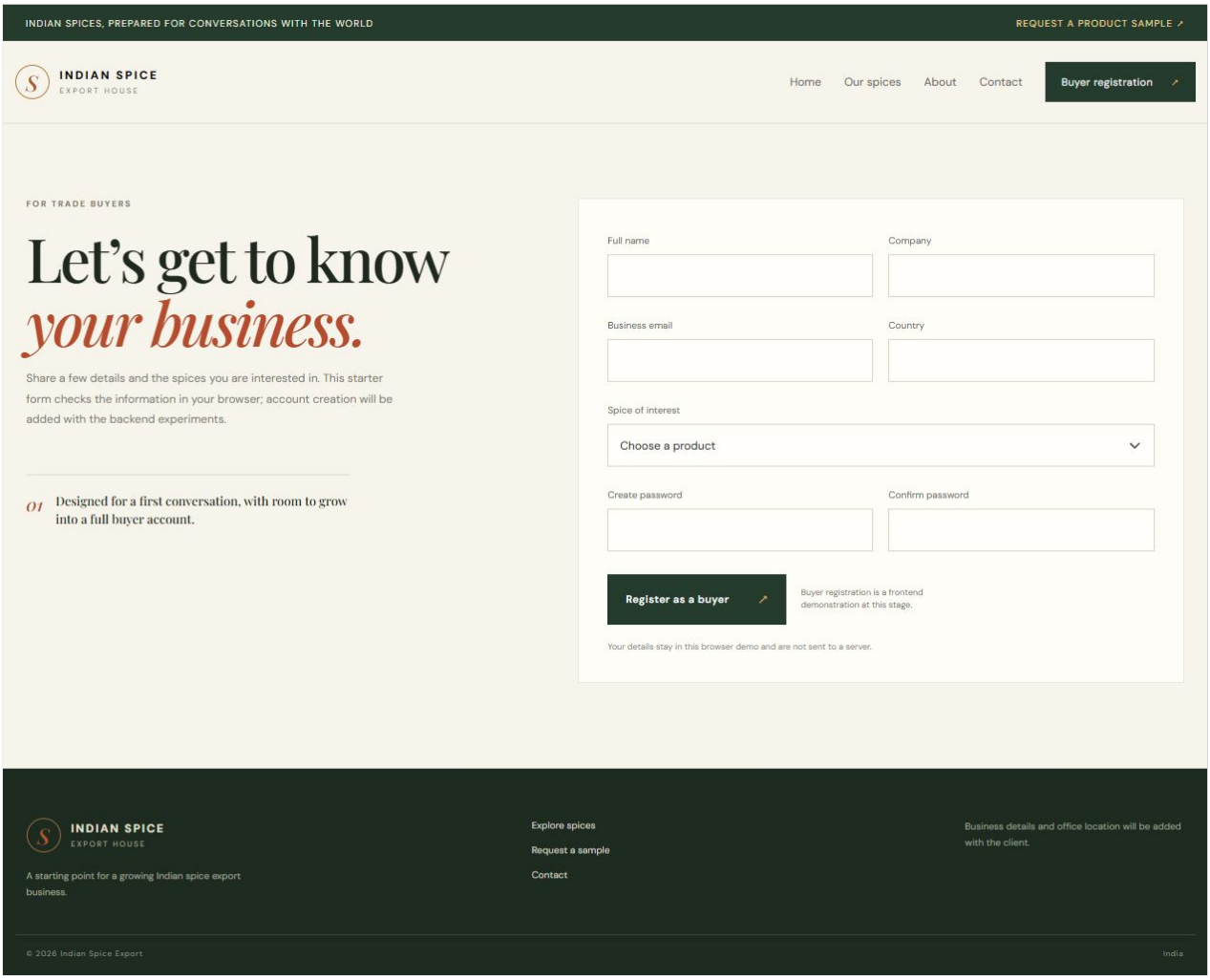
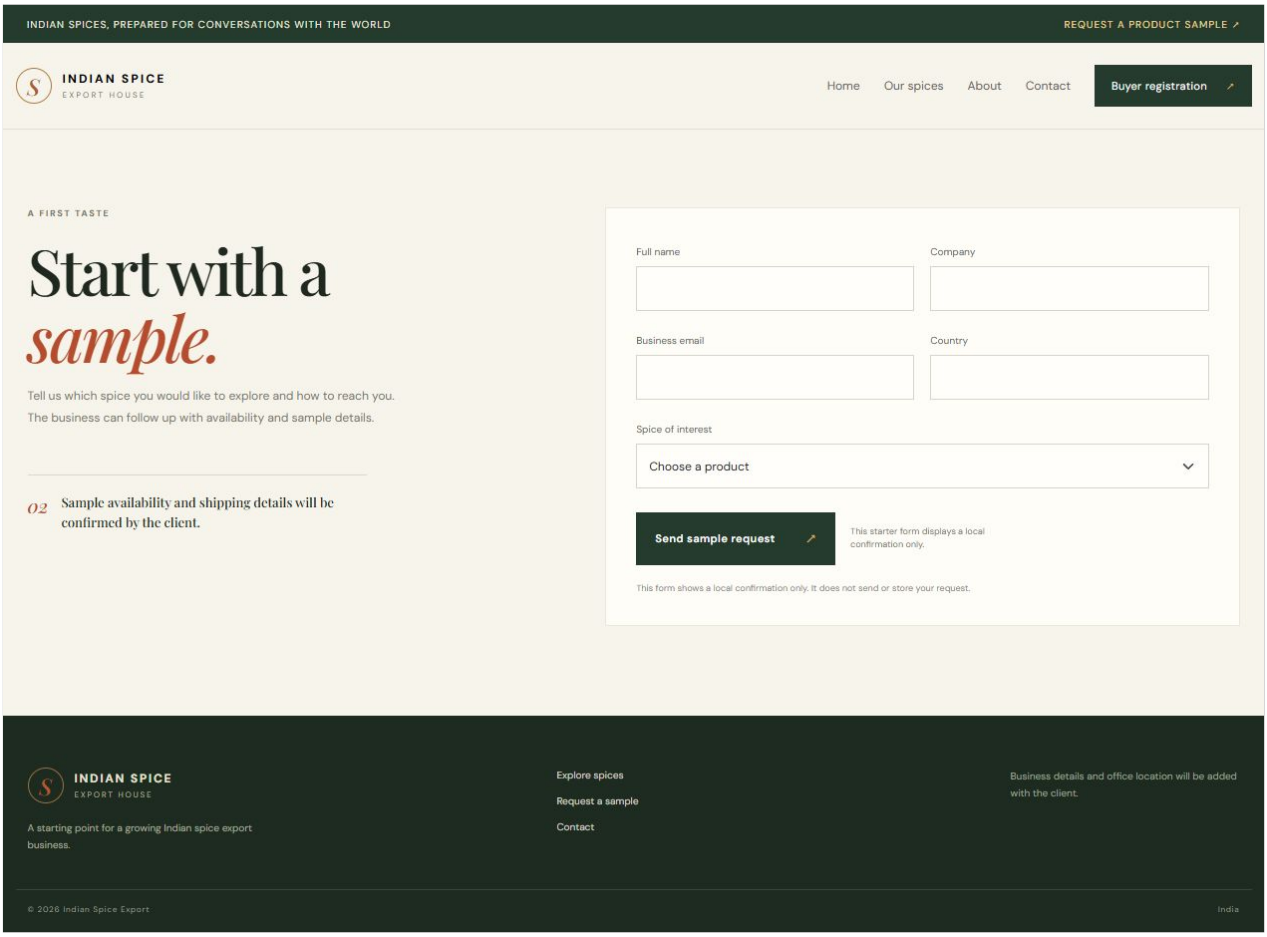


Figure 5.8: Sample request form (frontend demonstration).



Conclusion:

The experiment translated spice catalogue and buyer-enquiry requirements into Mongoose schemas and MongoDB collections. The actual local Mongoose validation run confirmed that a complete product object passes and that missing required values are identified, providing backend evidence for the model rules. This is schema-level validation; no document was persisted and no live CRUD result is claimed. The procedure records the database operations to perform once MongoDB Community Server or Atlas is configured. The distinction matters because a model can validate an object before save, but only a reachable database can demonstrate insert, query, update and delete. The schemas now define the data contract used by the Express controllers in the next experiments.

The schema should match the client's confirmed catalogue before seed data is treated as official. For a classroom CRUD run, use a clearly named test spice document, record the inserted identifier, verify the changed field after update, and delete only that test record. The screenshots in this revision must distinguish those database operations from the current Mongoose validation evidence so a reviewer can see exactly which stage has run.

SHUBHAM DHENGALE - 3574

Experiment No. 6

Title: Develop a Node.js Application using Express.js and Mongoose to Connect with MongoDB

Aim:

To run an Express server for the spice-export website, connect it to MongoDB with Mongoose and provide API routes for products and sample requests.

Tools Used:

Node.js, Express.js, Mongoose, MongoDB, npm, Visual Studio Code, browser or Postman.

Methodology:

The backend separates HTTP concerns from data access. Express middleware parses JSON and configures CORS, then mounts product and sample-request routers. The health route returns a small JSON response that can be checked without reading MongoDB; product and sample routes call controllers that use Mongoose. The backend was started locally by mounting the project Express app without running the database-connecting entry point. Browser requests verified the real health response and the not-found handler. Database-backed routes require MONGODB_URI and are not described as successful until that service is configured.

Theory:

Node.js executes JavaScript outside the browser. Express adds HTTP routing, middleware and response helpers.

Mongoose connects the server to MongoDB and maps schemas to models. CORS controls which browser origin can call the API. The server opens a database connection at startup and reuses it for requests.

Procedure:

1. Inspect the Express app, database connector, route modules, controllers and Mongoose models.

2. Install the project dependencies and place MONGODB_URI in a private local environment file before database testing.

3. Configure CORS for the approved frontend origin and enable JSON request parsing.

4. Register the /api/health route and mount /api/products and /api/samples routers.

5. Start the API entry point when MongoDB is reachable; for health-only inspection, run the Express app without claiming a database connection.

6. Send GET /api/health and confirm the JSON service status and HTTP 200 response.

7. Send a request to an unknown path and confirm the JSON 404 response.

8. After MongoDB is configured, request the product route and submit a valid sample request.

9. Record the API response and database state separately so a health check is not mistaken for persistence.

10. Check the health route and unknown-route response separately from database-backed product routes.

11. Keep credentials in a local environment file and confirm that the server reports a clear connection error when the database URI is absent.

Code:

```
  app.use(cors({ origin: process.env.CLIENT_ORIGIN || 'http://localhost:5173' }));
```

app.use(express.json({ limit:'32kb' }));

app.get('/api/health', (_req,res) => res.json({status:'ok',service:'spice-export-api'}));

app.use('/api/products', productRoutes);

app.use('/api/samples', sampleRoutes);

await connectDatabase(); // uses MONGODB_URI before app.listen()

Output:

Expected health check when the API is running:

GET /api/health -> HTTP 200

{ "status": "ok", "service": "spice-export-api" }

Database-backed endpoints require a reachable MongoDB connection; no live database result is claimed in this report.

Figure 6.1: Express health endpoint output captured in the local project.

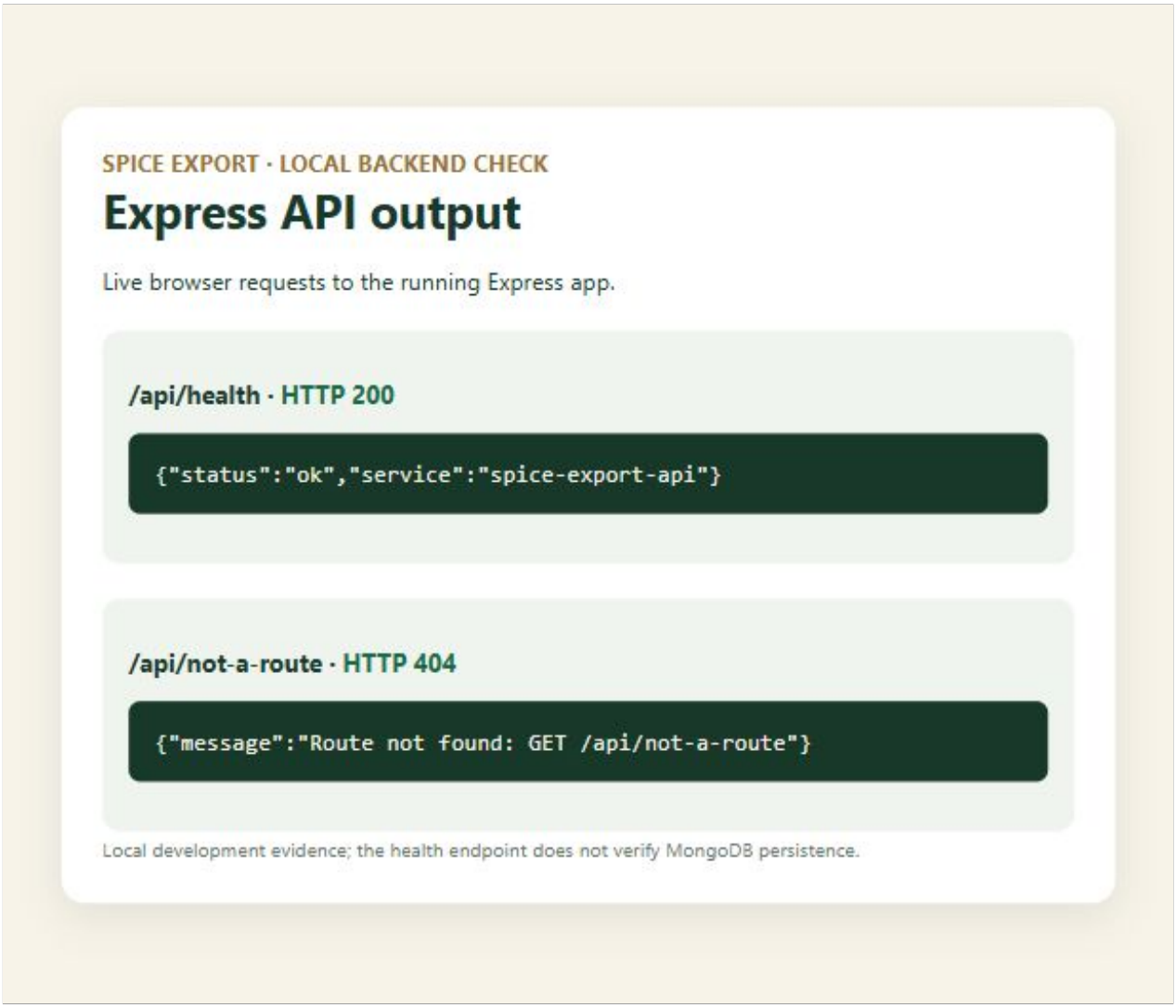


Figure 6.2: Mongoose model validation output without database persistence.

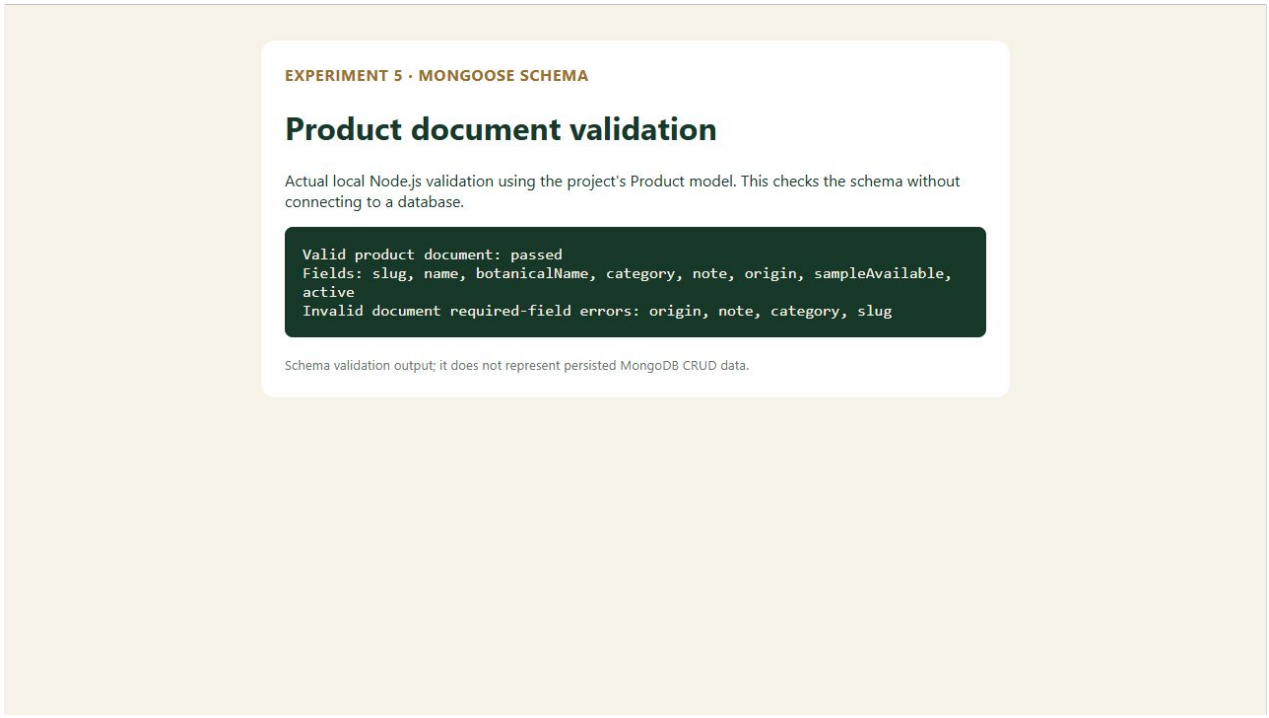


Figure 6.3: Node.js runtime output from the project script.

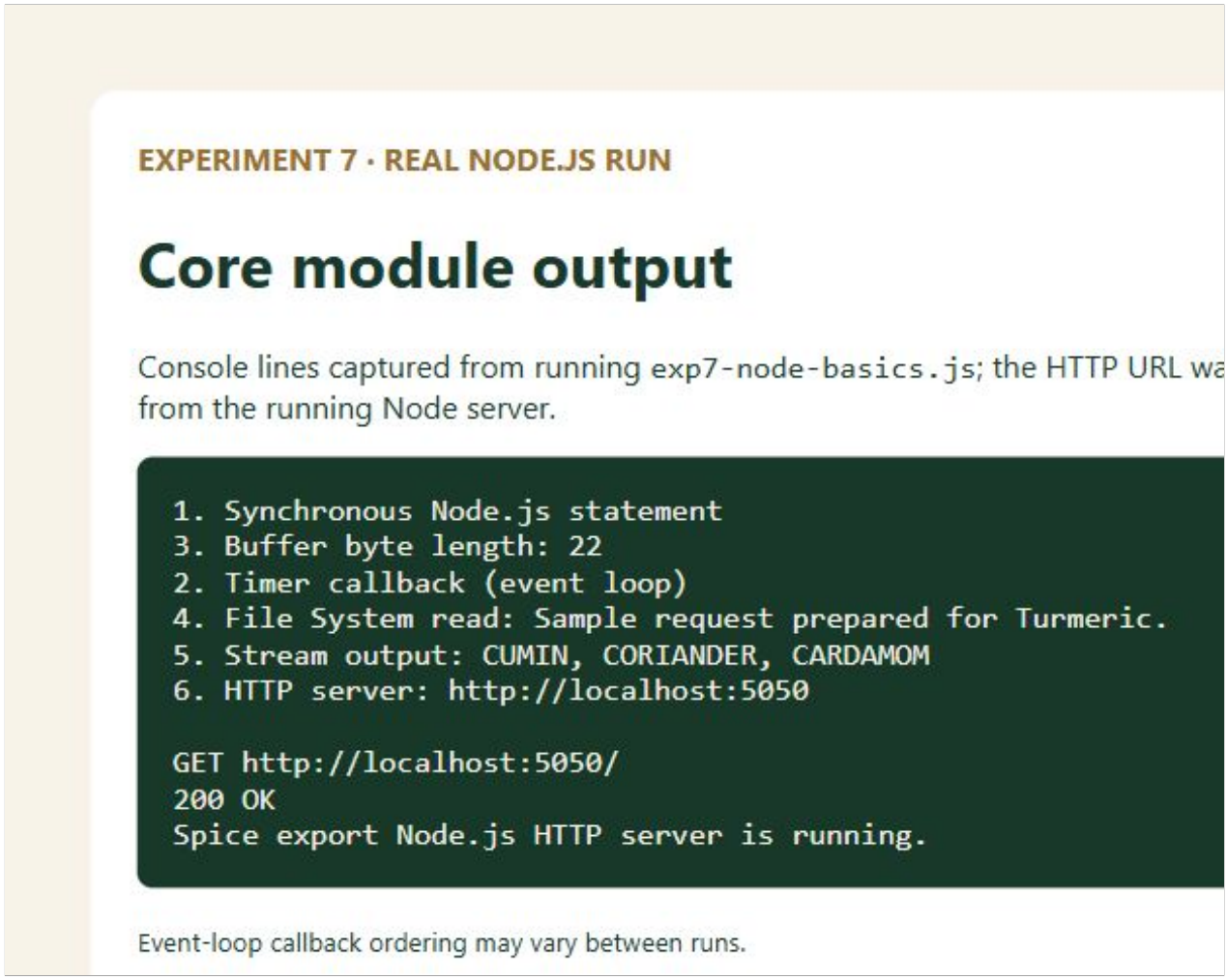


Figure 6.4: Client product route served by the local site.

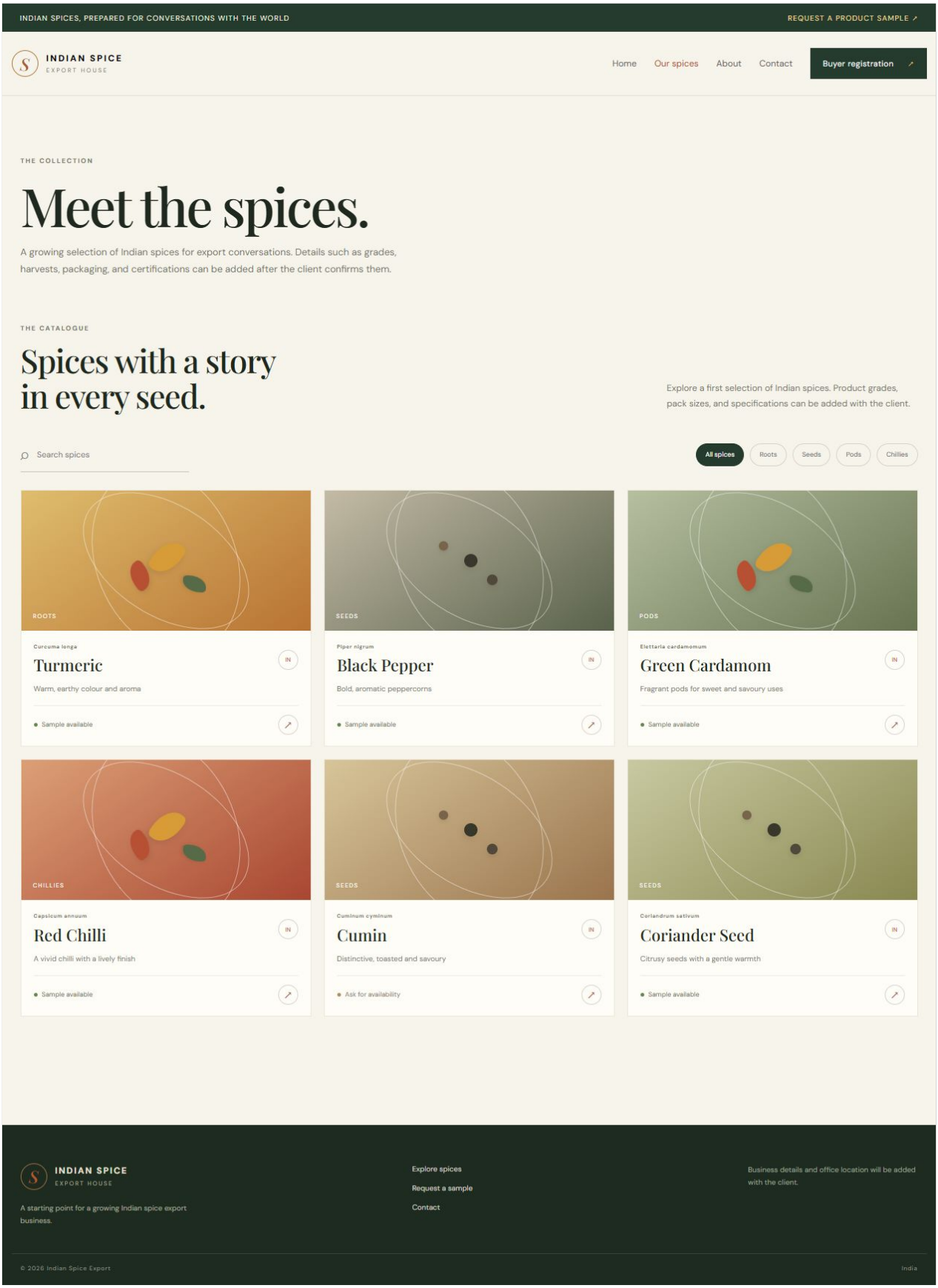


Figure 6.5: Sample-request user interface.

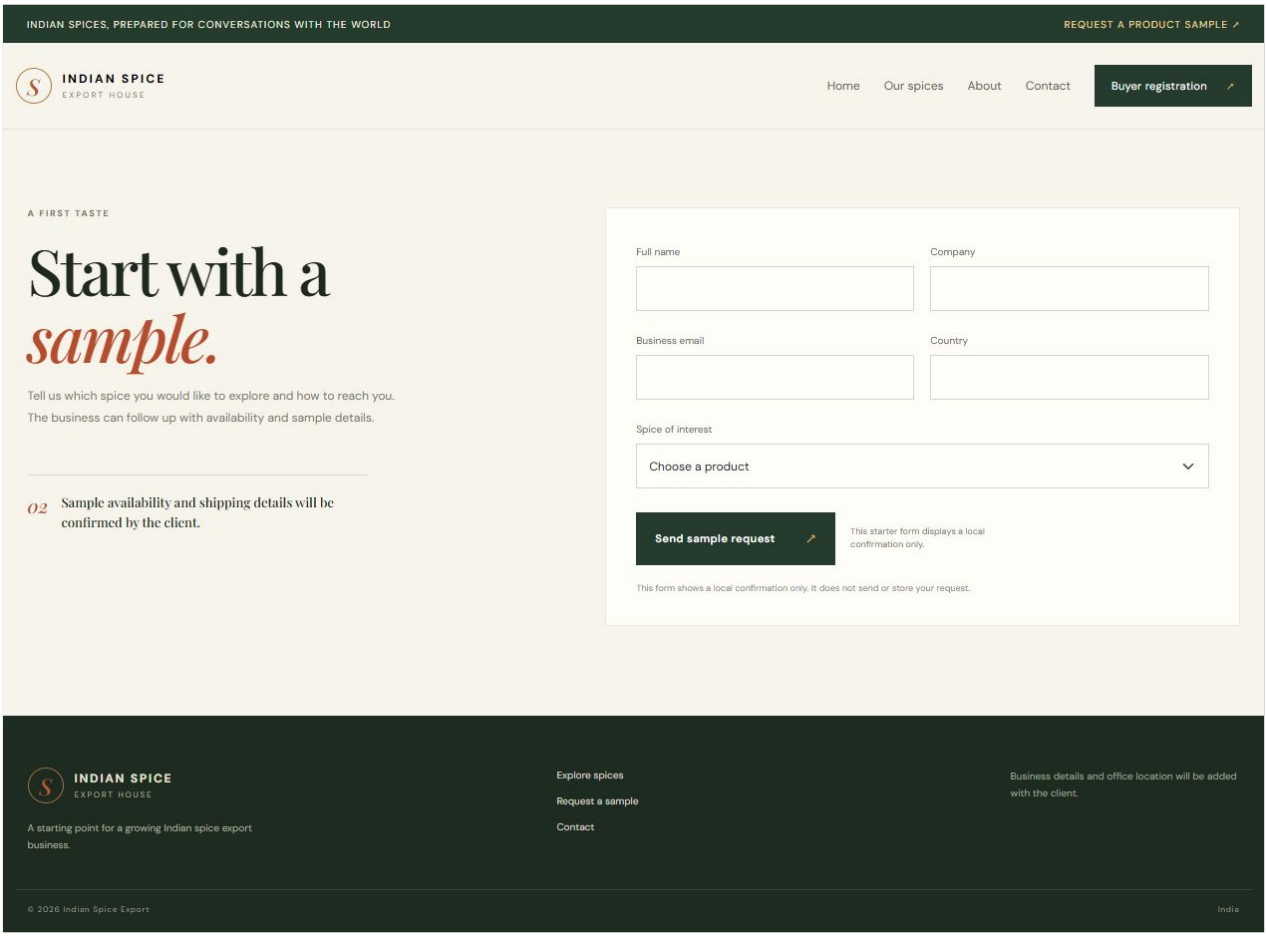


Figure 6.6: Buyer registration page.

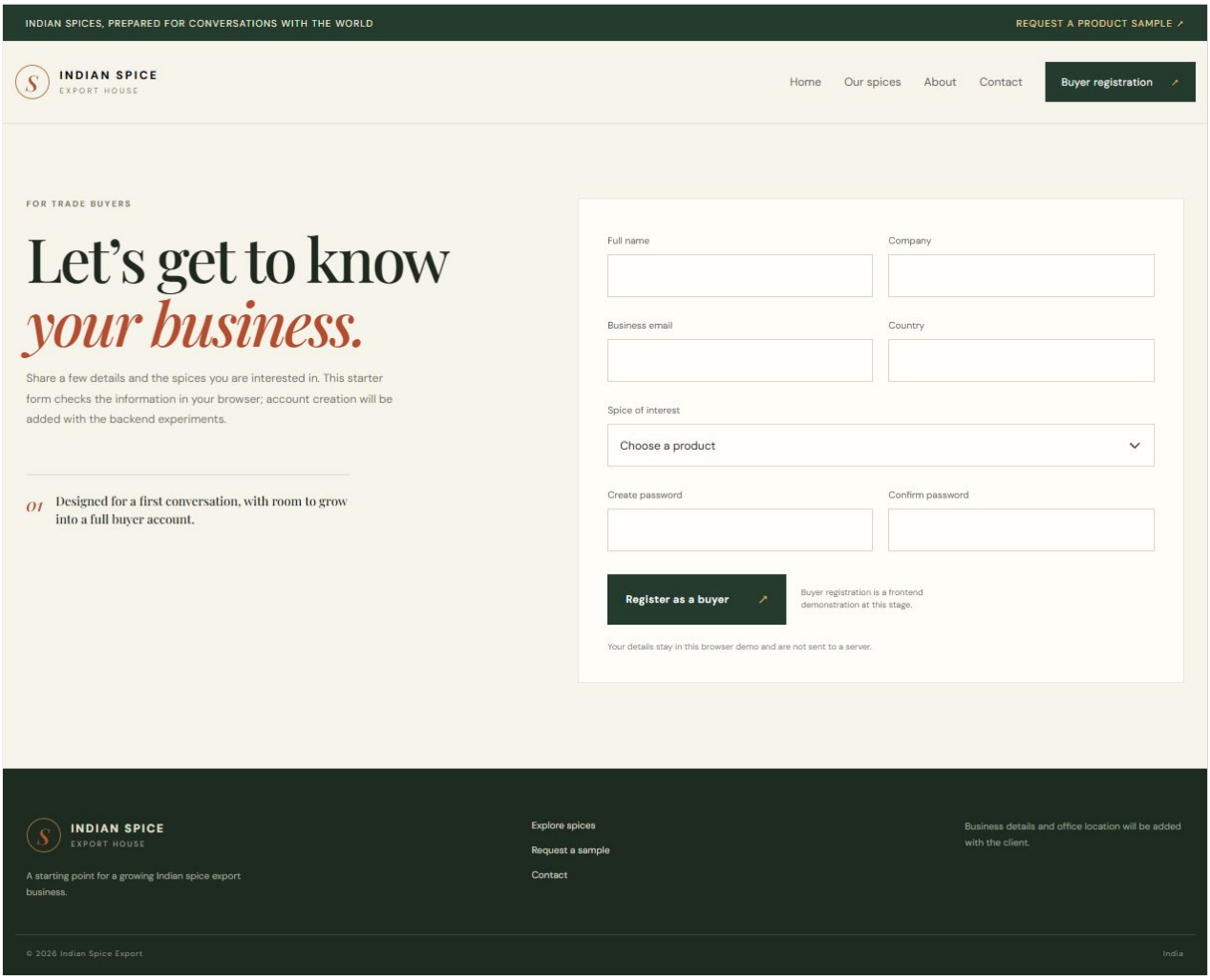


Figure 6.7: Contact route.

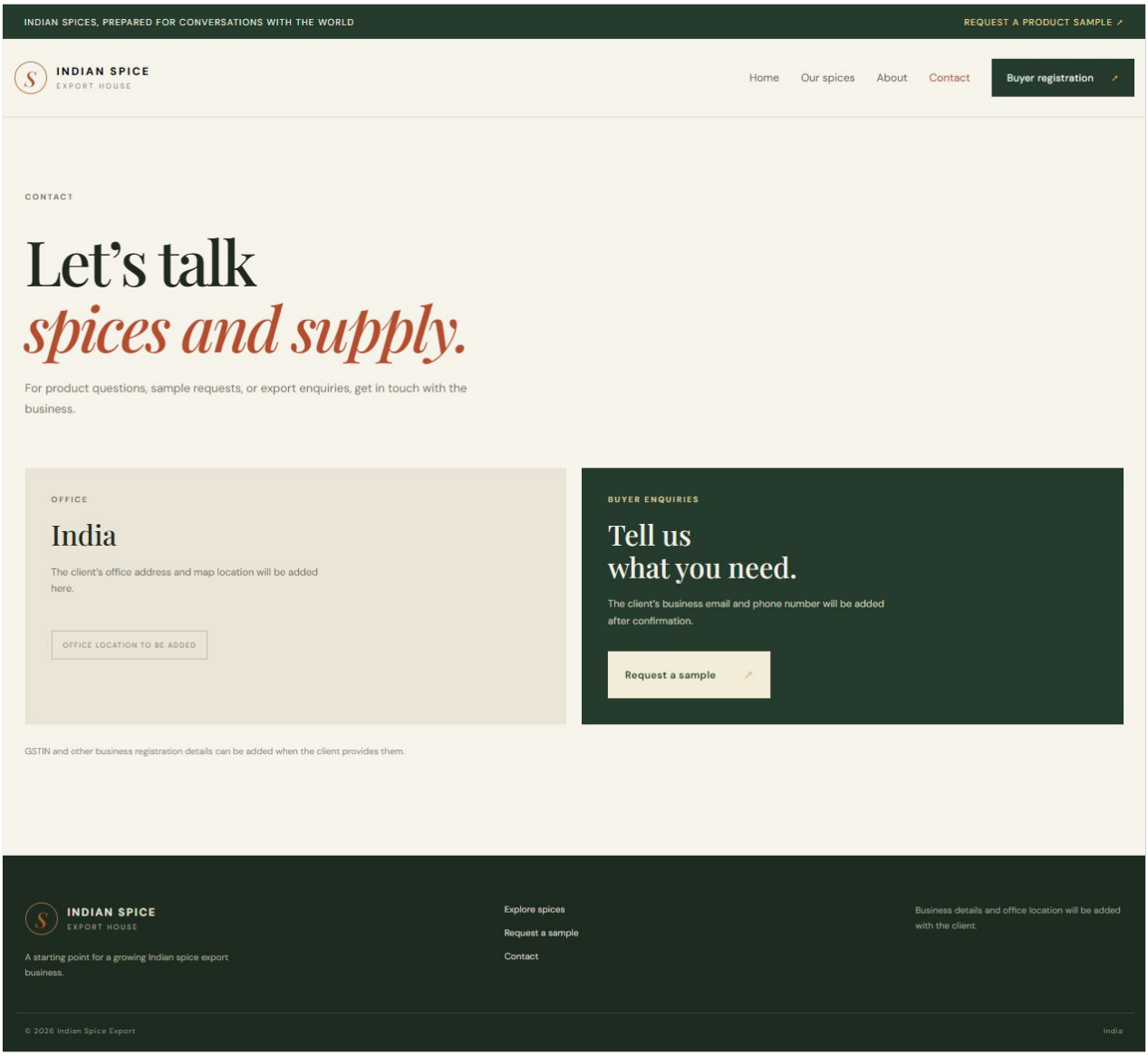
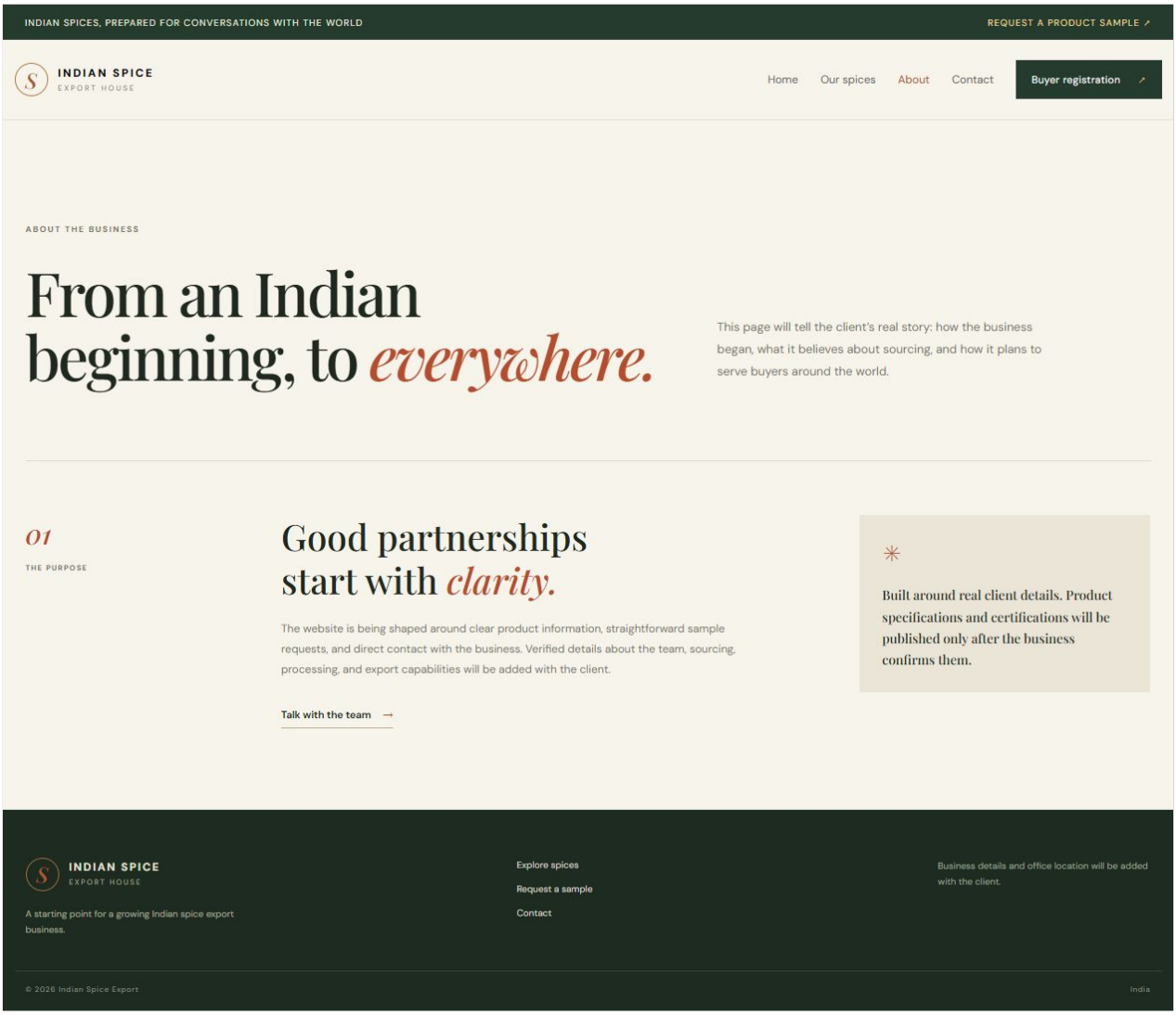


Figure 6.8: About route.



Conclusion:

This experiment connected the project's HTTP layer to its data models through Express routes and Mongoose controllers. The local API health check returned the expected service JSON, and the unknown path returned an HTTP 404 JSON message, confirming that the running app and its fallback handler respond. These browser captures are genuine local backend outputs. They do not prove that MongoDB is connected: the app can answer its health route without querying the database. To complete the database-backed part, configure MONGODB_URI, start the normal server entry point, then verify product retrieval and sample-request creation. The separation of health and database checks makes the current implementation status clear.

The health route is deliberately lightweight and useful for checking that the process is listening, but it should not be used as a database readiness signal. A later production health check can expose a safe dependency state without returning credentials or connection details. The route and controller separation keeps those checks independent from business operations and makes backend failures easier to locate.

SHUBHAM DHENGALE - 3574

Experiment No. 7

Title: Develop Basic Node.js Applications Demonstrating REPL, HTTP Server, File System, Buffers, Streams, and Event Loop

Aim:

To demonstrate Node.js REPL, HTTP, file-system, Buffer, stream and event-loop behavior with spice-export sample data.

Tools Used:

Node.js, built-in http, fs/promises, buffer and stream modules, terminal, Visual Studio Code.

Methodology:

This practical uses Node.js core modules with spice-export example data. The REPL accepts interactive JavaScript expressions; node:http creates a small HTTP server; node:fs/promises writes and reads a local enquiry note; Buffer reports the UTF-8 byte length; and Readable and Transform demonstrate a stream that uppercases a list of spices. A synchronous message and timer callback make event-loop scheduling visible. The supplied program was run locally, its console output inspected, and the HTTP response requested from the running server. Each result is tied to the corresponding Node API instead of being presented as a database or Express result.

Theory:

The REPL reads, evaluates and prints JavaScript expressions. Node core modules provide HTTP, file and stream operations without additional packages.

Buffers represent bytes, streams process data in chunks and the event loop schedules asynchronous callbacks after synchronous statements.

Procedure:

1. Open a terminal at the website project and start the Node.js REPL with node.

2. Evaluate a small spice array expression, inspect a value, and exit the REPL.

3. Run the exp7-node-basics.js demonstration from the project root.

4. Observe the synchronous statement and timer callback to compare event-loop scheduling.

5. Create a Buffer from the sample label and inspect its byte length.

6. Write the demonstration note with fs/promises, read it back and compare the returned text.

7. Pass the comma-separated spice list through the Readable and Transform streams and inspect uppercase output.

8. Request the program’s local HTTP URL in a browser and confirm the response text and status.

9. Stop the demonstration server after capturing the output; remove or retain the generated note as required by the lab.

10. Compare synchronous output with Promise and timer callbacks to observe event-loop ordering.

11. Confirm the file-system result and HTTP response, then close the server so the port is released.

Code:

```
import { createServer } from 'node:http';
```

```
import { readFile, writeFile } from 'node:fs/promises';
import { Buffer } from 'node:buffer';
import { Readable, Transform } from 'node:stream';
console.log('1. Synchronous Node.js statement');
setTimeout(() => console.log('Timer callback (event loop)'), 0);
const label = Buffer.from('Turmeric export sample', 'utf8');
await writeFile('./exp7-note.txt','Sample request prepared for Turmeric.\n','utf8');
const server = createServer((_req,res) => res.end('Spice export Node.js HTTP server is running.'));
server.listen(5050, () => console.log('HTTP server listening on 5050'));
```

Output:

Representative output from the supplied demonstration (stream and timer lines can interleave):

```
1. Synchronous Node.js statement
Buffer byte length: 22
Sample request prepared for Turmeric.
Stream output: CUMIN, CORIANDER, CARDAMOM
HTTP server listening on 5050
Timer callback (event loop)
Browser GET localhost:5050 -> Spice export Node.js HTTP server is running.
```

Figure 7.1: Node.js script output from the local runtime.

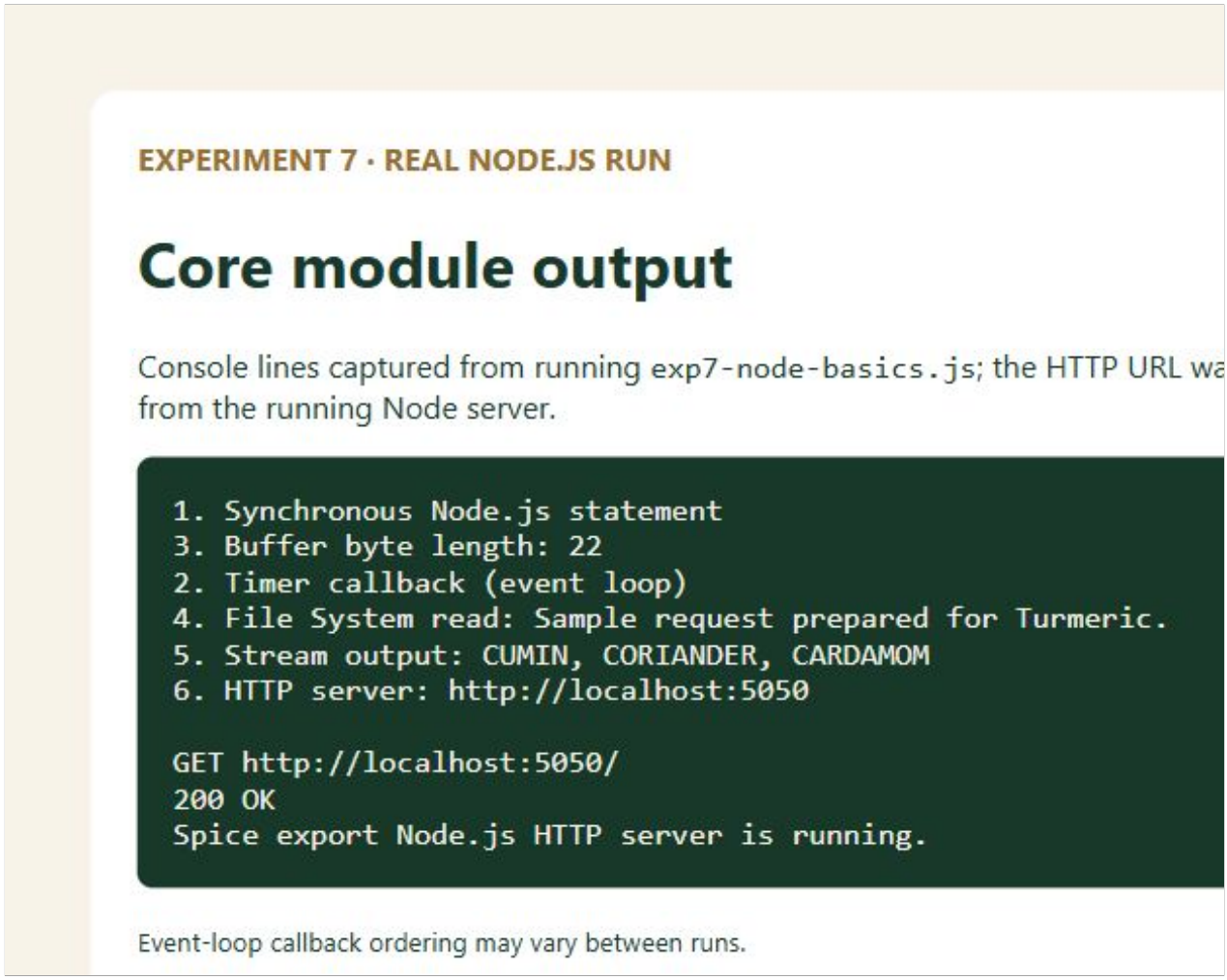


Figure 7.2: Express health endpoint output.

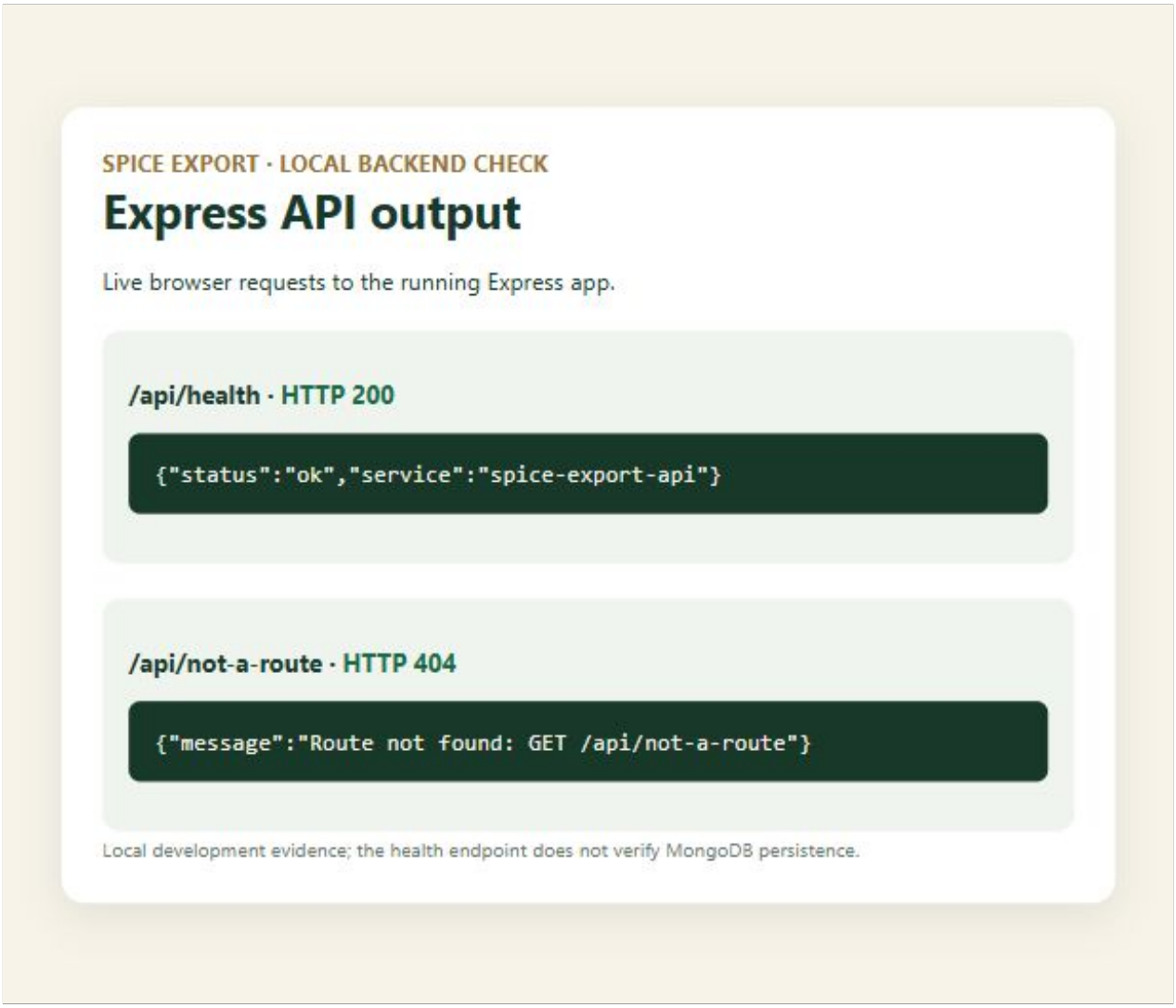


Figure 7.3: Mongoose schema validation output (no DB connection).

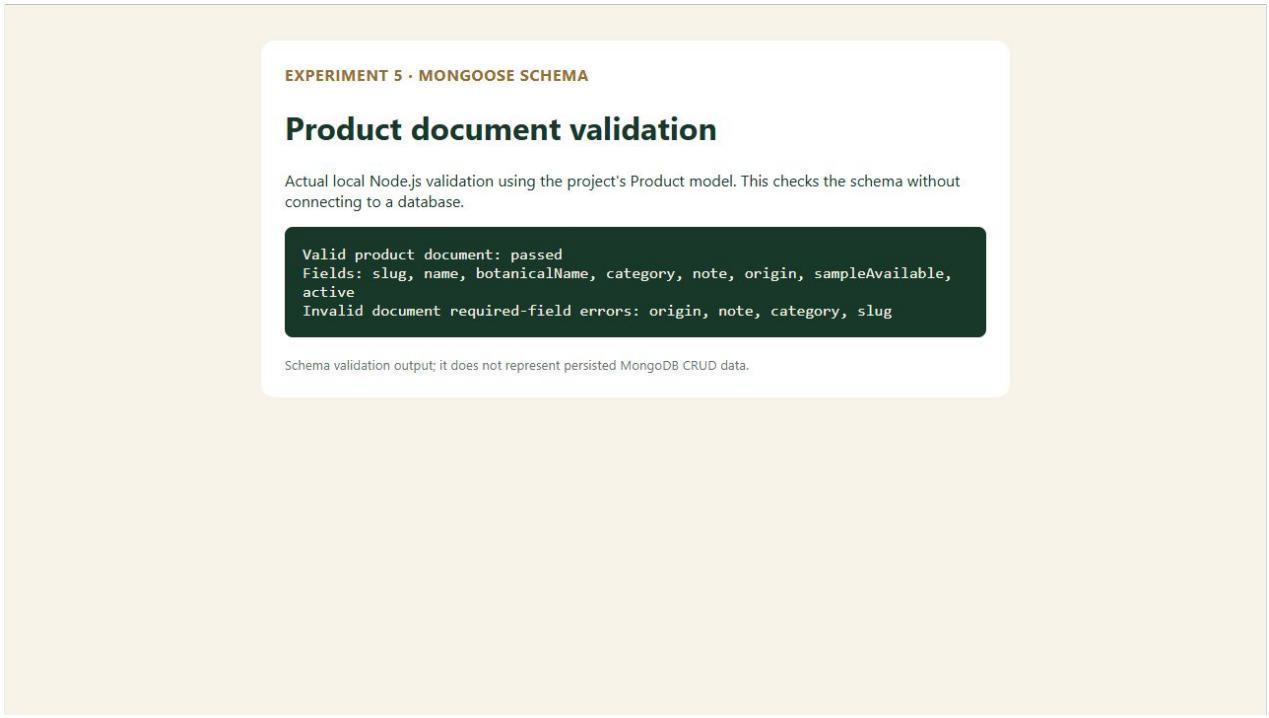


Figure 7.4: Project home page.

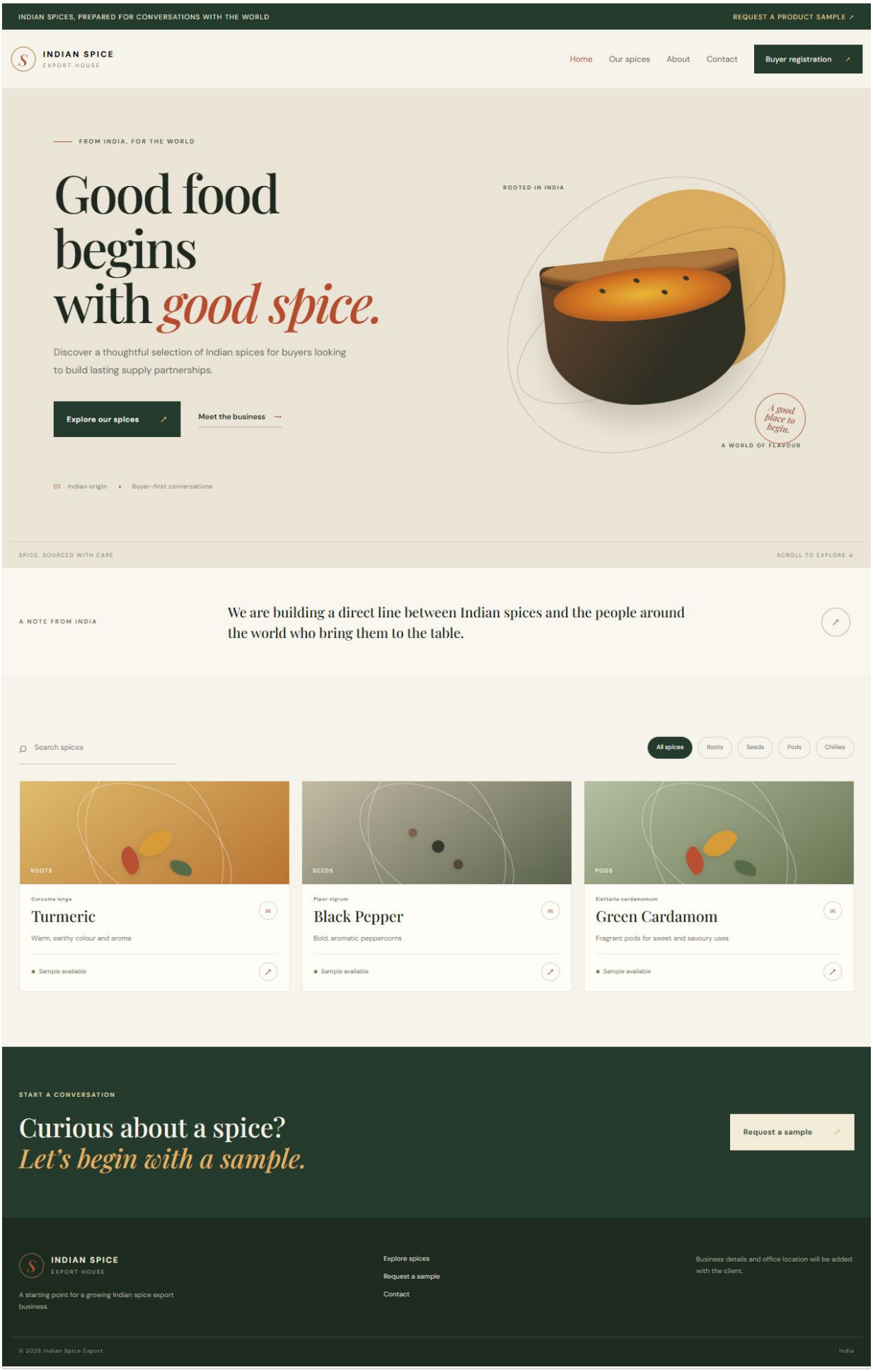


Figure 7.5: Product catalogue route.

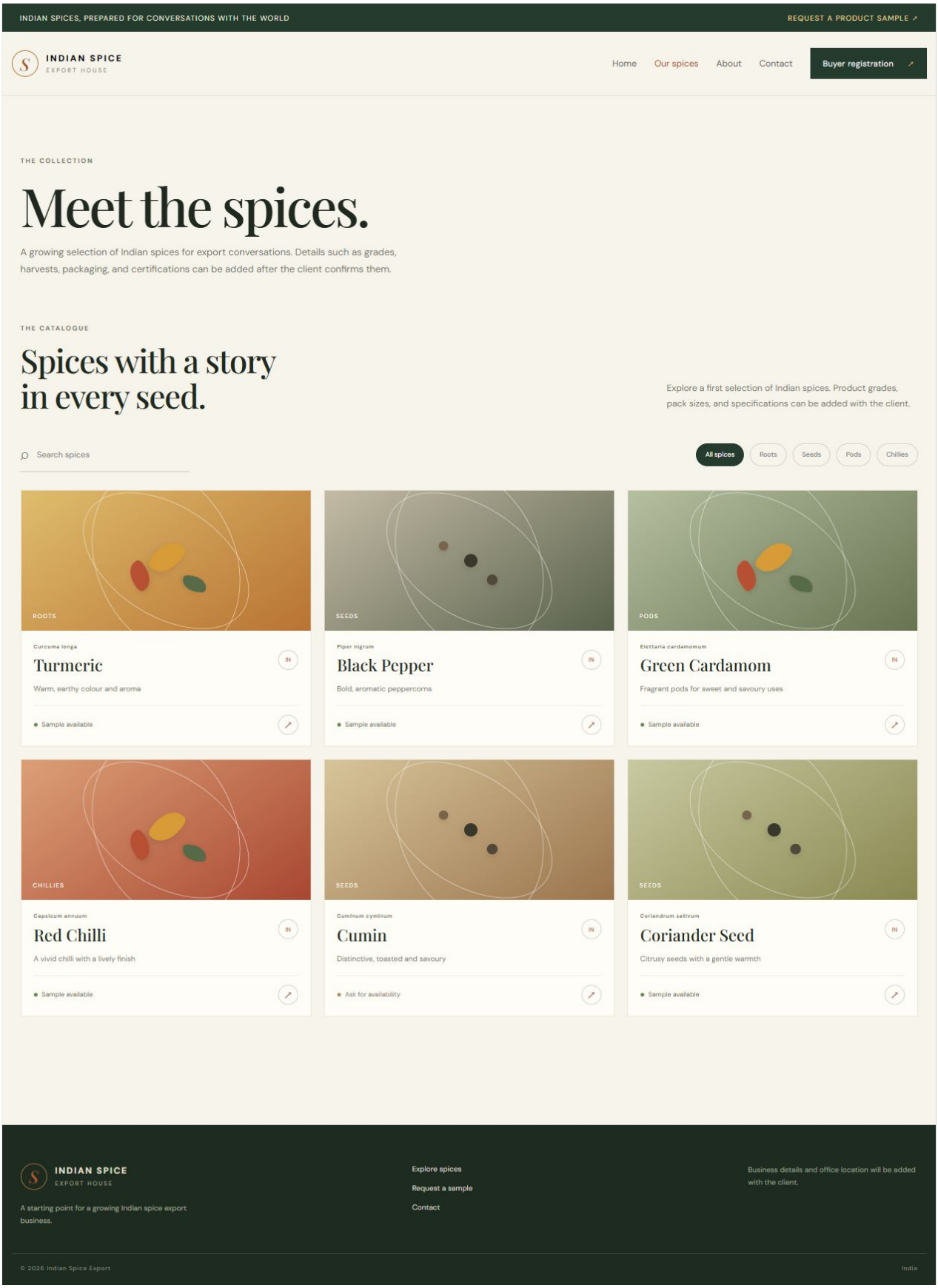


Figure 7.6: About route.

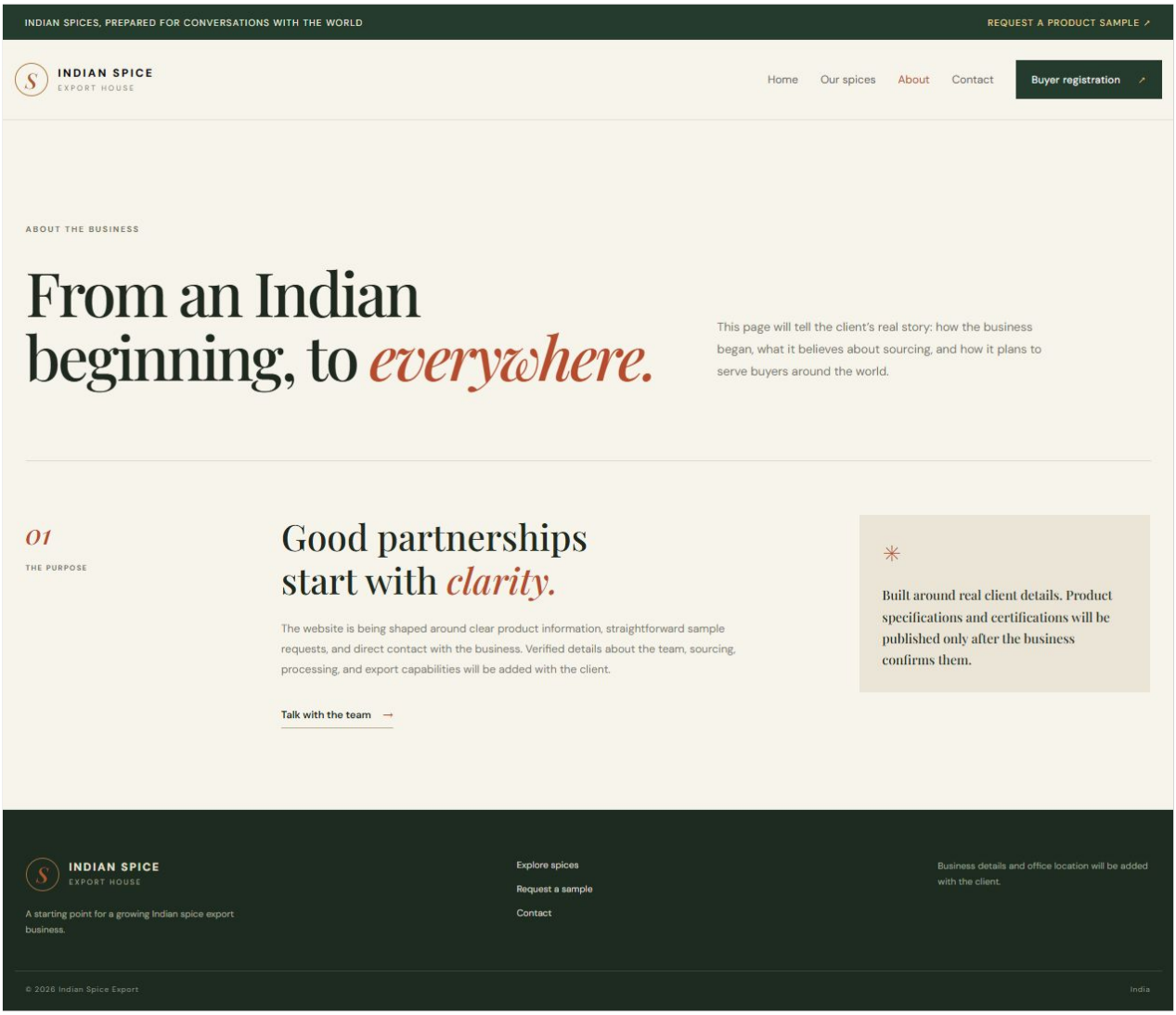


Figure 7.7: Contact route.

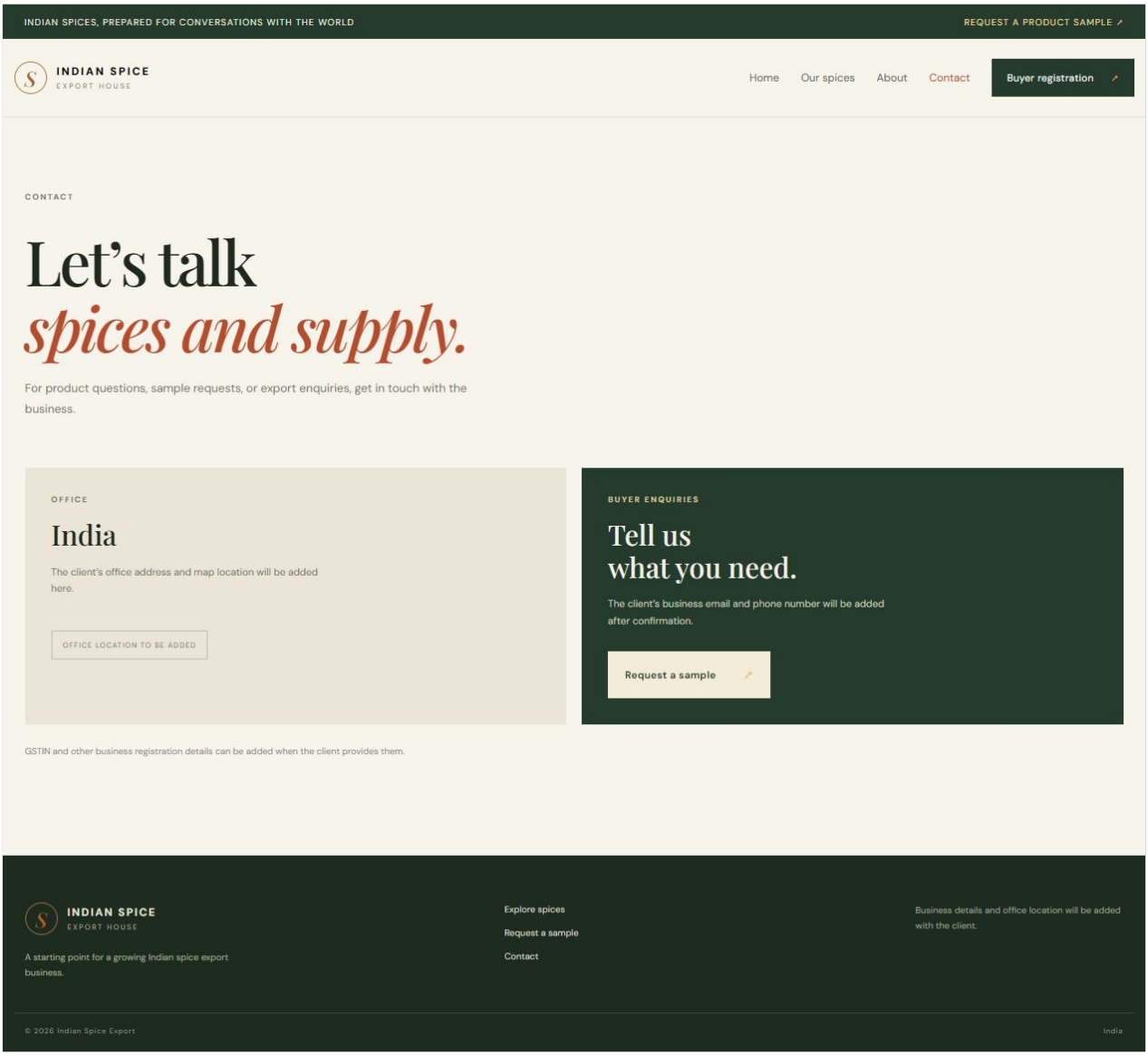
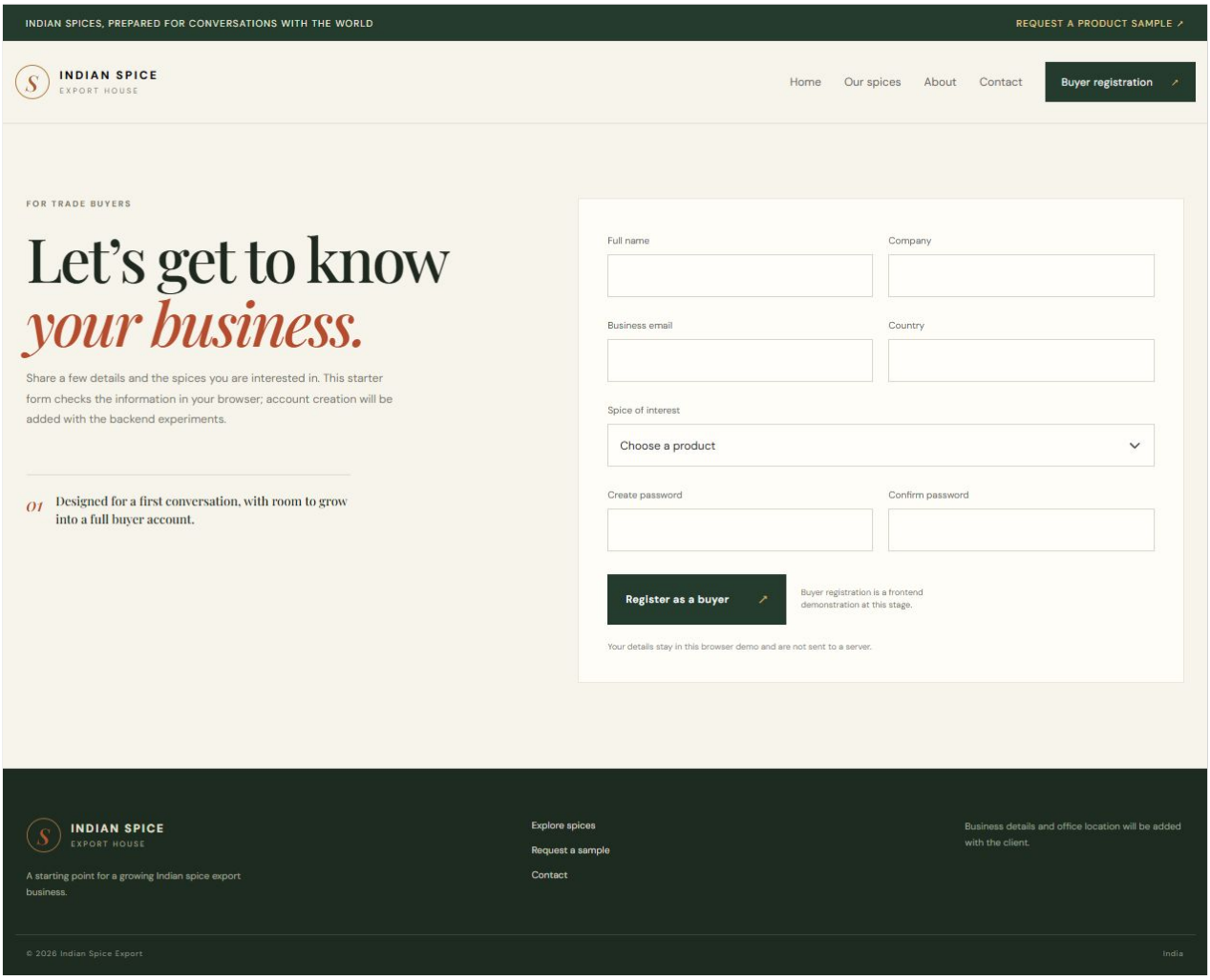


Figure 7.8: Registration route.



Conclusion:

The Node.js demonstration exercised the REPL and several built-in modules with practical spice-export examples. The run produced a byte count for the Buffer, wrote and read a local note, transformed spice names through a stream, and started a raw HTTP server that returned a browser-readable response. The event-loop lines showed that asynchronous callbacks are scheduled separately from synchronous statements; exact ordering can vary with runtime scheduling. These results were obtained from the supplied local script and are represented in the report's output capture. This is a Node core-module server, distinct from the Express API used in later experiments, and it does not connect to or save data in MongoDB.

The HTTP server in this exercise is deliberately small so the focus remains on the Node runtime rather than a framework. Its response, file output and stream transform are reproducible local observations, while the REPL gives a quick way to inspect a product array interactively. Running the script again rewrites its demonstration note, so the generated file should be treated as temporary practical output.

SHUBHAM DHENGALE - 3574

Experiment No. 8

Title: Build a RESTful API using Express.js and Test API Endpoints using Postman

Aim:

To expose spice products and buyer sample requests through REST endpoints and inspect methods, JSON responses and status codes in Postman.

Tools Used:

Node.js, Express.js, Mongoose, MongoDB, Postman and the project Postman collection.

Methodology:

The API design treats products and sample requests as resources and assigns HTTP methods to their operations. Express routes expose public health and product-reading paths, sample creation, and protected management actions. Controllers validate input and use Mongoose models; middleware handles JSON parsing, CORS, authorization and consistent errors. The project Postman collection records requests for repeatable checks. The local backend's health and unknown-route responses were tested in a browser, showing a real 200 response and a real 404 response. Product and sample database operations require the MongoDB service, and admin requests additionally require a private key.

Theory:

REST represents resources at URLs and uses GET, POST, PUT and DELETE to read, create, update and remove data.

HTTP status codes report outcomes: 200 for successful reads, 201 for creation, 400 for invalid input, 404 for missing routes or records, and 500 for unexpected errors.

Postman sends requests and displays status and JSON responses. Admin keys belong in private local variables and must not appear in screenshots.

Procedure:

1. Start the Express application and configure the local API base URL in Postman.

2. Import the Spice-Export-API collection from the project postman folder.

3. Set any admin key only in a private local variable; do not include it in a screenshot or report.

4. Send GET /api/health and verify status 200 and the JSON service name.

5. Send GET /api/products after MongoDB is available and inspect the returned product array.

6. Submit a valid POST /api/samples request and verify the creation status and response fields.

7. Try an invalid request body and inspect the validation status and message.

8. Test a protected management route without credentials and confirm it is rejected; repeat only with an authorized local key.

9. Check an unknown route for a 404 response and save the genuine Postman request/response evidence.

10. Send valid and invalid requests and inspect response codes, JSON bodies and validation messages.

11. Repeat database-dependent requests only after the API server has connected to the configured test database.

Code:

```
  app.get('/api/health', (_req,res) => res.json({status:'ok',service:'spice-export-api'}));
```

app.use('/api/products', productRoutes);

app.use('/api/samples', sampleRoutes);

router.get('/', listProducts);

router.post('/', createSampleRequest);

router.put('/:id', requireAdmin, updateSampleRequest); // protected management route

Output:

Expected responses with the API and database configured:

GET /api/health -> 200 OK { "status":"ok", "service":"spice-export-api" }

GET /api/products -> 200 OK with product documents from MongoDB

POST /api/samples with valid JSON -> 201 Created with the saved request

Live database responses require configured MongoDB and, for management calls, the admin key.

Figure 8.1: Express health response from the local backend.

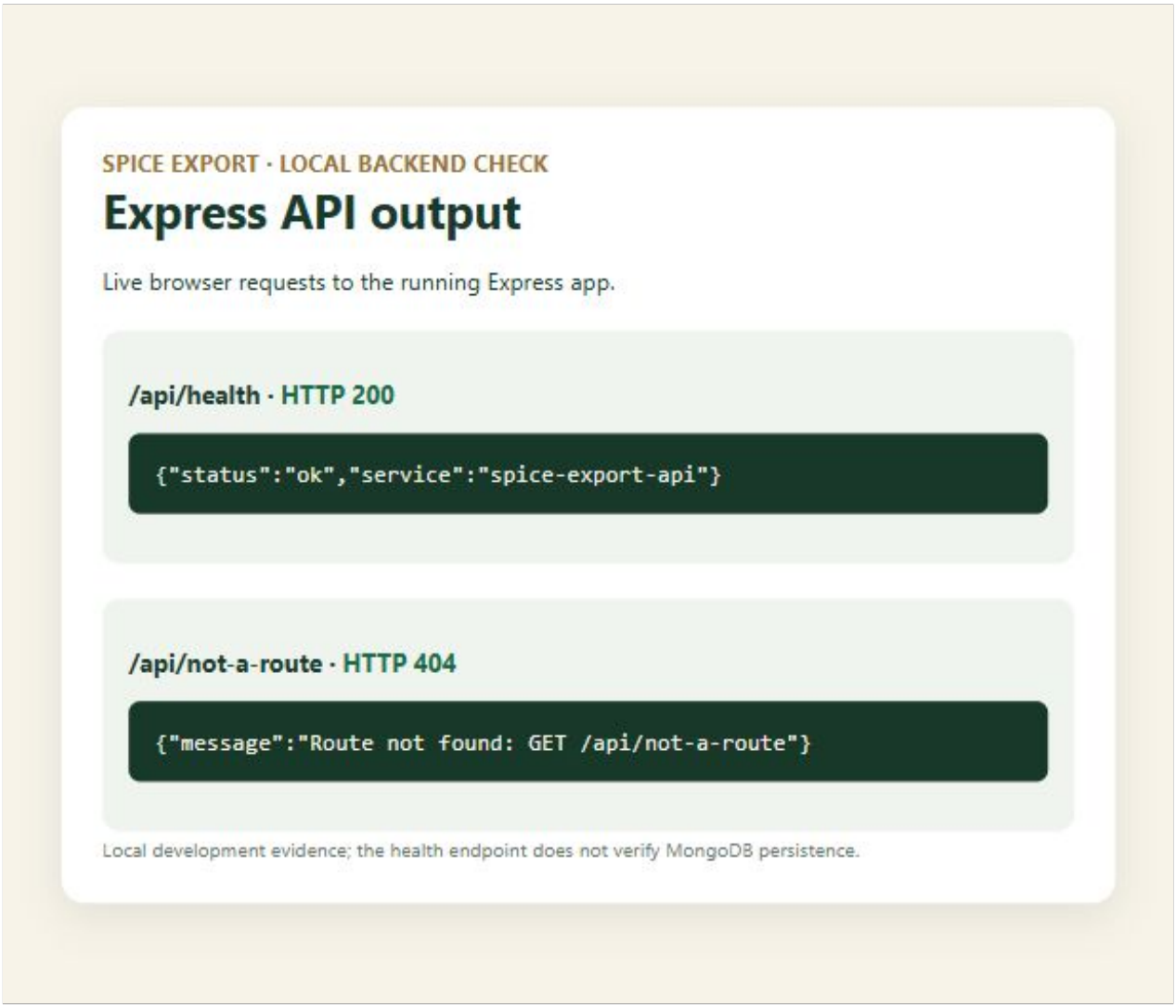


Figure 8.2: Mongoose validation output; CRUD persistence not configured.

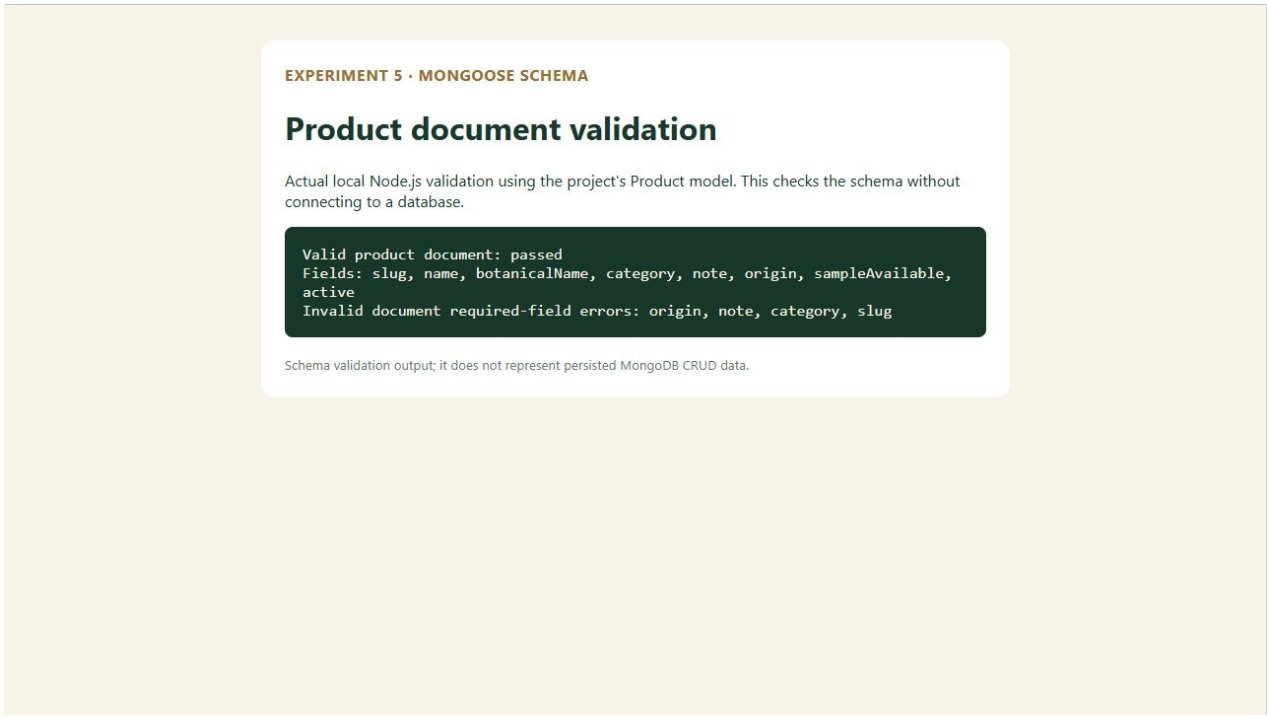


Figure 8.3: Product collection UI route.

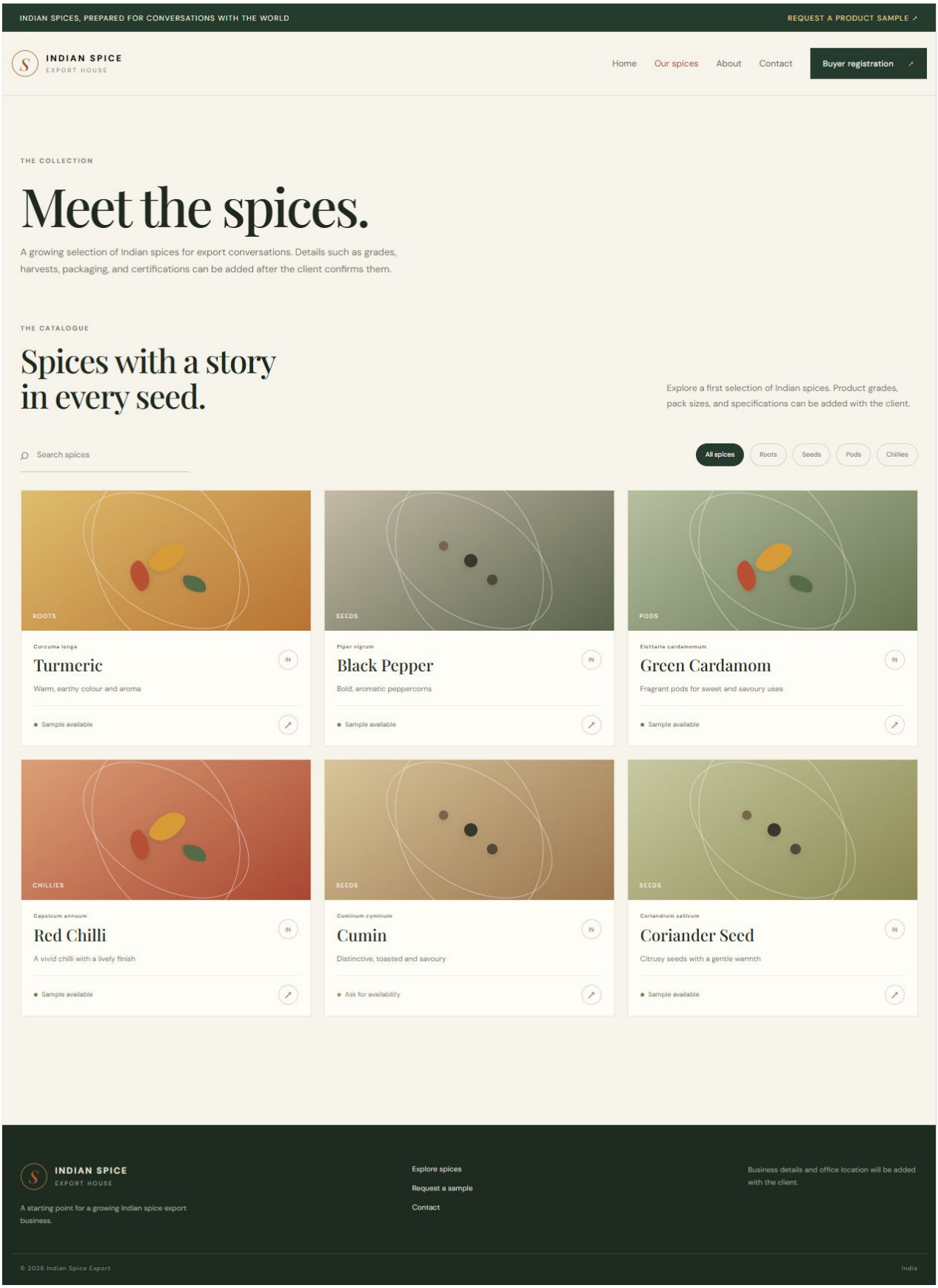


Figure 8.4: Category filter state.

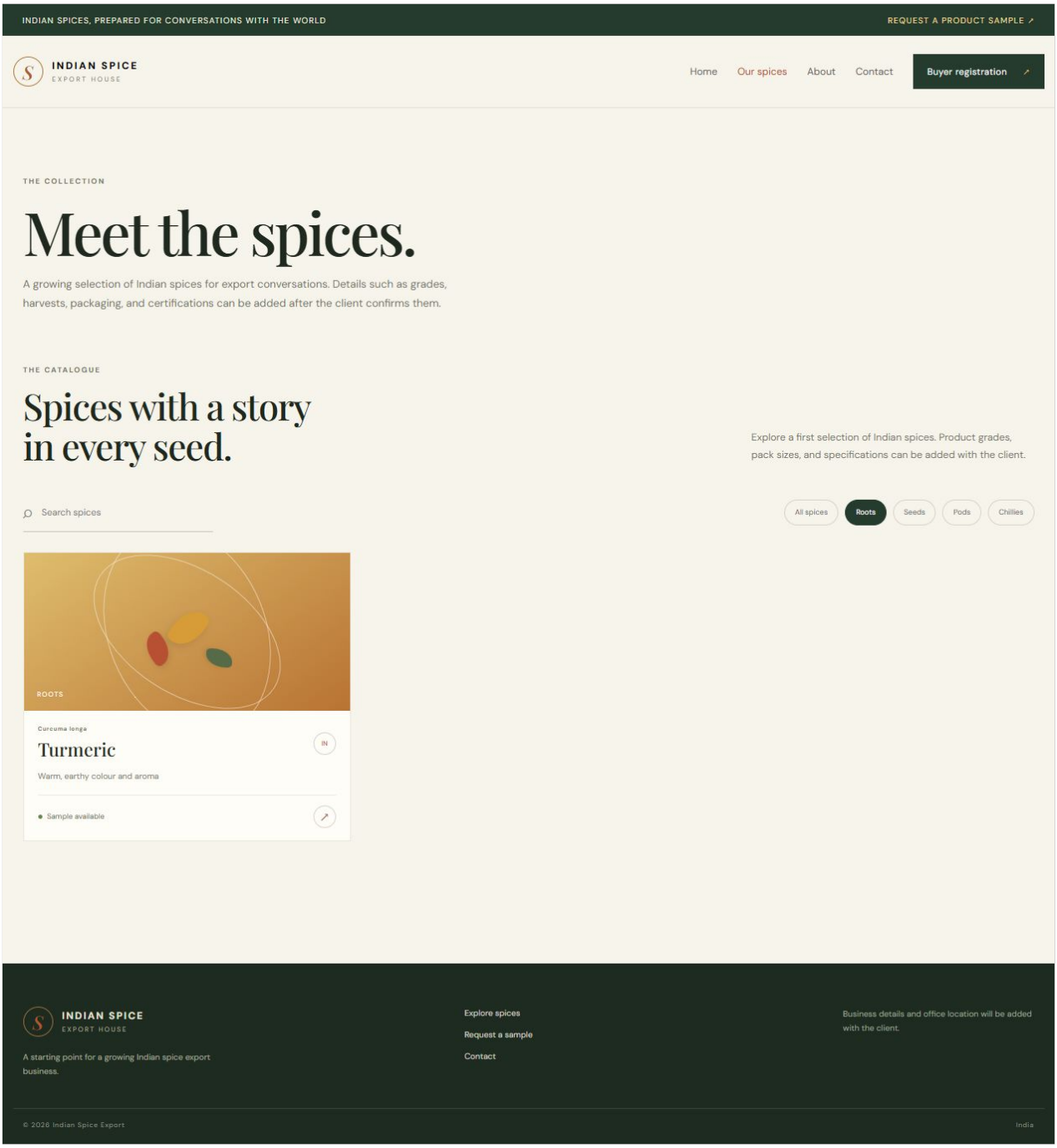


Figure 8.5: Search filter state.

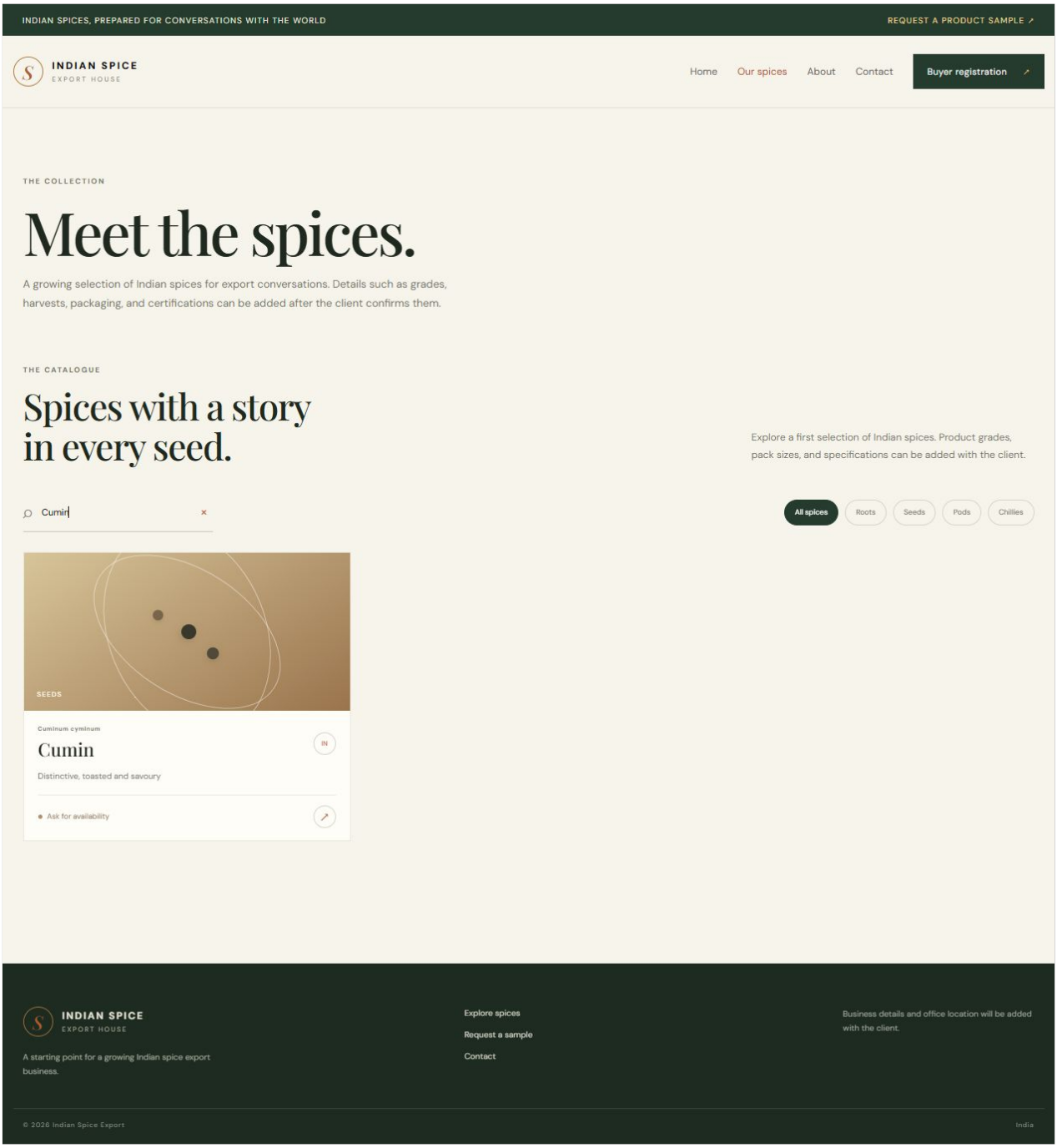


Figure 8.6: Sample request page.

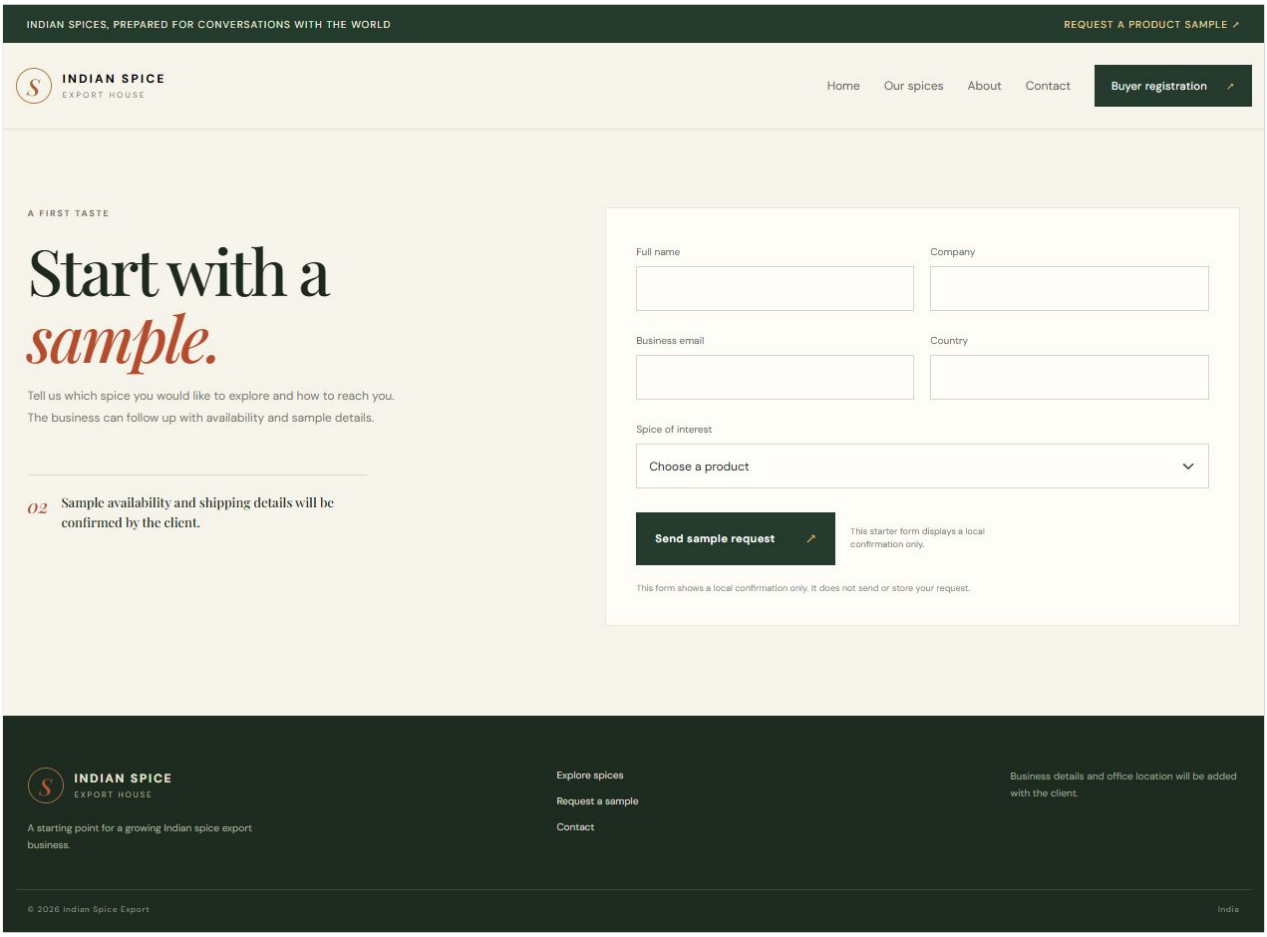


Figure 8.7: Registration page.

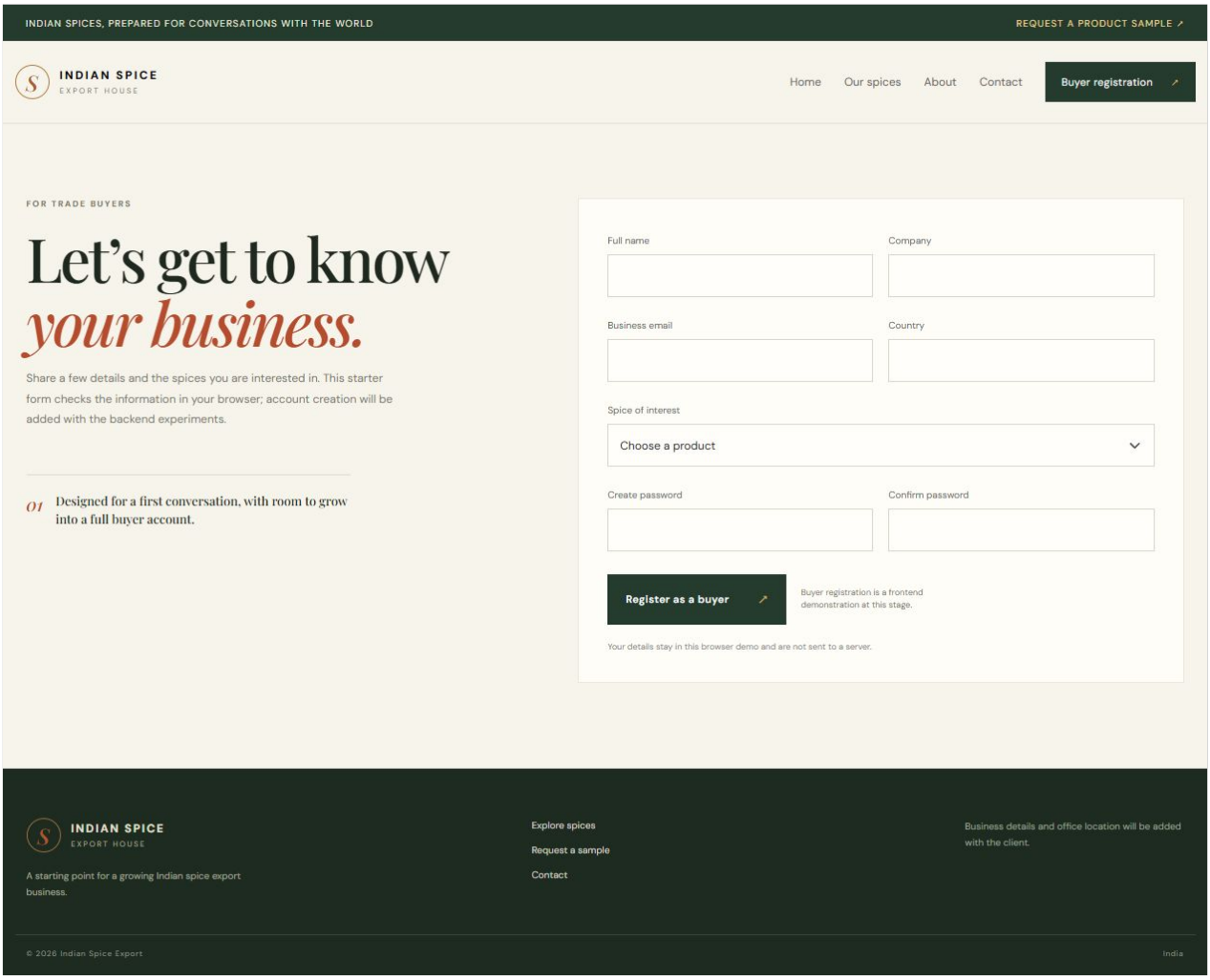
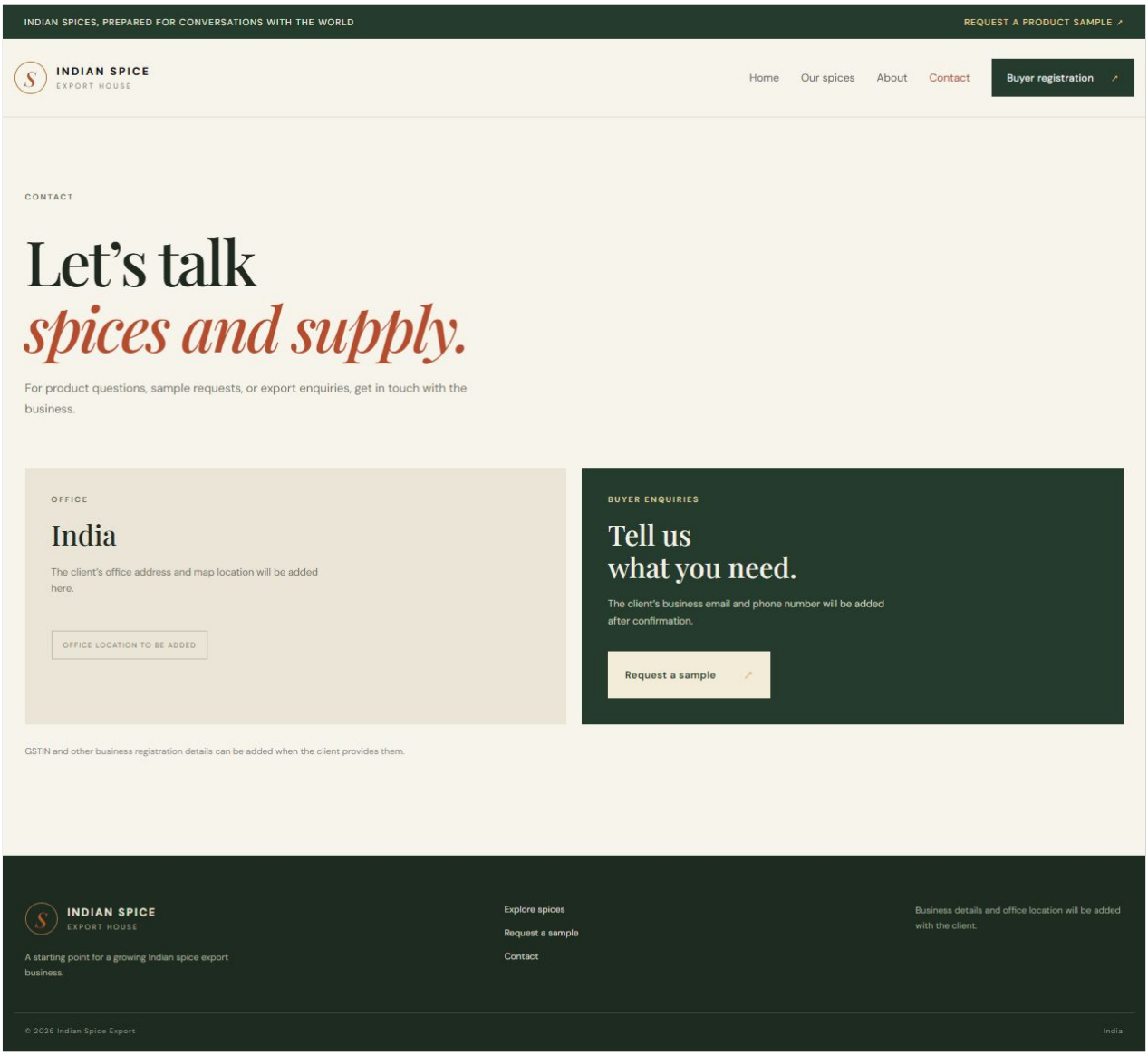


Figure 8.8: Contact page.



Conclusion:

The REST API experiment organized the spice-export backend around HTTP resources, JSON request bodies and status codes. The local Express output confirmed the health route with HTTP 200 and the not-found handler with HTTP 404, demonstrating both a successful request and a controlled error response. The Postman collection provides repeatable definitions for product and sample workflows. Database-backed product reads and sample creation still need a configured MongoDB connection, while protected management calls require the client's authorized admin key. The screenshots therefore document the working local API behavior without claiming that a record was stored. Once configuration is supplied, the remaining endpoint cases can be recorded with their actual status and response bodies.

A complete API record should include the method and URL, request body where relevant, response status, response JSON and a note about database state. Credentials must stay in private local variables and must not appear in practical screenshots. Once MongoDB is available, the existing collection covers successful reads and writes; validation failures, missing records and unauthorized management operations should be recorded as separate outcomes.

SHUBHAM DHENGALE - 3574

Experiment No. 9

Title: Develop a Full-Stack MERN Application by Integrating React Frontend with Express Backend using Axios/Fetch API

Aim:

To connect the React spice-export website to Express and MongoDB using Axios so products can be fetched and buyer sample requests submitted through the API.

Tools Used:

MongoDB, Express.js, React, Node.js, Mongoose, Axios, Vite and browser developer tools.

Methodology:

The full-stack workflow assigns the React client to presentation and user interaction, the Express API to HTTP processing, and MongoDB to persistent storage. The frontend API module uses Axios with a configurable base URL and API-mode flag. In demo mode it reads the local catalogue and displays a local sample-confirmation message; in API mode it requests products and posts sample requests. CORS must allow the real frontend origin, and the server must connect to MongoDB before persistent operations can complete. Current browser evidence covers the sample form demo and local API health separately; an end-to-end database transaction is pending configuration.

Theory:

MERN combines MongoDB, Express, React and Node.js. React renders the client, Express exposes routes, Node runs the server and MongoDB stores records.

Axios sends Promise-based HTTP requests and parses JSON responses. CORS response headers allow a browser page at an approved origin to call an API on another origin.

Procedure:

1. Review src/lib/api.js and note the API-enabled flag, base URL and local demo fallback.

2. Configure the backend MongoDB URI and frontend origin in private environment files.

3. Start the Express API and confirm its health endpoint before switching the frontend to API mode.

4. Set the frontend API mode and base URL to the running API, then restart the Vite frontend.

5. Open the products page and inspect the GET /api/products request in the browser Network panel.

6. Complete the sample form with buyer details and submit it through the website.

7. Inspect the POST /api/samples status and returned JSON in the Network panel.

8. Verify the new sample document in the configured database using a safe test request.

9. If API configuration is absent, keep demo mode enabled and label its confirmation as local-only evidence.

10. Review the browser network response and user-facing error state when the API is unavailable.

11. After backend persistence is enabled, submit one test request and confirm it through a read operation before documenting end-to-end success.

Code:

```
const api = axios.create({ baseURL: import.meta.env.VITE_API_BASE_URL || '/api', timeout:8000
});
```

```
export async function getProducts() {
if (!isApiEnabled) return spices;
const { data } = await api.get('/products'); return data;
}
export async function createSampleRequest(values) {
if (!isApiEnabled) return { demo:true };
const { data } = await api.post('/samples', values); return data;
}
```

Output:

Expected browser-to-API flow with the backend configured:

Products page -> GET /api/products -> HTTP 200 with spice records

Sample form -> POST /api/samples -> HTTP 201 with the saved request

With API mode disabled, the frontend uses local catalogue data and does not persist the request.

Figure 9.1: Local sample form confirmation; no API or database submission.

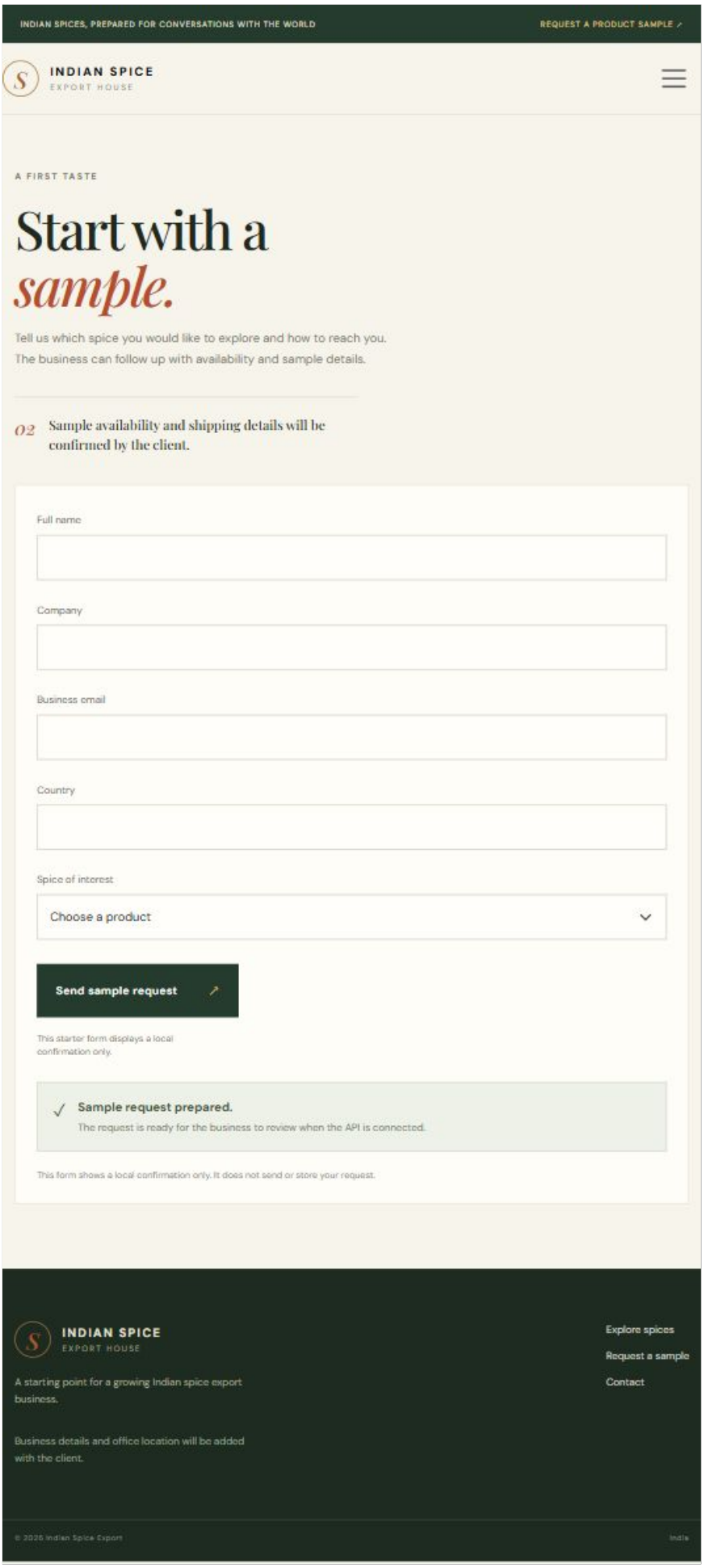


Figure 9.2: Backend health response in the local project.

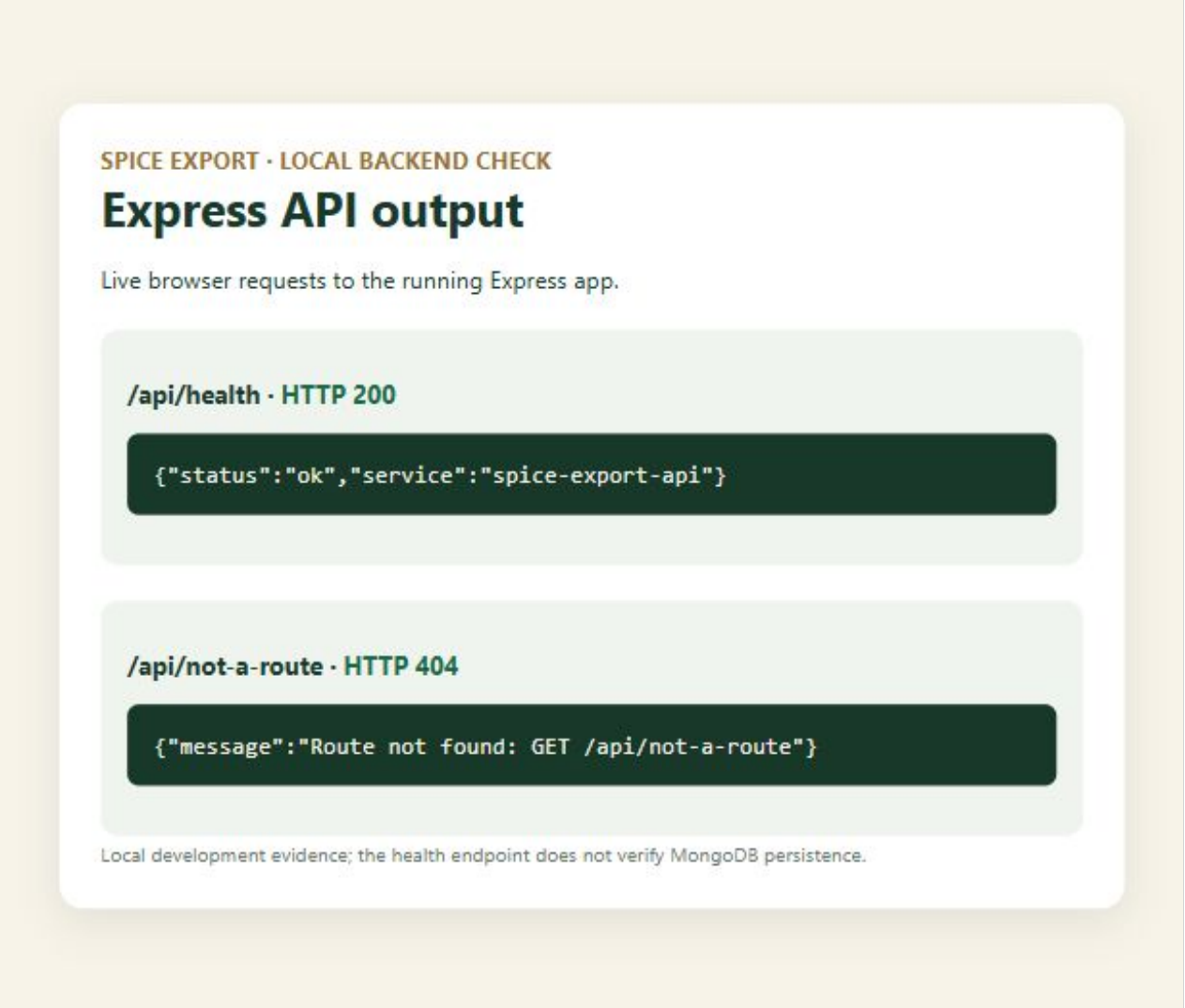


Figure 9.3: Sample form before or during local workflow.

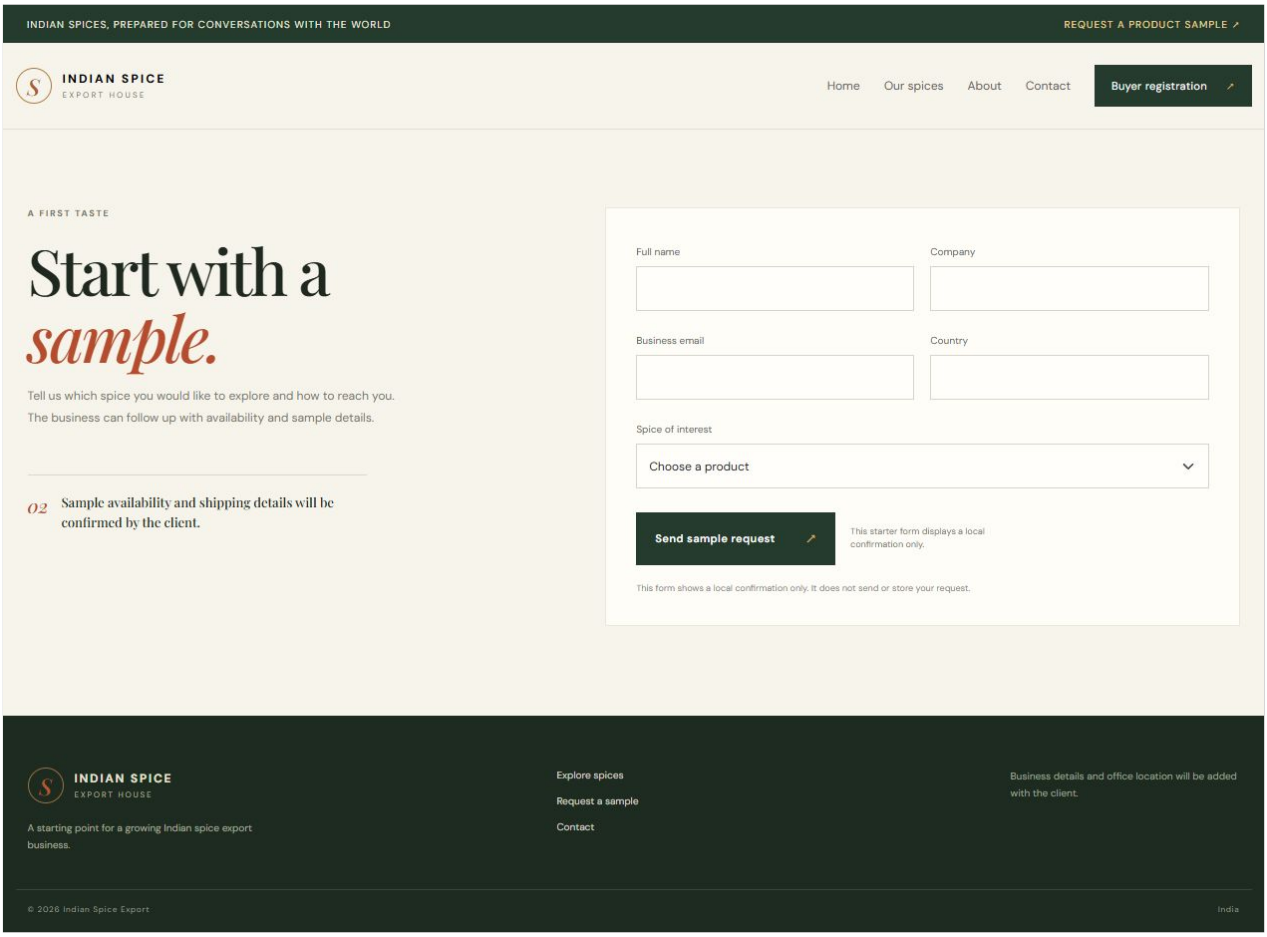


Figure 9.4: Product chosen for sample enquiry.

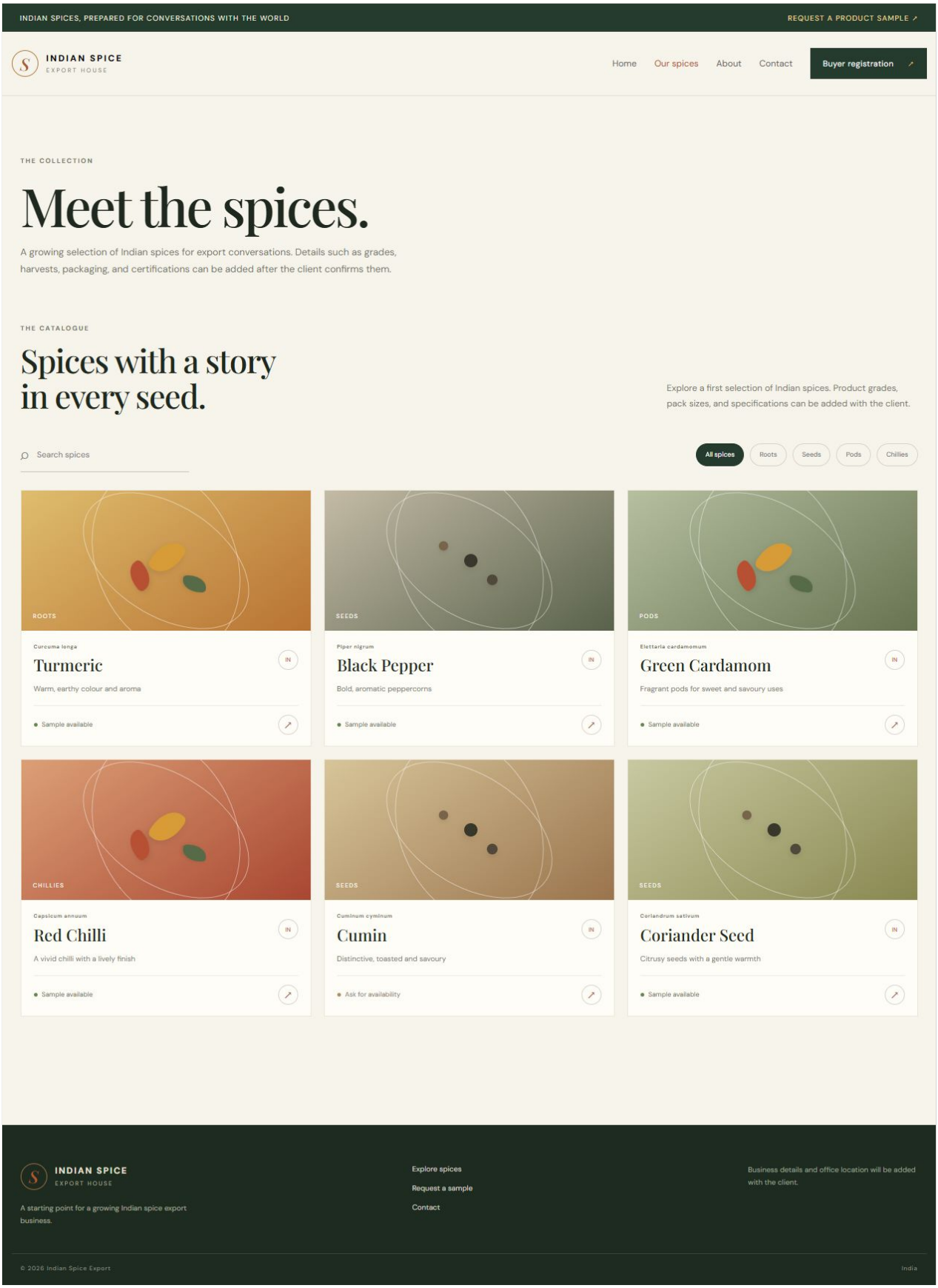


Figure 9.5: Filtered spice selection.

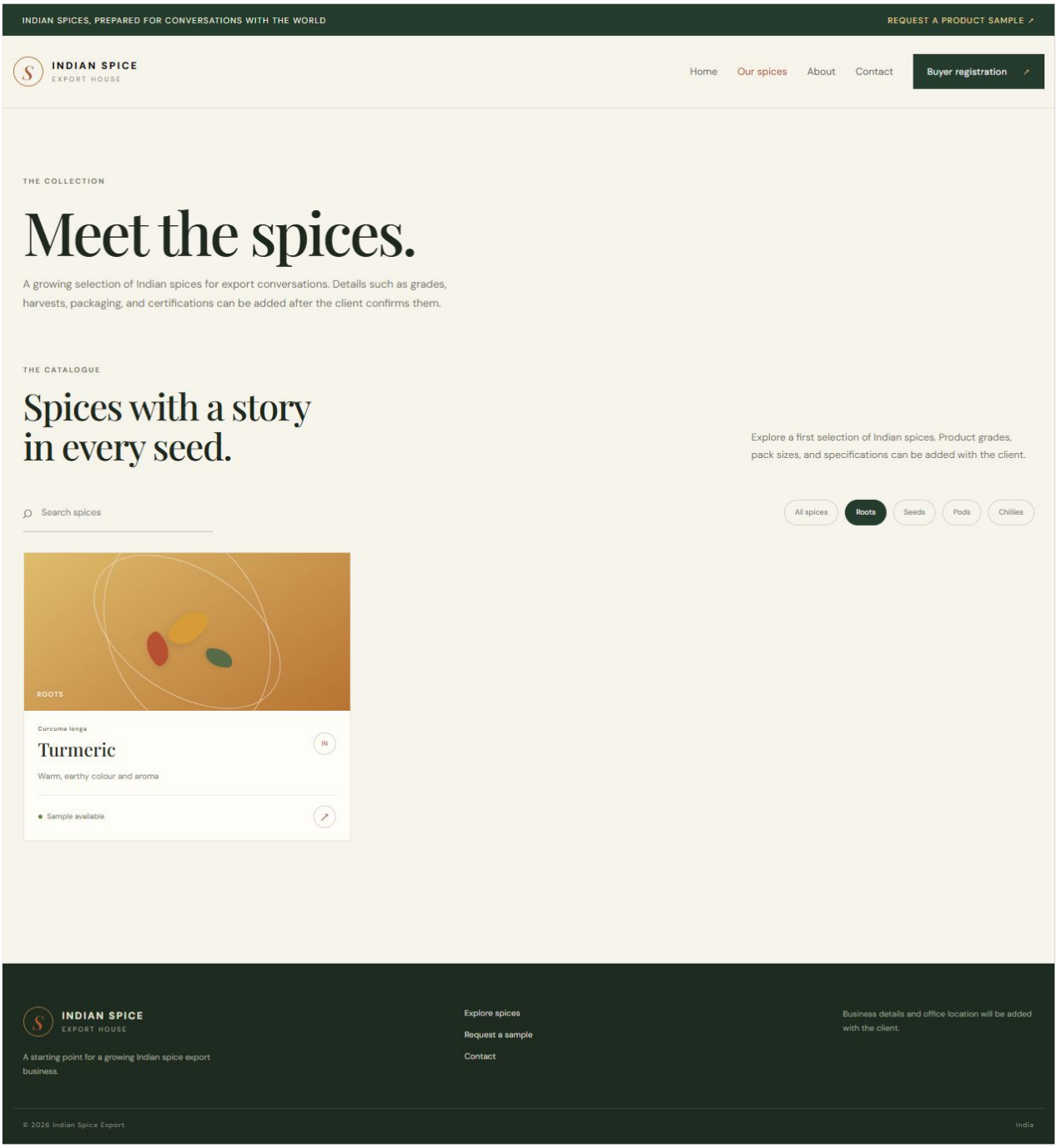


Figure 9.6: Contact route.

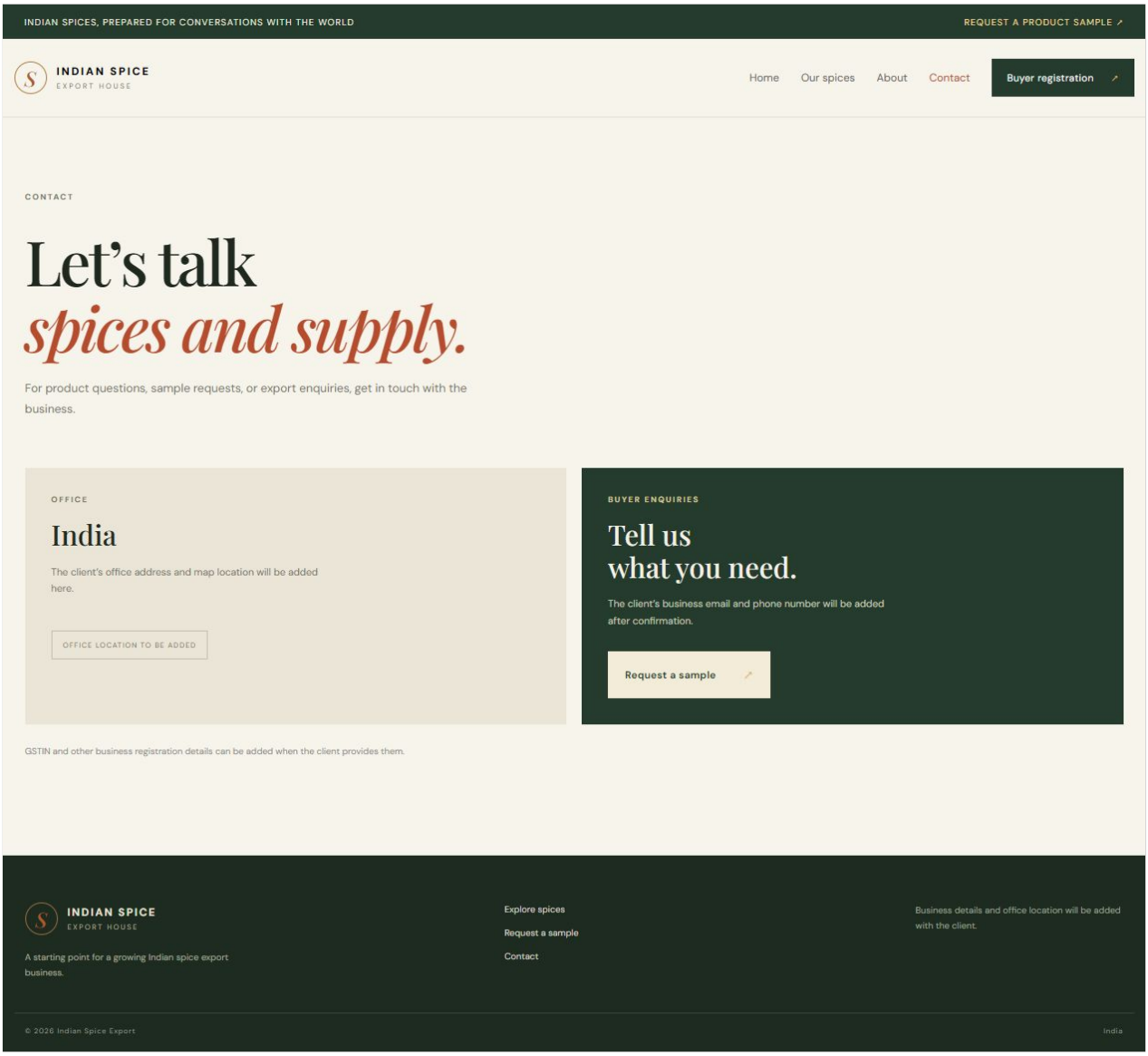


Figure 9.7: Registration route.

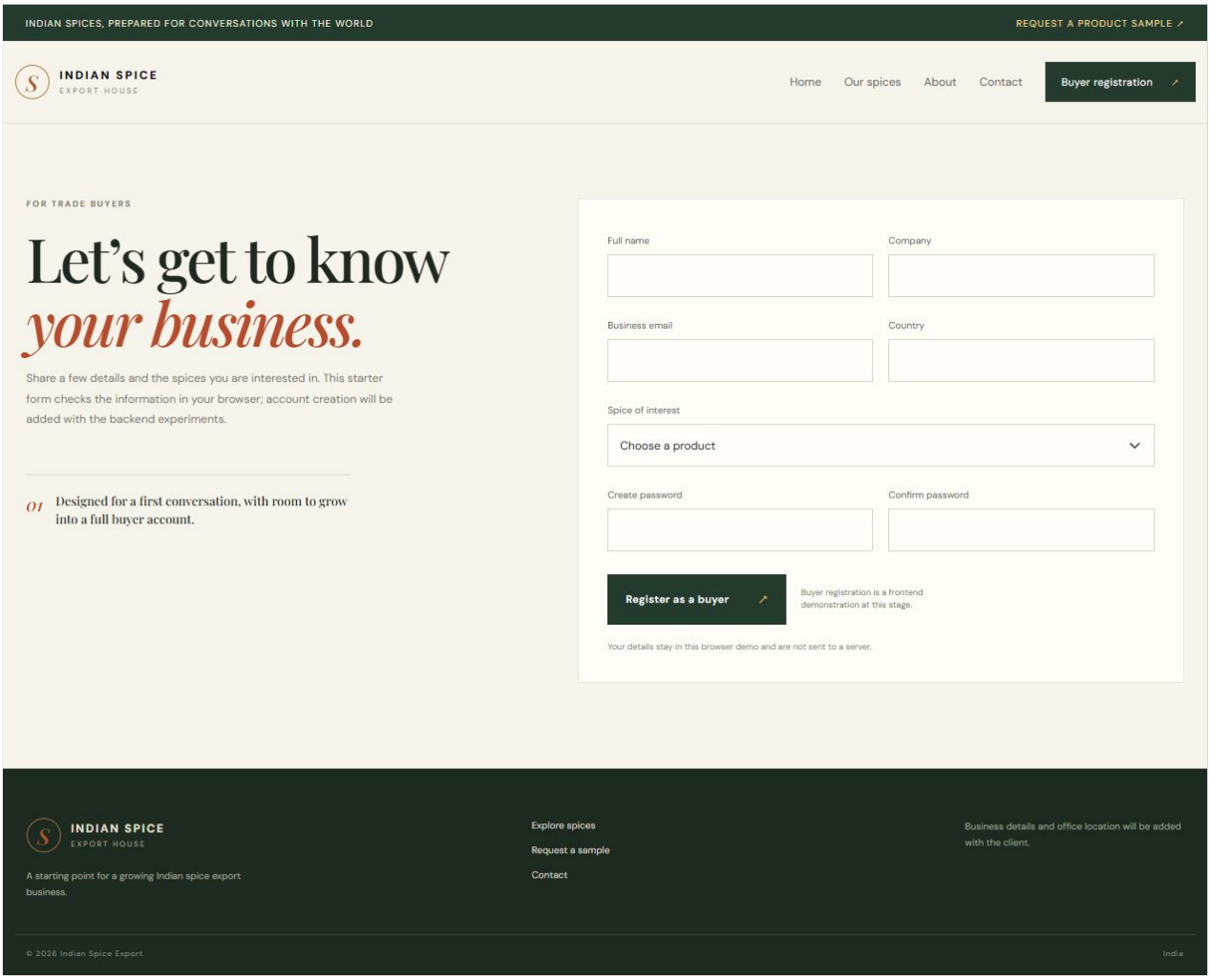
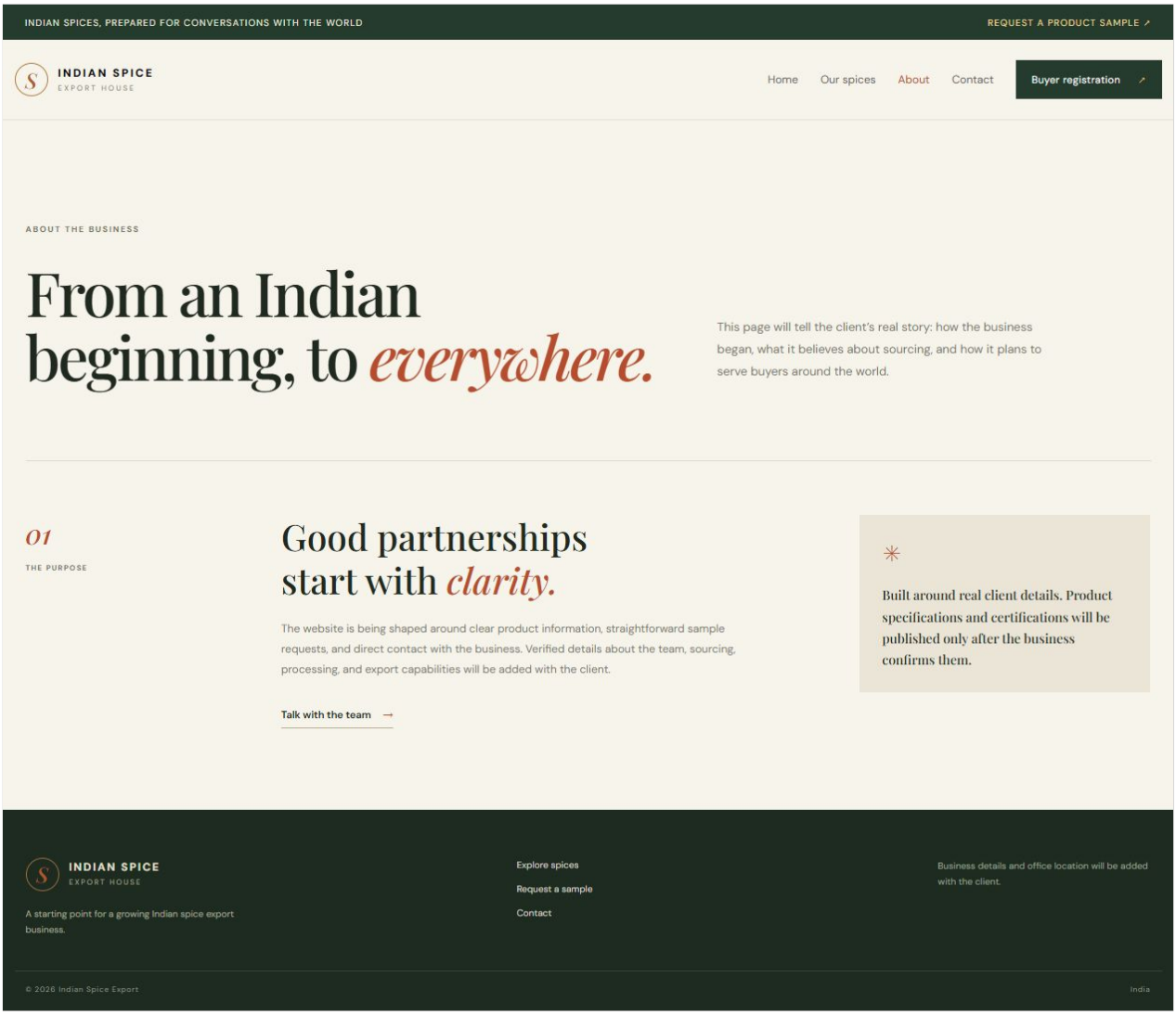


Figure 9.8: About route.



Conclusion:

This experiment defines the integration path between the spice-export React interface and its Express/MongoDB backend through Axios. The frontend module can either return the local catalogue in demo mode or issue requests to the configured API. The sample-request screen demonstrates the buyer journey, while the backend health capture confirms that the local API can respond. Together they show the interface and service as separate working pieces, but they are not an end-to-end persistence test. API mode, CORS and MongoDB credentials must be configured before claiming that product data came from MongoDB or a buyer request was saved. The next verification should capture both browser Network responses and the matching database record.

End-to-end verification requires tracing the same sample request through the form, browser Network panel, Express controller and database record. A confirmation shown only by the local demo branch is not proof of that complete path. Keeping the mode and result visible in the report helps show the work incrementally and makes the final persistent workflow straightforward to verify after the API configuration is supplied.

SHUBHAM DHENGALE - 3574

Experiment No. 10

Title: Deploy a MERN Stack Web Application on a Cloud Platform (Render/Netlify/Vercel) and Demonstrate End-to-End Functionality

Aim:

To deploy the spice-export MERN application with a React frontend, Express API and MongoDB database, then verify a sample request from the website through the backend.

Tools Used:

Git, MongoDB Atlas, Render, Vercel, environment variables, browser and Postman.

Methodology:

The deployment plan separates the static React frontend, Express API and MongoDB database so each service can be configured for production. The repository includes deployment guidance and cloud-platform configuration; private environment variables hold the database URI, allowed client origin and API settings. A complete release check would build the frontend, deploy the API, verify the health URL, open each direct route, submit a sample request and confirm the stored record. The current evidence is from a local website and local API only. No public URL or live cloud deployment is claimed because hosting accounts, client-approved details and production secrets have not yet been configured.

Theory:

Cloud deployment hosts an application for public access. A MERN site may host React separately from the Node/Express API and use managed MongoDB.

Environment variables hold service configuration and secrets outside source code. Production CORS should allow the deployed frontend origin. Verification includes health, direct routes, API requests and database persistence.

Procedure:

1. Confirm the client-approved company name, contact channels, office location and GSTIN before public launch.

2. Create a MongoDB Atlas database and restrict network access and credentials appropriately.

3. Push the reviewed project to the chosen Git repository and deploy the Express API to Render or another supported host.

4. Set MONGODB_URI, CLIENT_ORIGIN and any admin secret privately in the API host dashboard.

5. Deploy the React build to Vercel, Netlify or the selected static host.

6. Set the public API base URL and enable API mode in the frontend deployment settings.

7. Open the production health URL and verify the expected HTTP 200 JSON response.

8. Open each website route directly, then submit a test sample enquiry and inspect the API response.

9. Verify the test document in MongoDB, remove it if appropriate, and capture production screenshots with the real deployed URL.

10. Check the production build and environment-variable settings before selecting a cloud target.

11. Verify the public URL, API connectivity and a complete test request after deployment has actually succeeded.

Code:

```
  Render API environment: MONGODB_URI, CLIENT_ORIGIN, ADMIN_API_KEY
```

Vercel frontend: VITE_API_ENABLED=true

VITE_API_BASE_URL=<deployed API base URL>/api

Health check: GET <deployed API base URL>/api/health

Output:

Expected checks after deployment:

GET /api/health -> HTTP 200 with status ok

Routes /, /products, /about, /register, /samples and /contact load directly.

Submitting the sample form -> HTTP 201; the request is visible in MongoDB.

The project is not deployed yet. Add actual live URLs and screenshots after configuring accounts, client details and secrets.

Figure 10.1: Local website preview; not a cloud deployment.

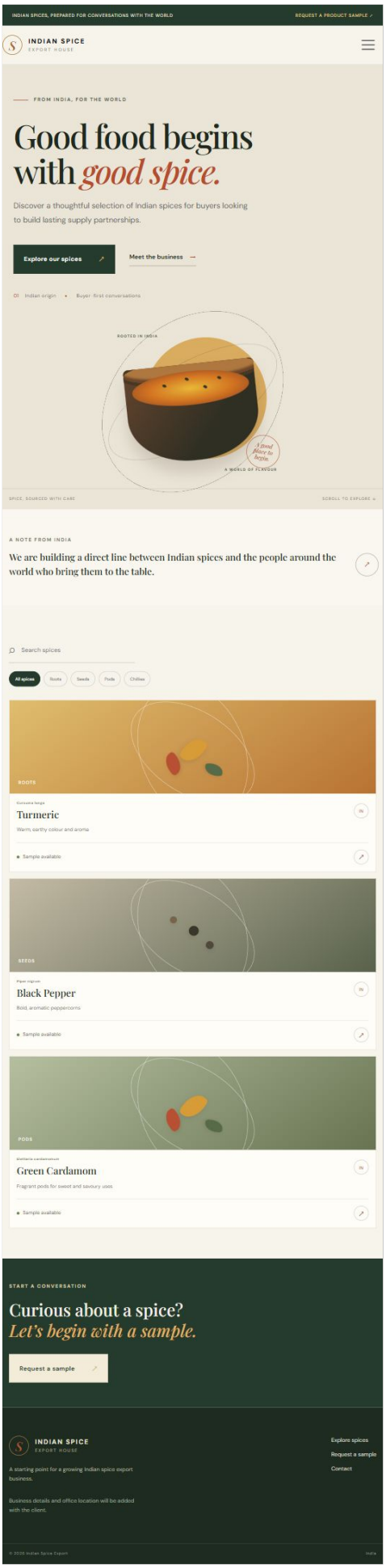


Figure 10.2: Local backend health output; not a public deployment.

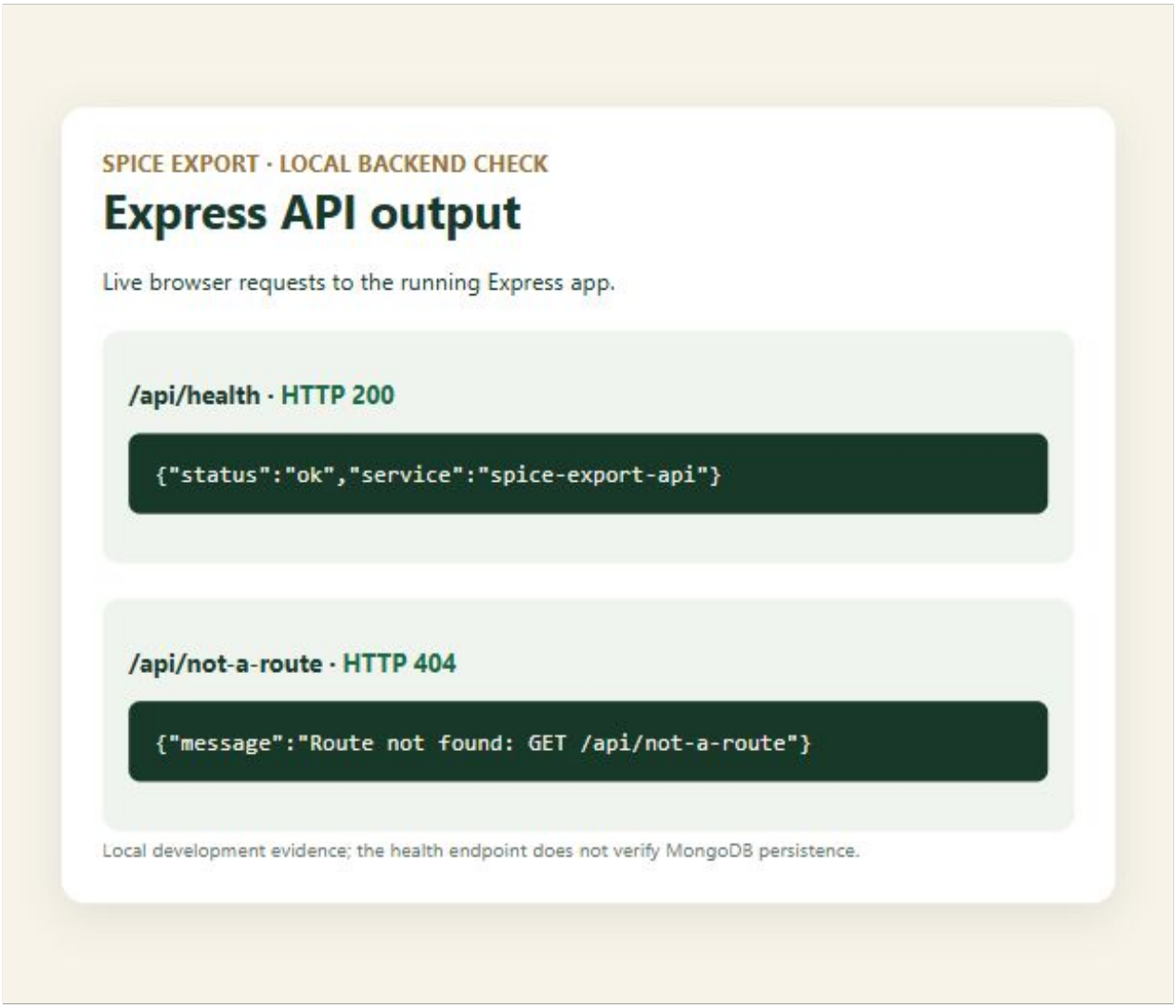


Figure 10.3: Site home page running locally.

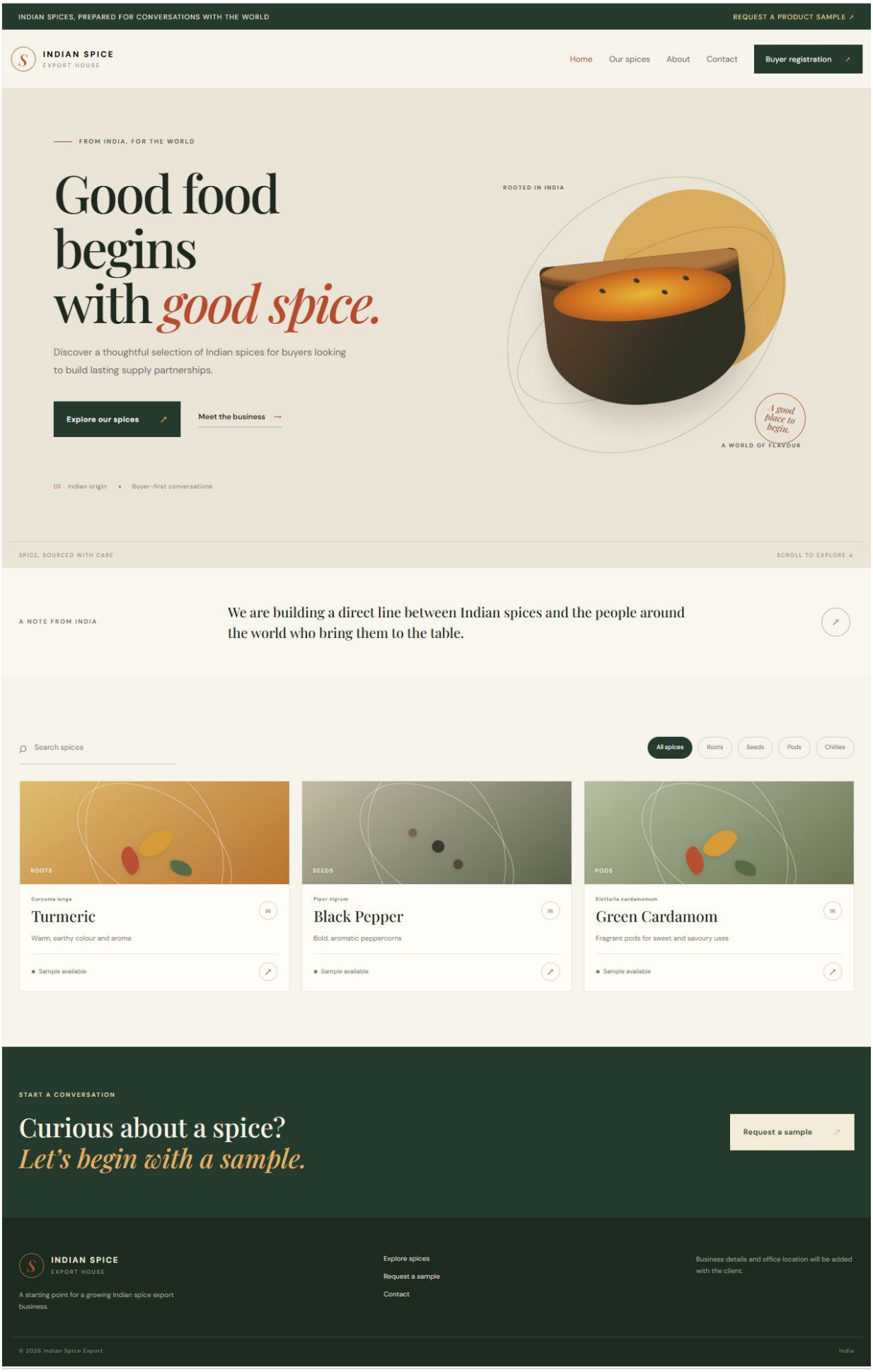


Figure 10.4: Product catalogue on the local build.

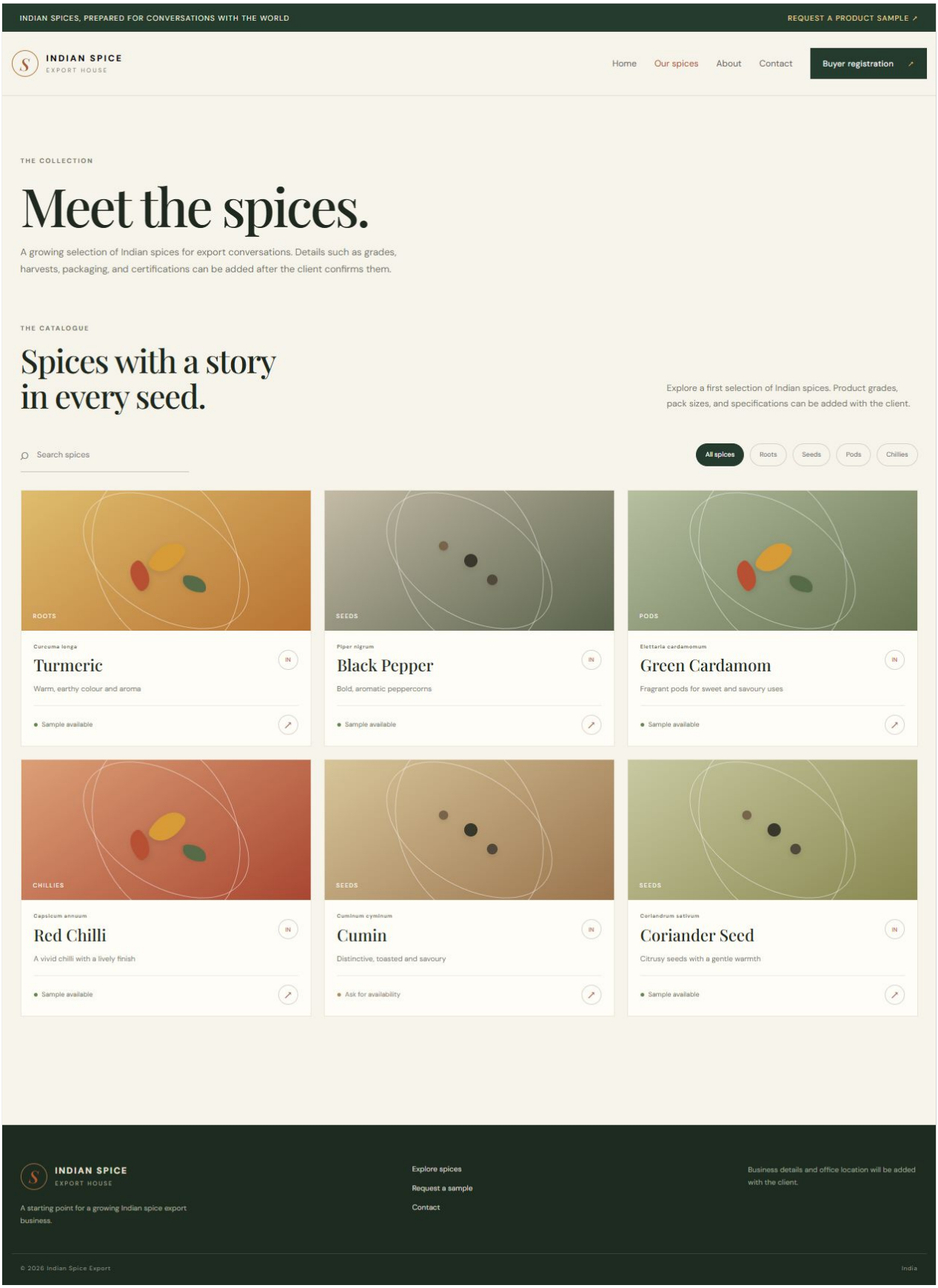


Figure 10.5: About route in the local build.

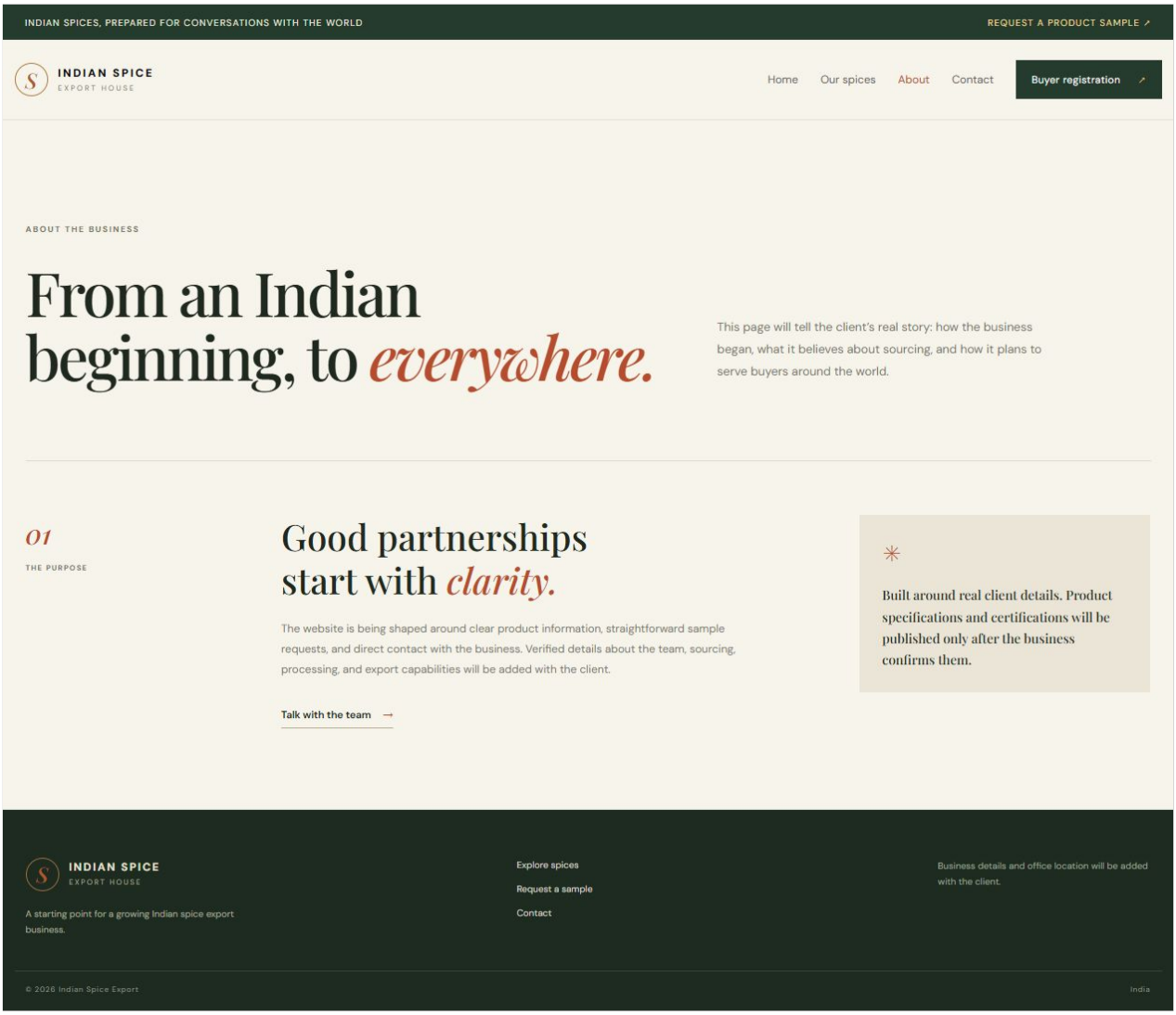


Figure 10.6: Contact route in the local build.

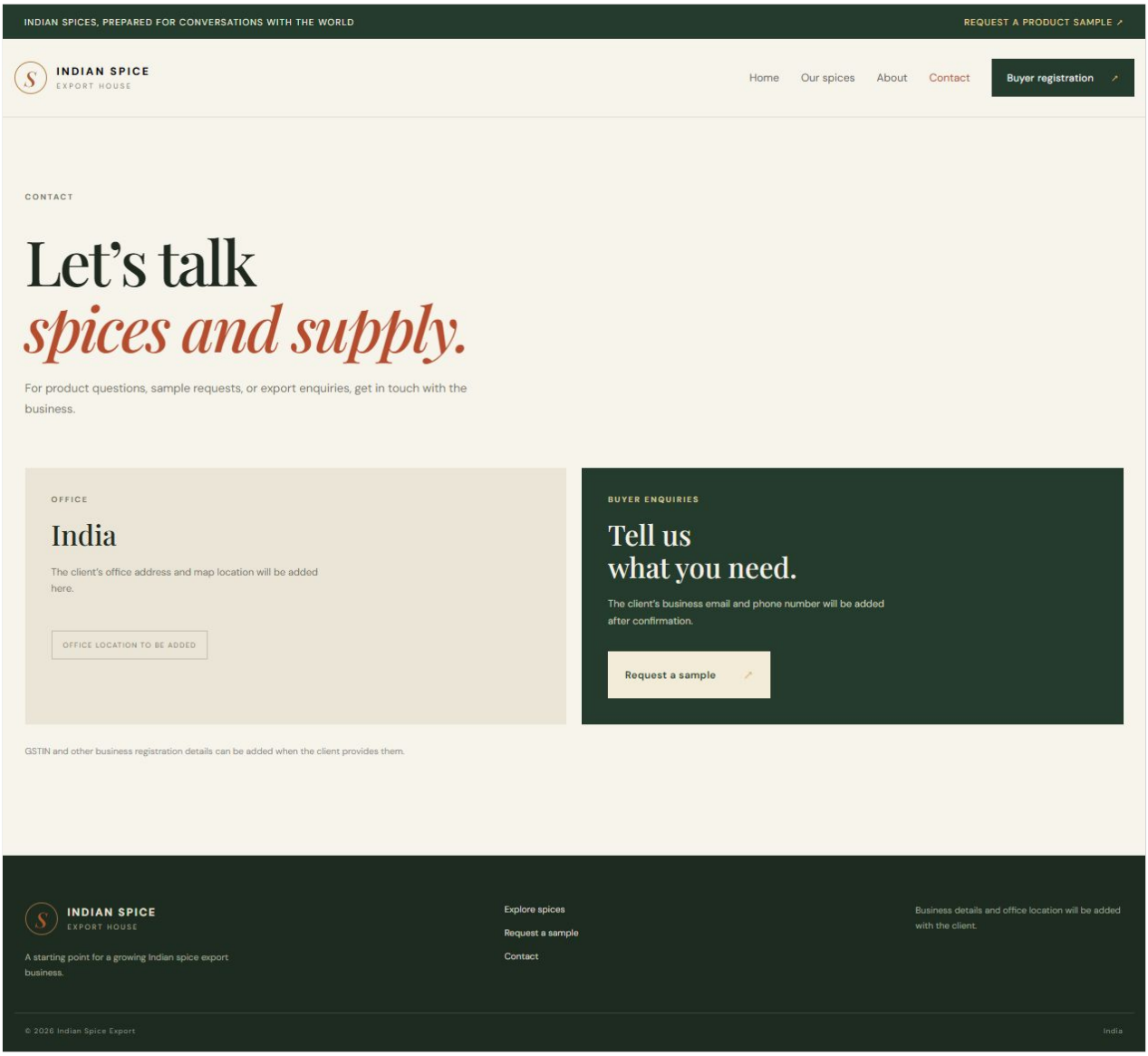


Figure 10.7: Sample route in the local build.

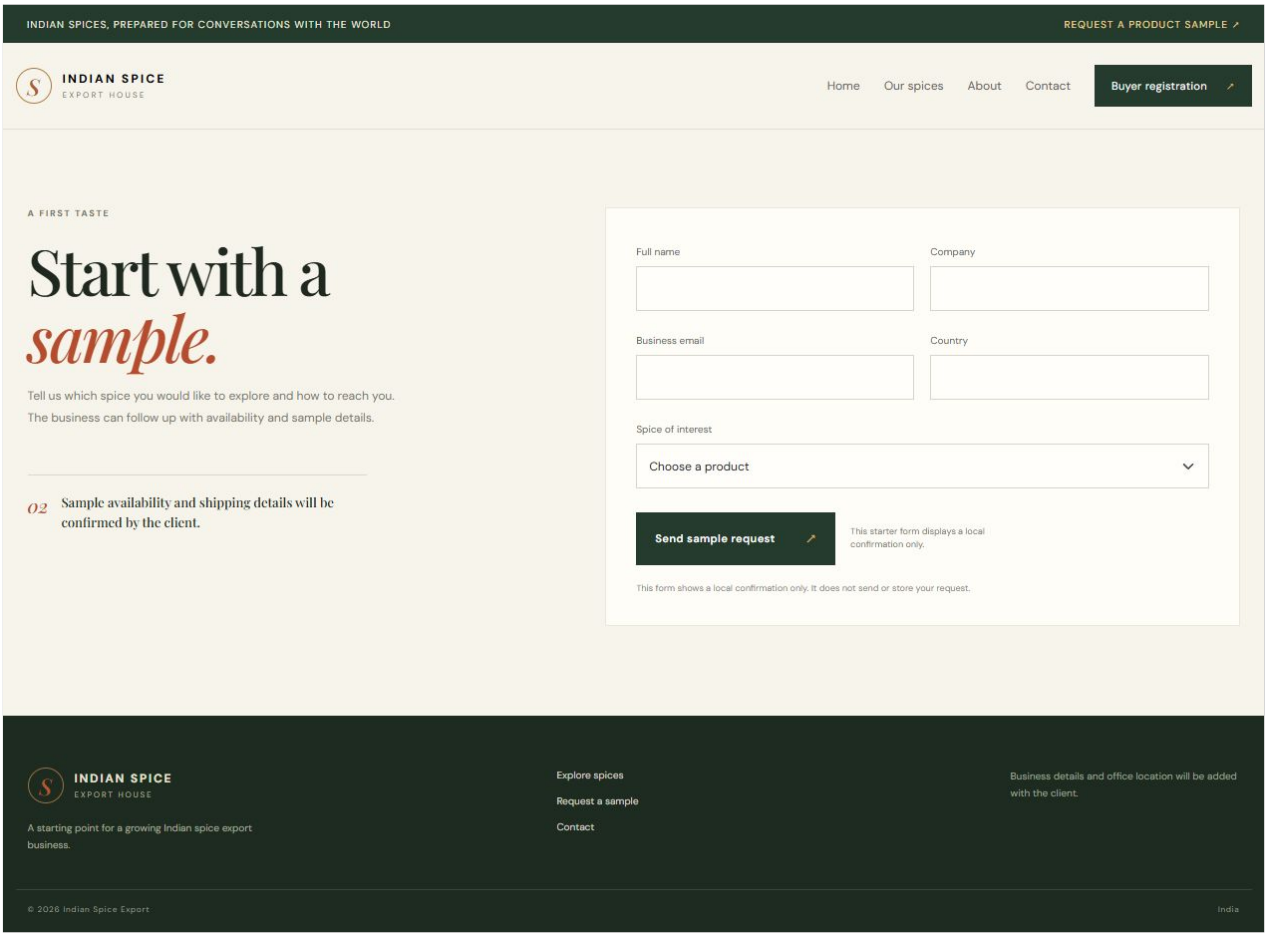
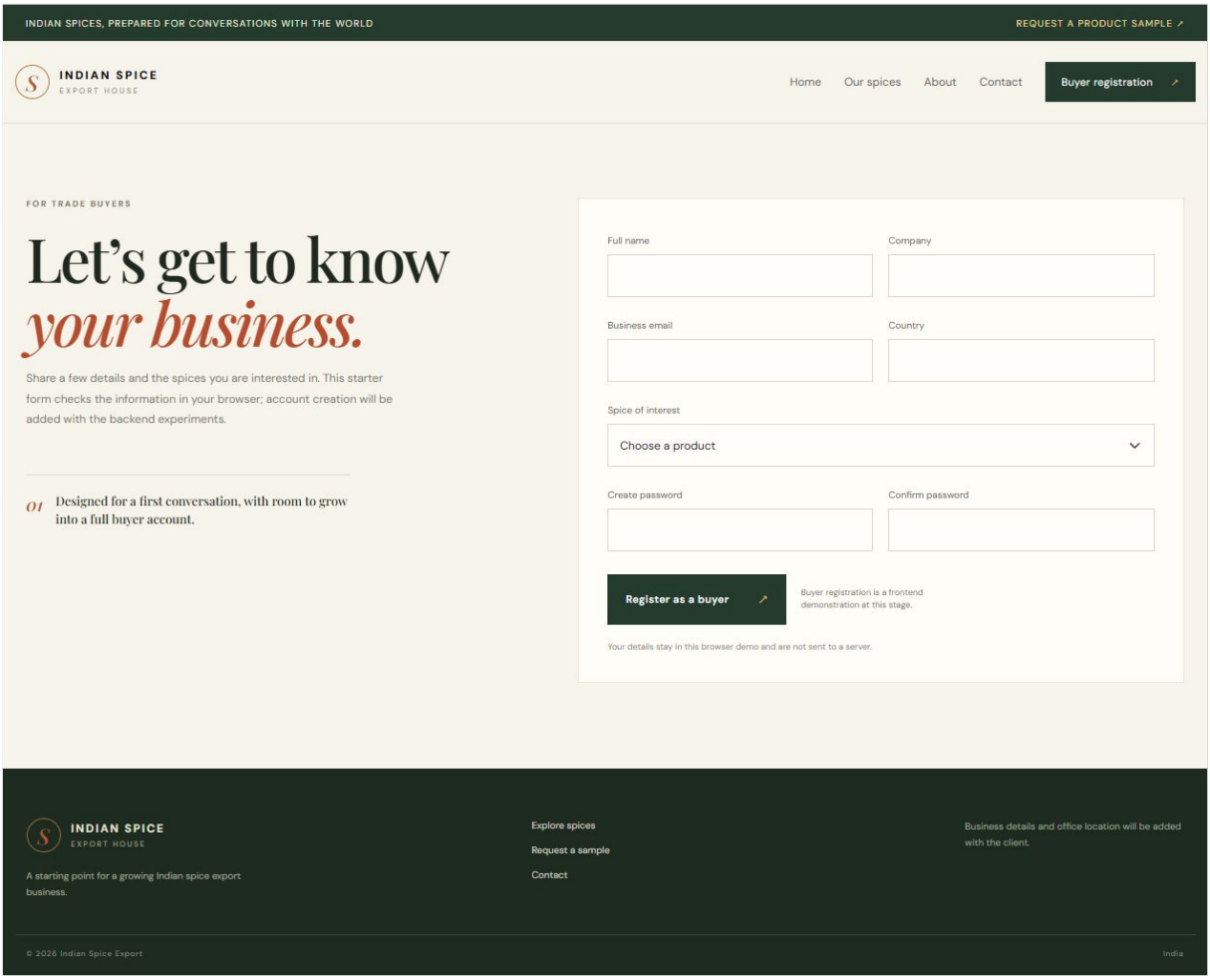


Figure 10.8: Registration route in the local build.



Conclusion:

This experiment prepared the steps needed to publish the spice-export application as a MERN website. The local project has a React frontend, Express service, Mongoose models and deployment notes, and the local site and API can be inspected before release. A successful deployment still depends on client-approved public details, a production database, hosting accounts, private environment variables and a domain or public service URL. The included local screenshots document the current build and backend health response; they are not cloud-hosting evidence. After deployment, the health route, direct website routes, sample-request API response and saved MongoDB record should all be checked from the public environment and recorded as actual production output.

The final report should replace each local-development capture with a production capture only after the public URL has been deployed and verified. A cloud screenshot should show the real host, the API response should come from the deployed endpoint, and the submitted test enquiry should be located in the production test database before it is removed. This preserves a clear distinction between preparation and a completed release.